THIS IS A COMPUTER NETWORKS PRACTICAL COURSE

PROF. SANTORO FEDERICO FAUSTO

UNIVERSITY OF CATANIA
DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

HELLO!

I am Santoro Federico Fausto

I am here because I love divulging what I learnt during my working experience.

You can find me at @fedyfausto

INSTRUCTIONS FOR USE

GOOGLE IS YOUR FRIEND

Before asking for questions, it's a good habit to look for a solution with your search engine (I always do that while looking for the best solution to solve a problem).

STACKOVERFLOW IS NOT ALWAYS THE BEST SOLUTION

Please do not choose the first answer that you find, but try to understand why the solution works.

WORK TOGETHER

Cooperation is vital in this work, it could save you a lot of time sometimes.

PRACTICE, PRACTICE AND PRACTICE

It is a good practice to learn how to use the introduced technologies and to continue practicing outside the course.

DO NOT BE AFRAID TO ASK FOR HELP

1.

INTRODUCTION

NET
WORK
ING?

PROF. SANTORO FEDERICO FAUSTO

WHAT IS A NETWORK?

A Network is two or more devices, connected together, to share resources.

- What is a device?
  Network Printer, PC, Laptop, Television, Smartphone.

- How connected together?
  Via cable, via radiowaves, via coax, via light. Then via a Medium or **Media**.

- What are resources?
  Files, currency, games, movies, songs, web pages, simply information.

HOW DO WE MAKE A NETWORK?

We need to define the elements of a network.

- A network of two devices are made by a **sender** and a **receiver**. From now, these are **nodes**.
- The sender need to send a **message**, carried by a **signal**.
- The signal is sent via a **medium** (or **media**).
- Devices, trying to pass messages, need a set of rule, **a protocol**, to govern the structure of the message (the language of the message).
- Messages can be the **resource** or the **request** to the resource.

Remember, a network could be the connection of two or more devices, then if a device is a sender, then the other ones could be receivers! We need to define a **topology**!

WHAT ARE NODES?

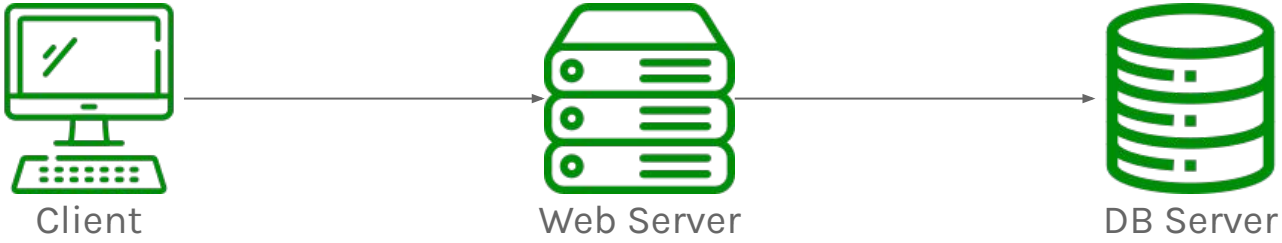
As we said, **Nodes** (or **Hosts**) are any devices which sends or receive traffic. For example, you **computer**, laptop or smartphones are all hosts. Also, **servers** and **cloud resources** are hosts. Moreover, any **Internet of Things** (IoT) devices are host!

If a Host want communicate with the others, must follow the same rules to communicate each other, then the **same protocol**!

Typically, Hosts fall in one of two categories, **clients** and **servers**.

WHAT ARE NODES?

Clients initiate requests, **Servers** respond to them.

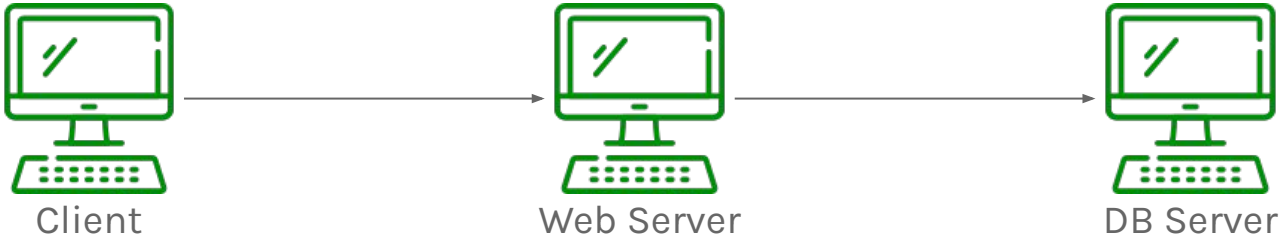


The role of client and server is **relative to specific communication**. For instance, at some point, the web server will send a request to a Database Server to save or retrieve some information.

In this case, the Web Server is the client and the database is the resource of the Database Server.

WHAT ARE NODES?

Then servers are simply computers with some software which responds to specific requests.



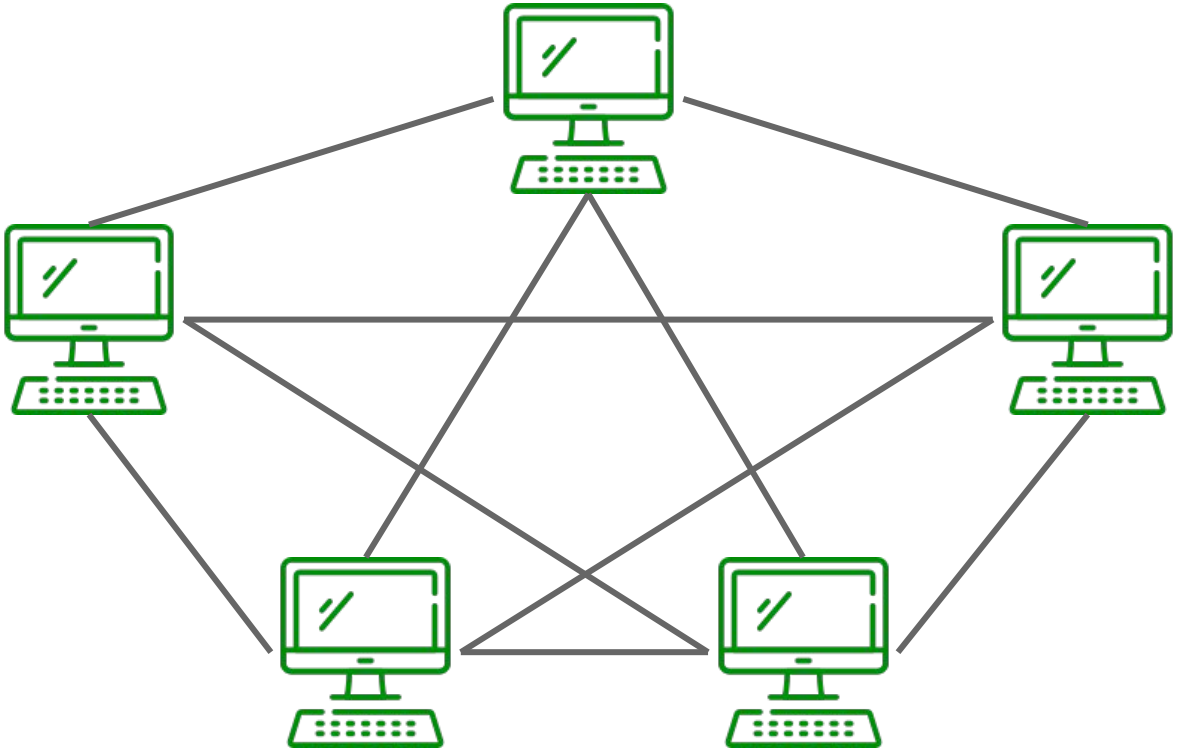
But how can we connect these hosts together?

HOW CONNECT NODES?

A network can be created anytime when two nodes are connected, for instance, using a cable.

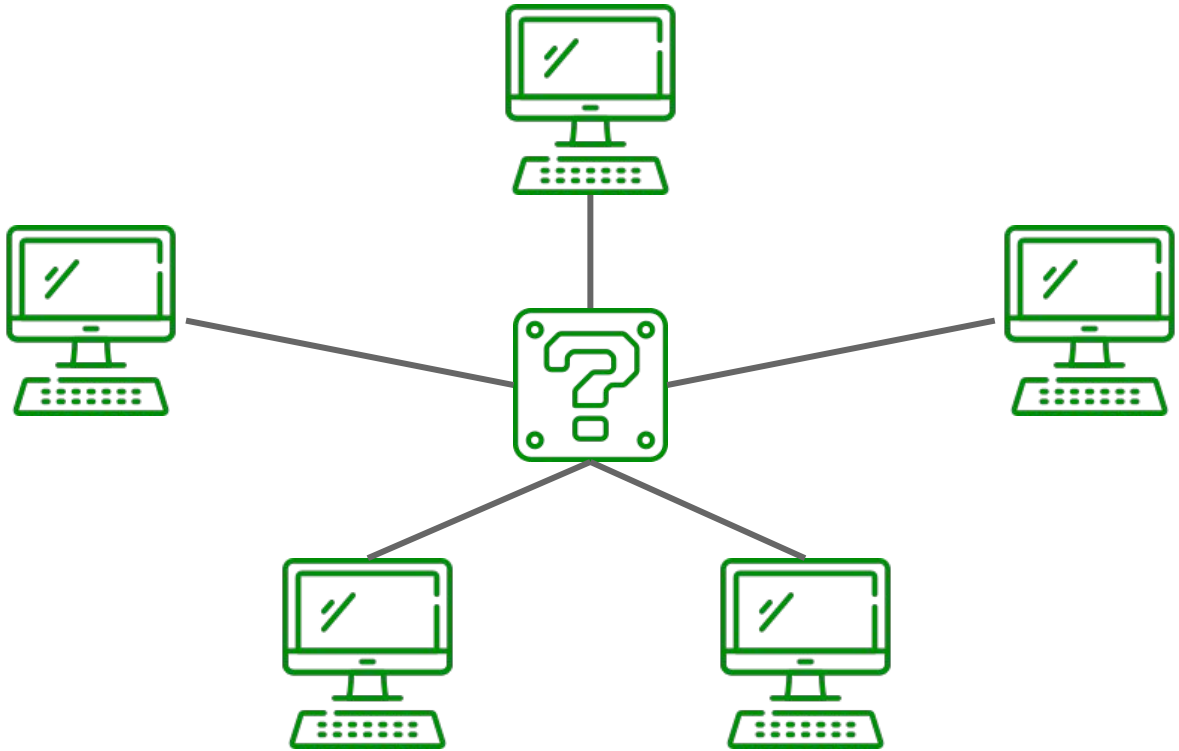
This solution works but a network is composed by various number of Nodes connected together. The problem here are the **Network Interface Controllers** (or **NIC**s) available in the Nodes. In fact, a cable can be used only in one NIC of a node. These could be called **Ports** or Network Ports.

HOW CONNECT NODES?



Connecting Nodes directly to each other simply does not scale!

HOW CONNECT NODES?

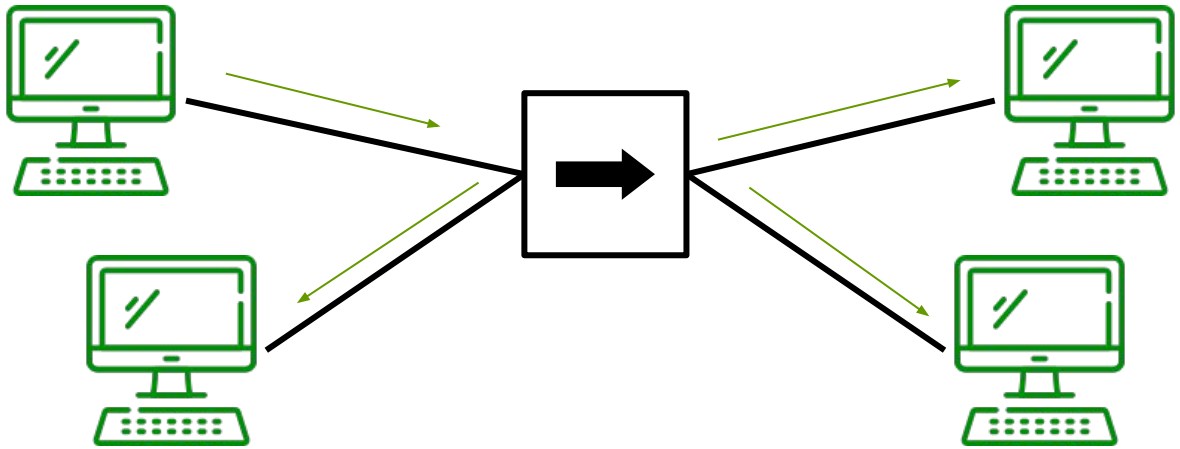


We can use devices which we could put at the center of every network and connect all the hosts to those devices!

HOW CONNECT NODES?

A **Hub** is nothing more than a multiport repeater. Hubs regenerate signal as a Repeater, except they do it across multiple ports.

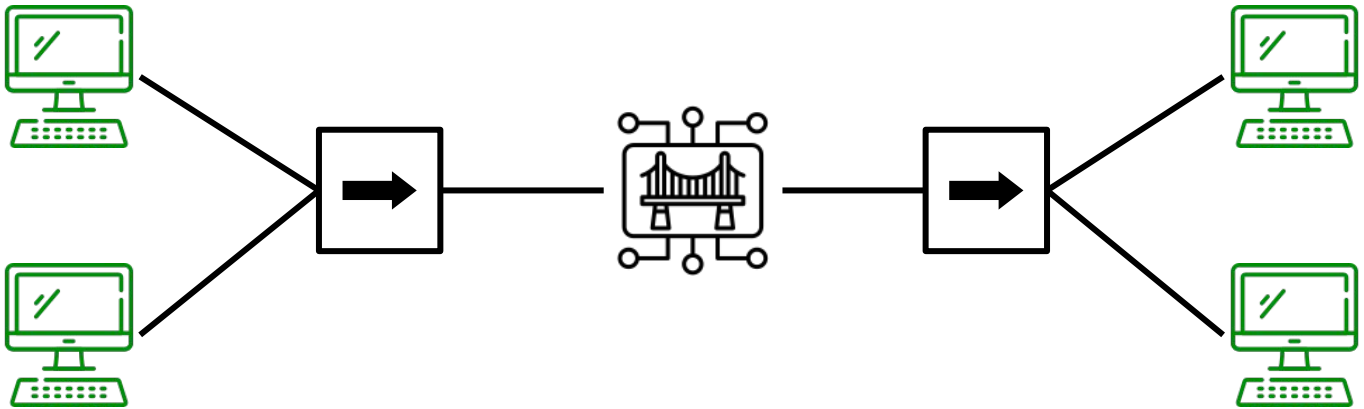
For instance, if a node need to communicate, this node send a message to the Hug which will simply duplicate that message and send it out to all remaining ports.



HOW CONNECT NODES?

A **Bridge** is meant to sit between hub connected nodes. By definition, a Bridge is a set of only two Network Port that connect the two Hubs (physically).

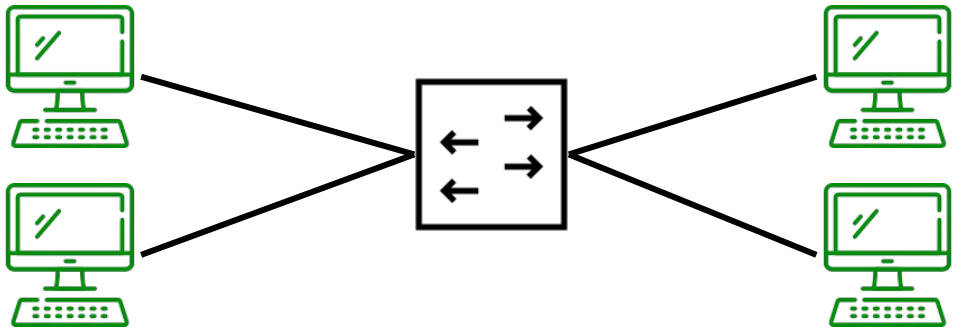
Bridge will learn which hosts are on which side of the bridge, this allow the bridge to **contain communication** to only the side that is necessary!



HOW CONNECT NODES?

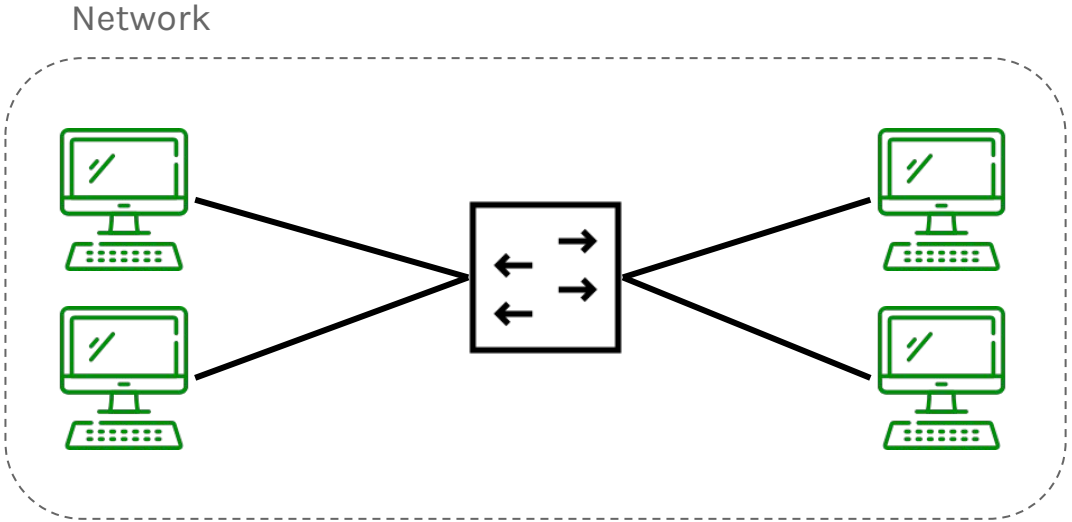
A **Switch** is sort of combination of Hubs and Bridge (and this will be useful in the future). Like Hubs, many nodes can be connected to the Switch and It can learn which node are connected to each port. This means that the messages are not duplicated but **only send to the correct port**.

Then a Switch is a device which facilitates communication **within** a network.



HOW CONNECT NETWORKS?

Then a **Network** is a logical grouping of Nodes which require similar connectivity, which means all of these devices belong to the same network!

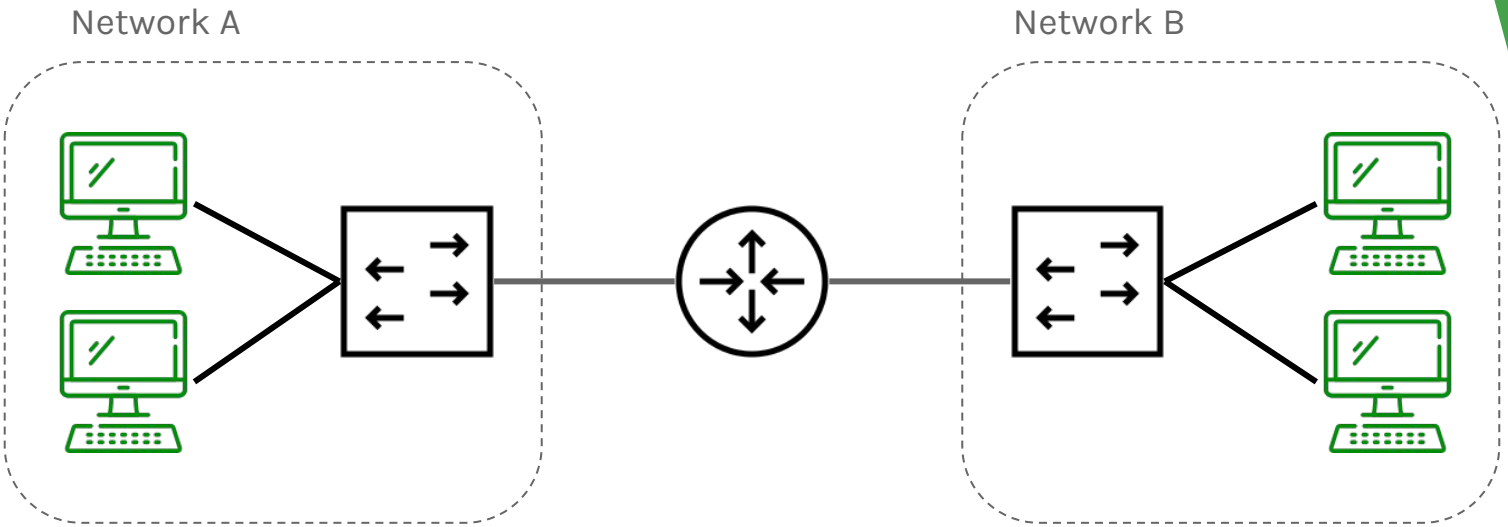


Then if we have multiple network, how could we connect those?

HOW CONNECT NETWORKS?

A **Router** is a device whose primary purpose is facilitate communication **between** networks.

Routers provide traffic control points between networks, then security policies, traffic filtering or even redirecting traffic elsewhere. They learn which networks that they are attached to. From perspective of Network nodes, the router acts as **Gateway** to the other networks.

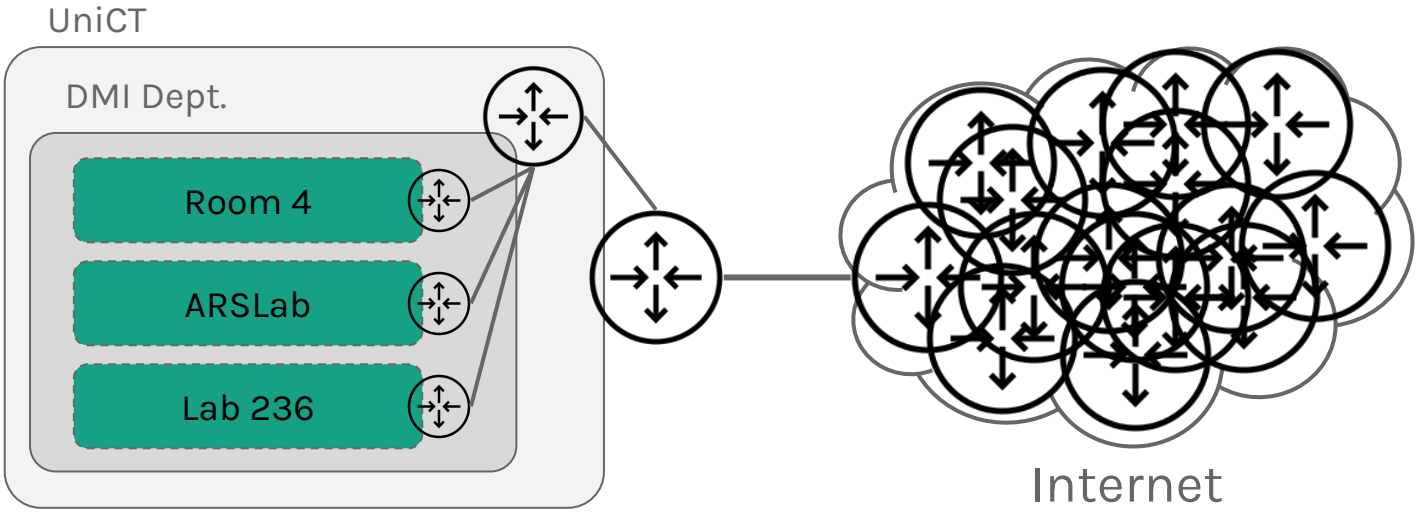


HOW CONNECT NETWORKS?

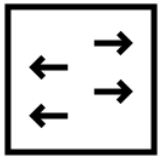
Routers create the hierarchy in network and the entire **Internet**.

Each network can use its Gateway to communicate to another network.

All the top routers are connected to the Internet, which is nothing more than a bunch of different routers itself.



THE ROLES OF DEVICES



Switching is the process of moving data **within** networks.
A Switch is a device whose primary purpose is Switching.

Routing is the process of moving data **between** networks.
A Router is a device whose primary purpose is Routing.

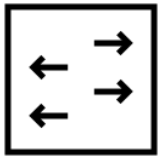
There are many other types of network devices, such as Access Points, Firewalls, Load Balancers, Proxies but all of these devices are going to perform Routing, Switching or both!

EXERCISE TIME!

DESIGN A NETWORK

Try to design a network topology as follow:

- There are a Home Network
- There are 5 devices connected to the network
- The Home Network is connected to the City Network
- In the City Network there are other 2 Home Network
- Via a Gateway, the City Network is connected to the Internet
- At some point, a YouTube Network is connected to the Internet
- Inside the YouTube Network, there are 2 nodes, the Video Streaming Server and the Database Server.



RECAP!

- We defined what is a network
- We introduced the elements of a network
- Now we know how connect devices together!
- Now we know the existence of basic network devices
- Then, how the Internet really works!
- But how those devices really work?
- What is happening inside a node?
- How communicate using a set of rules? Then a **protocol**?

2.

NETWORK STACK

Network is magic!

PROF. SANTORO FEDERICO FAUSTO

HOW COMMUNICATE?

Until now we talk together in some specific language.

This is a big **assumption made by a human**, such as “I know that everybody in the classroom talk in Italian or English Language”

Another example is when you will go to another country, where you will assume that in that country everybody talk a specific language.

Then, you are going to make assumptions, but **a computer can not do that**! For them, we need to define a communication **model**.

THE MAIN GOAL

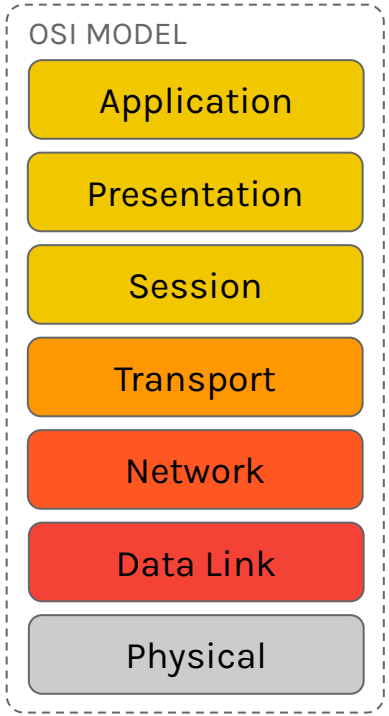
Remember, our main goal is to allow two nodes to share data with one another.

To do this, nodes must follow a set of rules. This is no different than any human language.

The rules to communicate, then for networking, are divided into seven different layers, known as **OSI MODEL**.

Then from this point, we are going inside of a network node.

THE OSI MODEL



This was defined by **ISO** (International Standards Organisation). The base idea is that we can foster competition but yet still the **compatibility**.

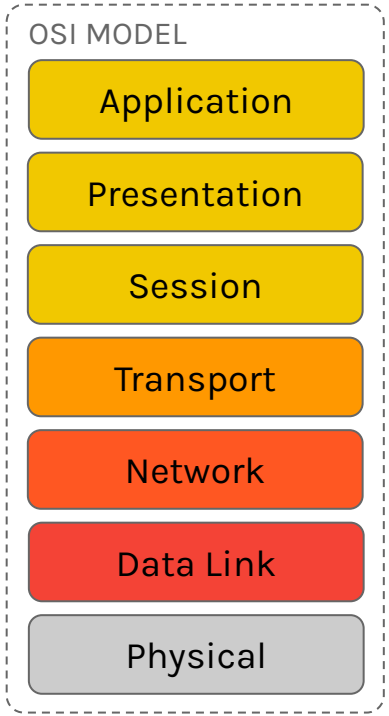
Then this is the difference between proprietary protocols and **open protocols**. In term of proprietary protocol, a private corporation owns them and they are the only ones allowed to change them. In addition, if you want use them you need a license.

Open protocols are something where we are going to create **standards** and you are allowed to look at them, modify them and, in some cases, create a **variation** of them! Then we are talking about a open platform! (you can go to IEEE site and read the RFC document relativ a specific protocol, in free way).

Then the OSI model was created for foster competition but it also helps to delegate **responsibilities**.

It is use to understand the communication process, from the sender to the receiver.

THE OSI MODEL

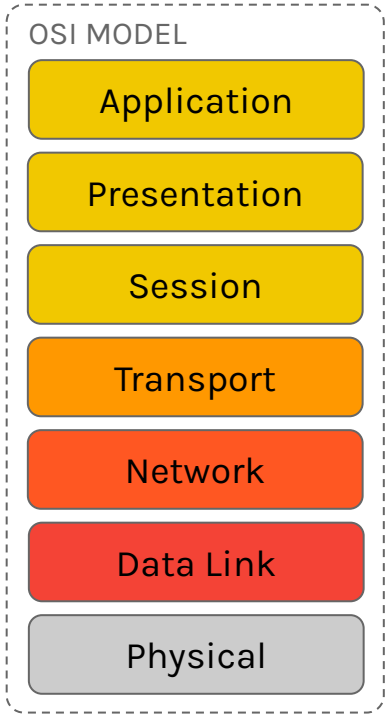


- **Application (Layer 7)**
  Typically where software developers works, where applications are made. Applications give to our node some purposes. This level acts as **Interface** to the network, if the application uses the network connection. An example is the development of a Web Browser (not Web Page!) that must uses a standard protocol to understands the entire process, in this case is HTTP Protocol.
  Their responsibility is to work with the user to prepare the data to be ready for the network, going down.

- **Presentation (Layer 6)**
  Its responsibility is to encode or decoding the data from or to Application, using ASCII, UTF-8, SSL, TSL. This layer is responsible of **compression** of the data.

- **Session (Layer 5)**
  Its responsibility is just going to be management dialogue to keep connections alive.

THE OSI MODEL

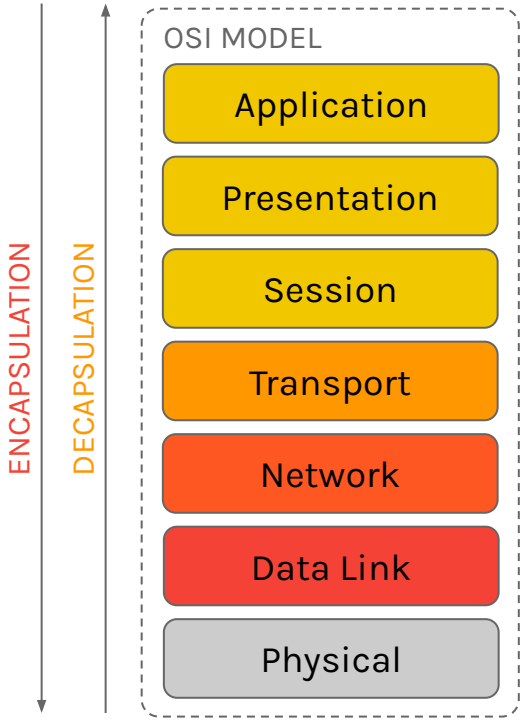


- **Transport (Layer 4)**
  Its responsibility is to get data to the right application or service (then do not confuse with transport name). Remember a service is an application but designed to run in the background (in linux are called **demons**). These are listening and you can have a lot of these application, then this layer is responsible to deliver the data to the right application (such as software delivery). In addition, it ensure to have a reliable delivery and the reconstruction of the data received and security, in terms of un-knowledge of the original data (sent or received).

- **Network (Layer 3)**
  Where data are being distributed across the network, its responsibility is to route these messages along their way, then identify the sender and the receiver.

- **Data Link (Layer 2)**
  Its responsibility is to connect two nodes together, bringing the binary data to the physical device, in fact, this layer has two sublayers, the **logical layer** and the **MAC layer** (media access control layer)

THE OSI MODEL



Remember, the OSI Model is more a guideline to help us delegate responsibilities and to understand what the communication process entails.

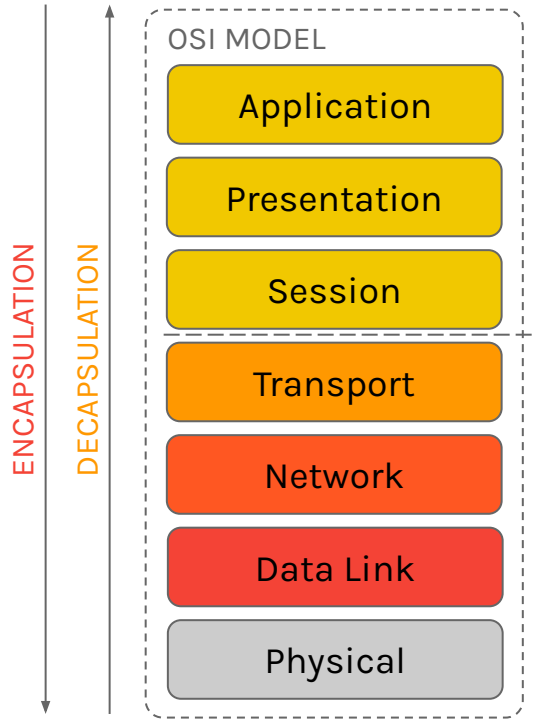
As the nodes in a network communicate each other, the layers need to communicate to the top or bottom layer and vice versa.

Going down the stack is what we call **Encapsulation** and going up the stack we call **Decapsulation**. This process is very critical to the communication to get the message from the sender to the receiver.

Remember, we are talking how communication works inside a node.

Then, each layer creates a **Protocol Data Unit (PDU)** that represents the message from a layer to another one that had a **Header**.

THE STACK COMMUNICATION



Layer 7 to 5 can be grouped in, in fact the sole responsibility for the three of these is to produce what for the Transport Layer. The PDU is called **Data or Message** and is a binary format. Depends on presentation, these binary can be really trivial.

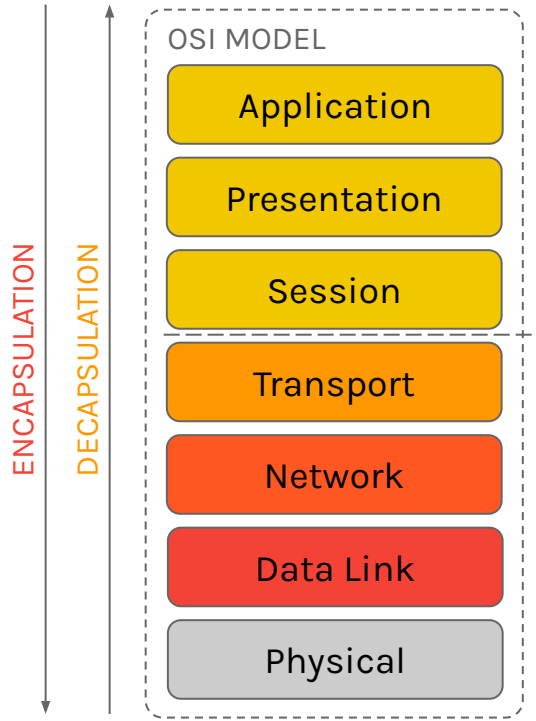
PDU	Addressing
DATA	File Extension
01100100 01100001	.EXE, .JPEG, .PNG

Typically, to identify the binary by a file extension or magic number.

But we also have identifications used in networking, sometimes we call these **addresses**, to identify the application of the service.

This is the first thing that the Transport Layer is going to do.

THE STACK COMMUNICATION



The **Transport Layer** is going to identify what application making the request and what service is going to receive that. To identify them is through **Port Addresses.**

The PDU is called **Segment**, because takes the Data PDU from application and breaks it up into pieces, called Segments.

The Data is segmented for security, performance and allow multiple communication to occur, called **Multiplexing**.

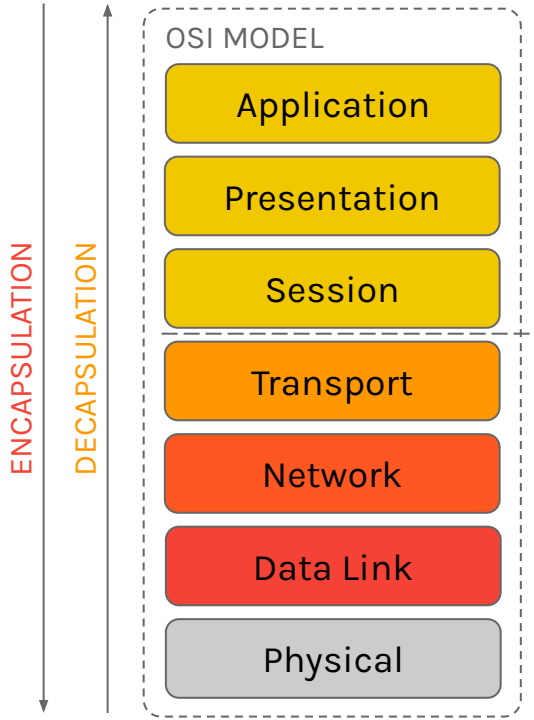
PDU	Addressing
Segment	Ports
Source, Destination, Data Segment	80, 443, 8080

One of the protocol that tells how produce the PDU is called **TCP** which is time relevant, sacrificing time over reliability. Another protocol is **UDP** with a lower overhead.

The **destination port** is defined by the service, but the **source port** is assigned by the client and the O.S. randomly.

The segments are going to be traveling all over the network or Internet, using different pathways, we need some kind of **mechanism** that when that segment gets to it, it knows what pathway to send it to destination.

THE STACK COMMUNICATION



The **Network layer**, when those segments are being distributed across the network, this layer has got to be used by a particular device to **route** it along its way.

The PDU is called **Packet** and the most popular protocol on this layer that tells the O.S. how to take a segment and make it into a packet is called **Internet Protocol (IP)**.

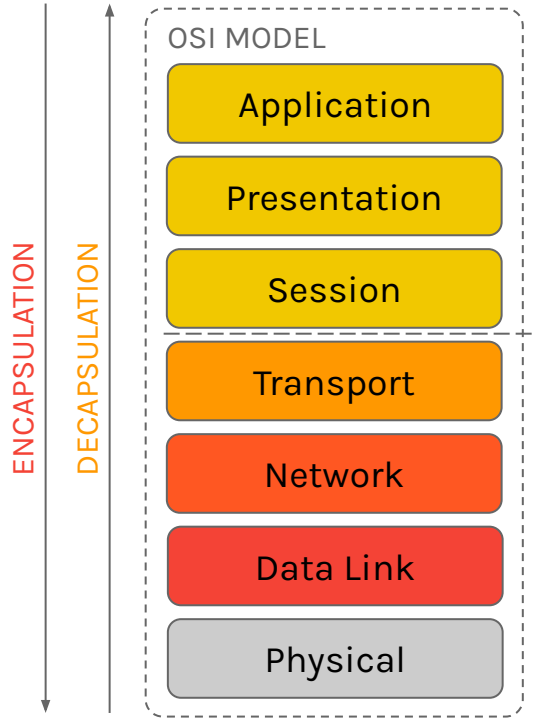
Also there we have a Source and Destination **IP addresses** that are used to identify the **Devices** on the Network.

PDU	Addressing
Packet	IPs
Source, Destination, Segment	192.168.1.10

Then the IP addresses are used to identify devices or specialised devices in a **logical way**.

The IP protocol is not the only one, we have also Appletalk, Net Value, IPX, but the IP is the most popular because is an open free protocol.

THE STACK COMMUNICATION



Everything up to this point we have only logical **data**, then a set of bits that compose some **data**.

The **Data Link layer** is called in this way because it is taking **logical data** and linking it to the Physical Layer, then to something real. **It is a Hop to Hop delivery Layer**.

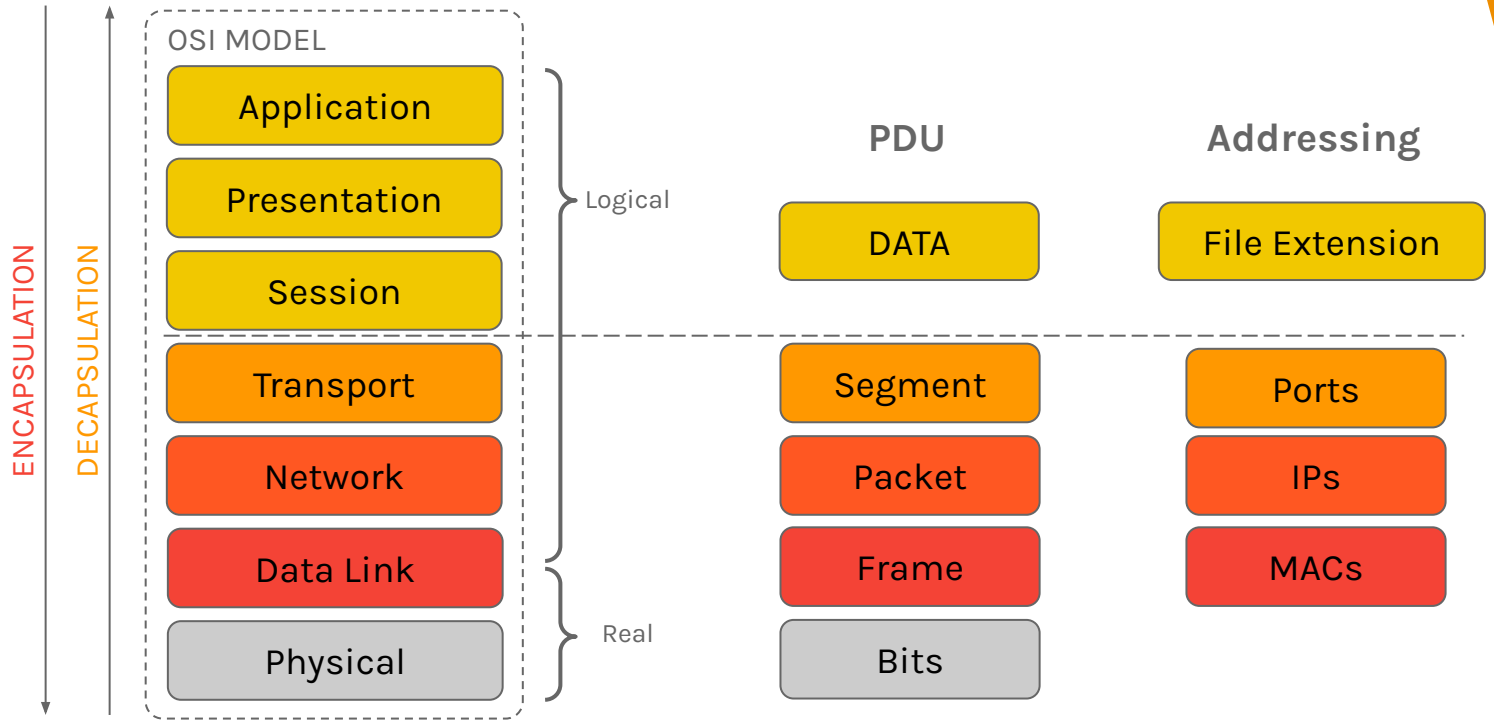
This Layer is the last layer of the encapsulation process, in fact the PDU is called **Frame**, composed by a classic Header and an additional part called **Trailer**.

How translate the virtual things to real things depends on to Network Interface Controller (NIC), using Standard Protocols, such as **Ethernet** or Wi-Fi protocols.

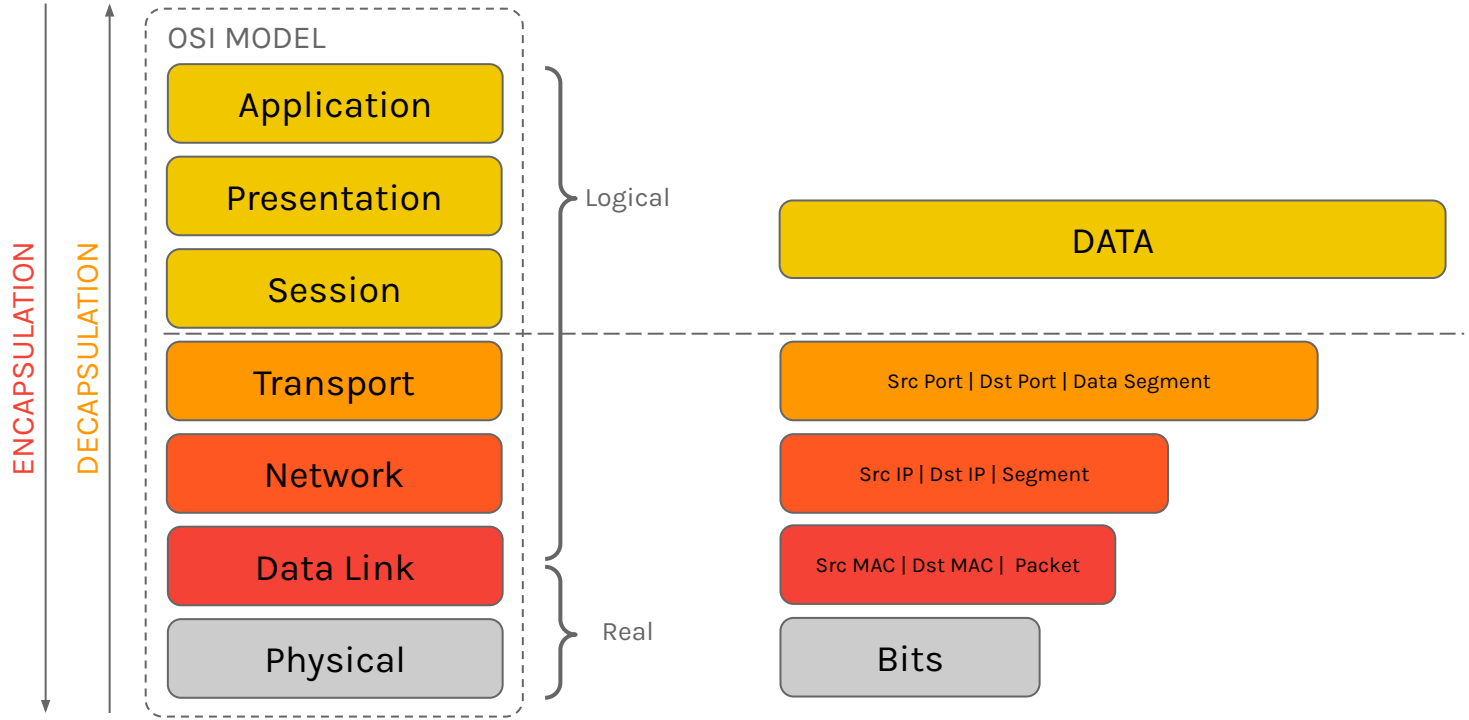
PDU	Addressing
Frame	MACs
Source, Destination, Packet	3131:abab:e6e6

This level is the combination of two layers, the Logical Layer and the **MAC Layer**. The **MAC Layer** links the logical world with the physical world using **MAC Addresses** used to identify **physical** connections.

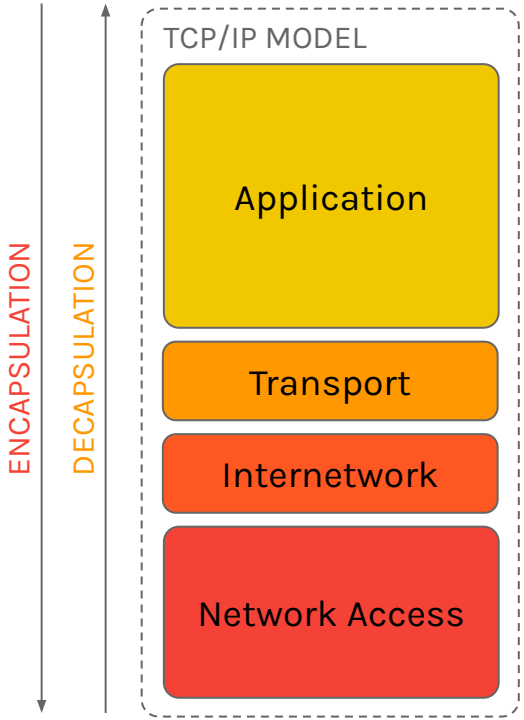
THE STACK COMMUNICATION



THE STACK COMMUNICATION



THE TCP/IP MODEL



The OSI Model could be too much elaborated, then it was simplified and implemented on the most famous model that is called **TCP/IP Model**.

The models is called in this way because the focus was only on two protocols, **TCP** and **IP** protocols.

They needed to make sure the data was reliable because when back, when it was called ARPANET everything was very sensitive, research and during Cold War situation to connect missile silos.

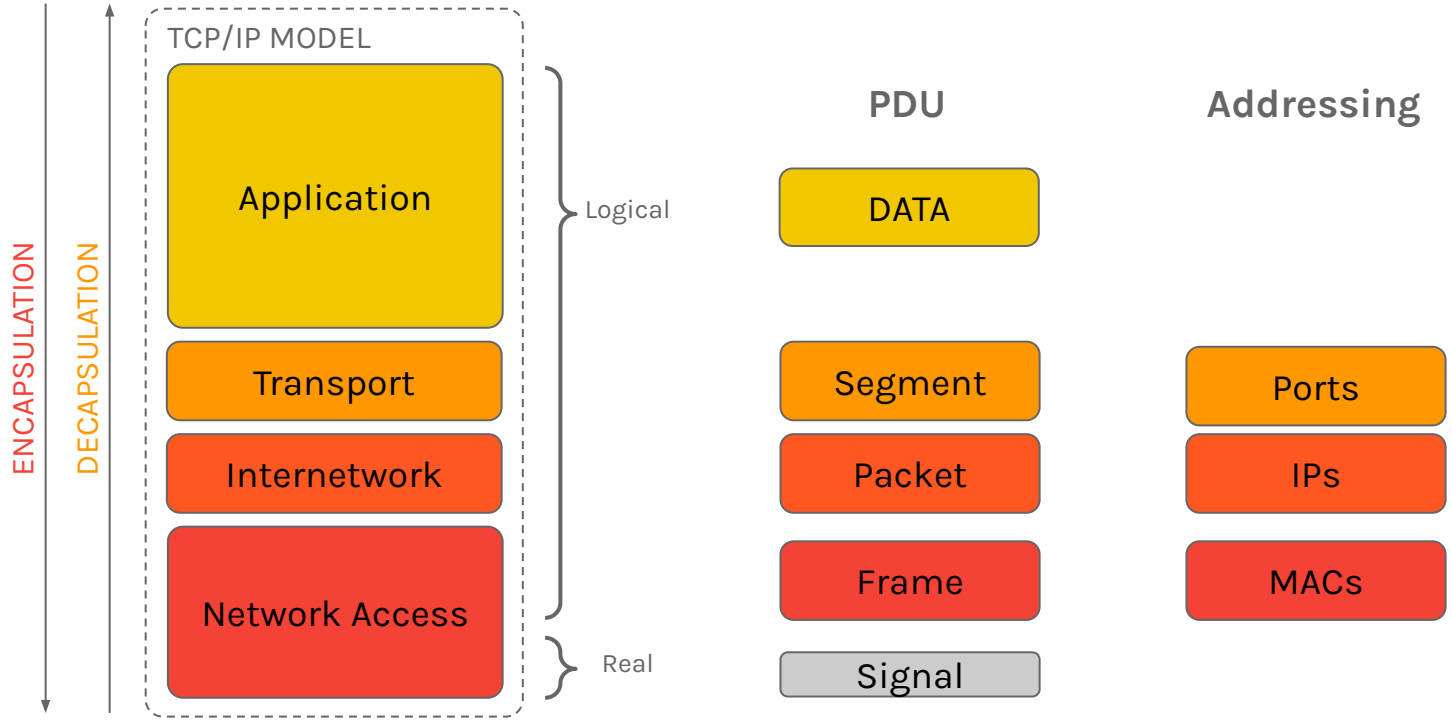
After that, they defined the model where the **Application Layer** is completely responsibility of the developers, the application, the security, the presentation and the session.

Transport layer name remains.

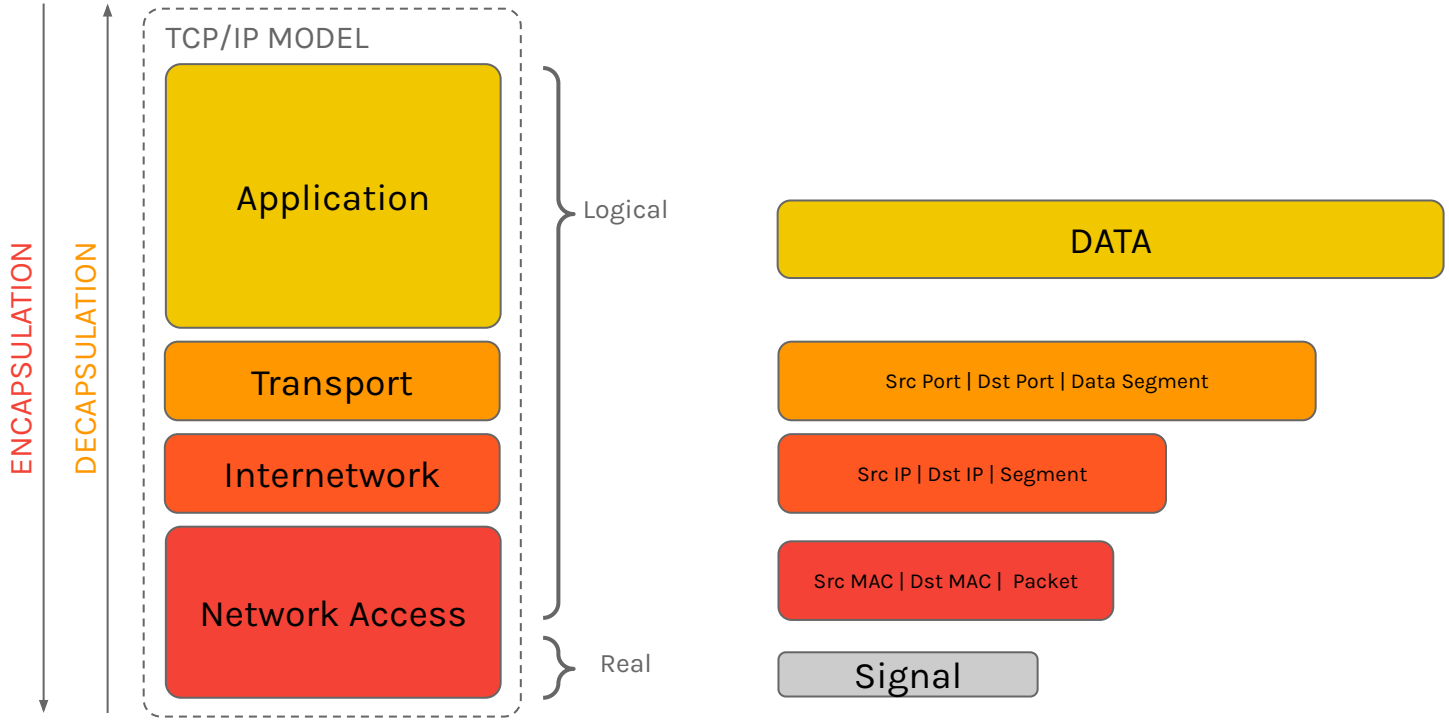
The Network layer is called **Internetwork Layer** that means Interconnected Networks.

The last two layers are merged into **Network Access Layer**, where the **Drivers** translate the information between Logical and Real World.

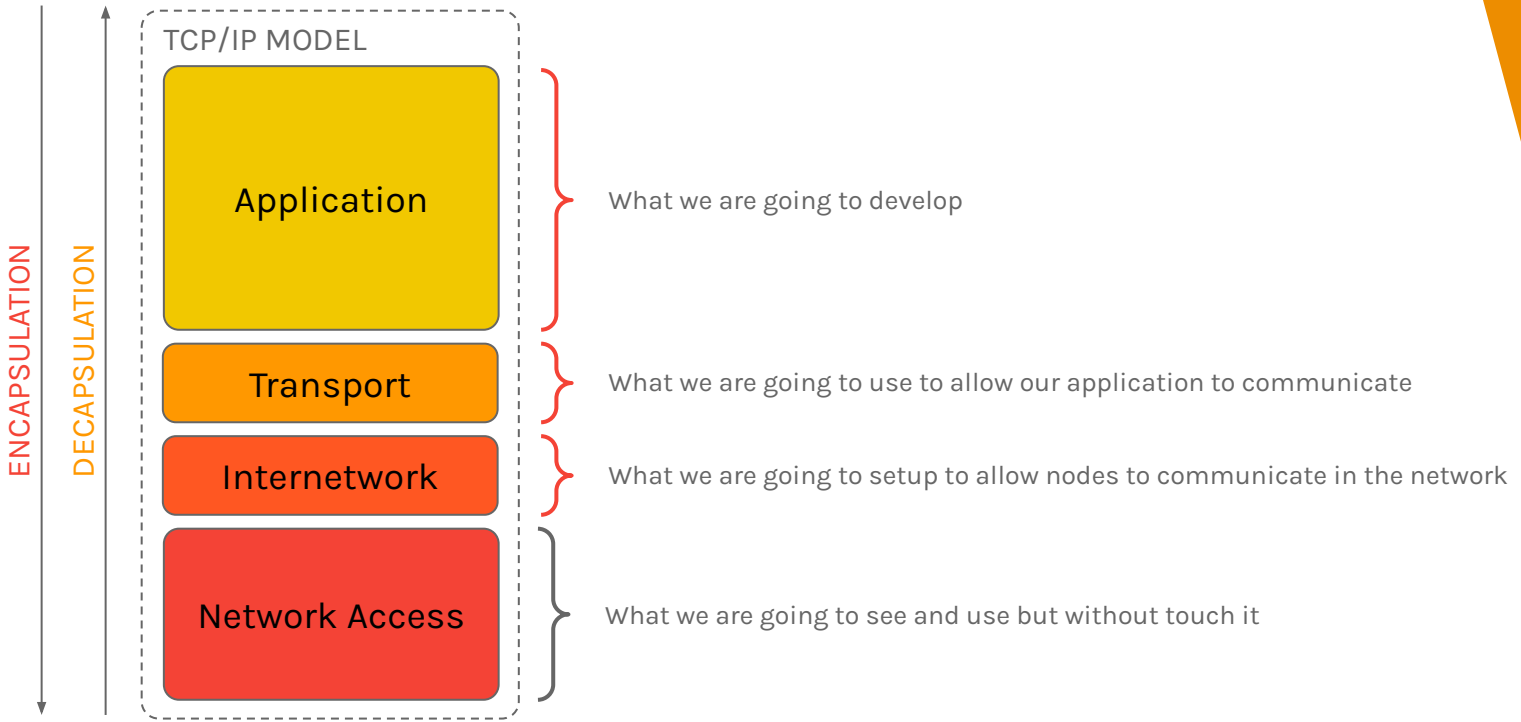
THE TCP/IP MODEL



THE TCP/IP MODEL



THE TCP/IP MODEL



EXERCISE TIME!

IMPLEMENT THE NETWORK STACK

Not really but now you can design a software solution, using your preferred Object Oriented Language to try to implement each stack of TCP/IP model and emulate two nodes as separated processes or thread where each one have one instance of stack to communicate, sending a string for instance.

- Design an **Application Class** (extendable or abstract)
- Design an **Transport Class** that take Data from application, segment it and create **Segments Class** (using such as max segment size?)
- Design an **Internetwork Class** which create **Packets Class** from Segments.
- Finally, create a **NetworkAccess Class** which create **Frames Class** with Packets inside and put these frames into a Shared Memory (Stack).
- You must create a **Device Class** where each one had one of each layer instance that are used to Encapsulate and Decapsulate the PDUs.
- The NetworkAccess instance must check new frames on the Shared Memory.
- The Shared Memory acts as the Media Access Controller.
- Each layers is connected in some way, maybe reference or pointers?

RECAP!

- We introduced how the communication works inside a Device
- We introduced the OSI model
- The Encapsulation and Decapsulation process
- The PDU of each Layer
- The Address of each Layer
- The TCP/IP Model
- But how the communication arrive to destination?
- How setup a device as Node, Switch or Router?
- How develop an Application that will use a Transport protocol?

3.

NETWORK ROUTING

Oh no, which way now?

PROF. SANTORO FEDERICO FAUSTO

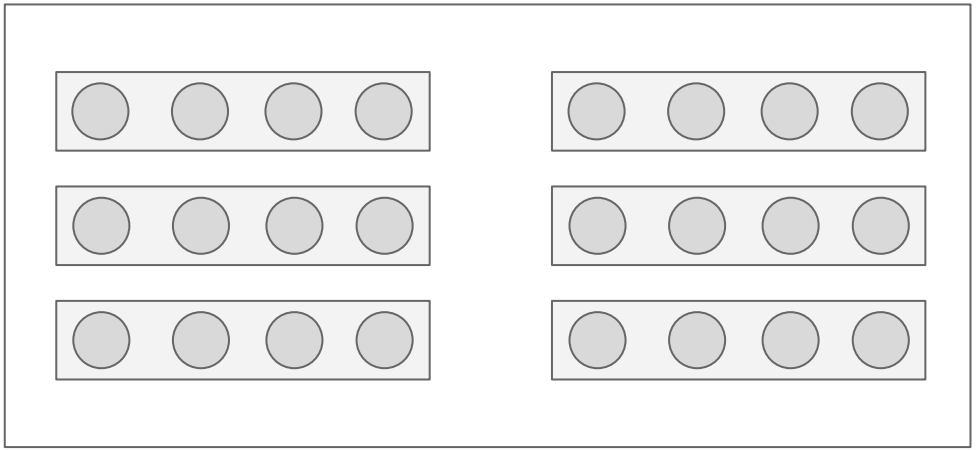
PHYSICAL AND LOGICAL ADDRESSES

Before start to the Routing, or how the messages reach the destination, we must talk about the **Physical** and **Logical** Addresses.

Remember, a network topology is a map, a model!

Then now imagine a Classroom inside University, that is in some department.

In this classroom there are a lot of seats, divided by rows and sides.



We need to be able to identify each person here, both logically and physically.

Then start about the physical addresses!

PHYSICAL AND LOGICAL ADDRESSES

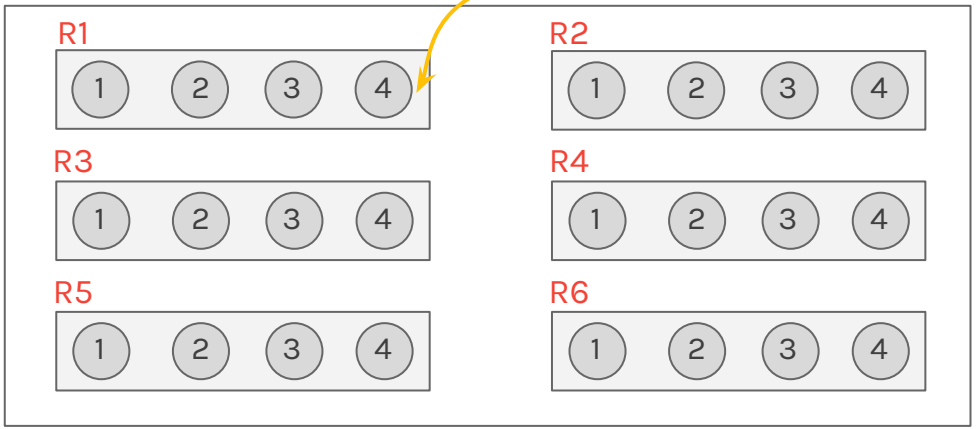
Remember, the first thing is identify the room! Such as **Room 24**!

Then we need to identify the rows in some way, for instance, with **R#** notation!

Finally, we need to identify, uniquely, the seats, to give a message to someone. We can just numerate them! (1, 2, etc.)

Then we can identify, univoqually, a particular seat, for instance: **ROOM-24:R1:4**

This is the **Physical Address** of that seat and we identified this defining set of rules, a protocol!



In this way, each seat, each row and each room can be identified with their Physical Addresses.

What about the Logical Addresses?

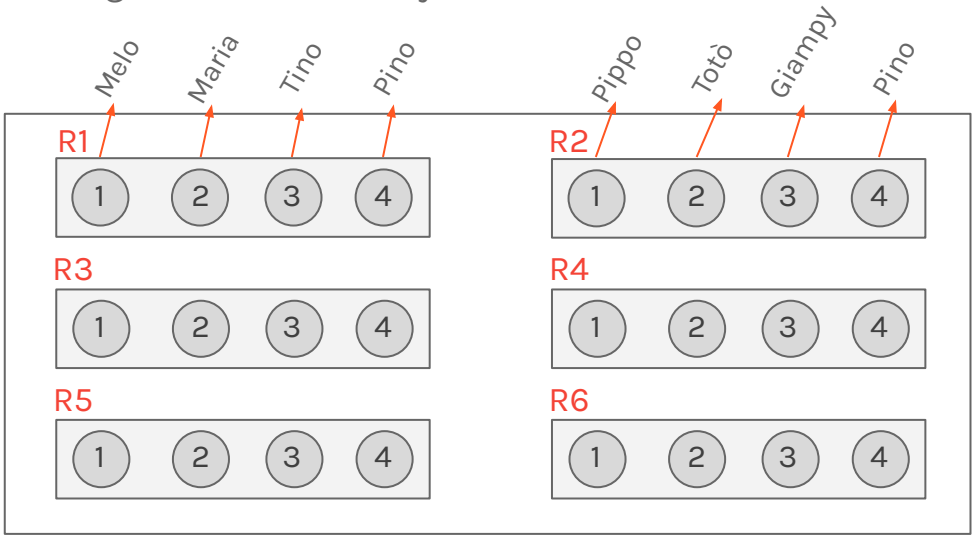
PHYSICAL AND LOGICAL ADDRESSES

For instance, we can use the First Names of each student as **Logical Address**.

Instead of using a "Physical Description", such as "that guy with long black hair", someone give to these students a Logical Name, because it made sense to call them with their Logical Names, whatever.

This is also a Protocol, in fact everyone will get a first name, to be identified.

Another way to identify a person is using the job occupation, such as "Hey, look, the engineer!" or titles (Hey the Prof is here!).



But we have a problem, in this class there are two **Pino**s.

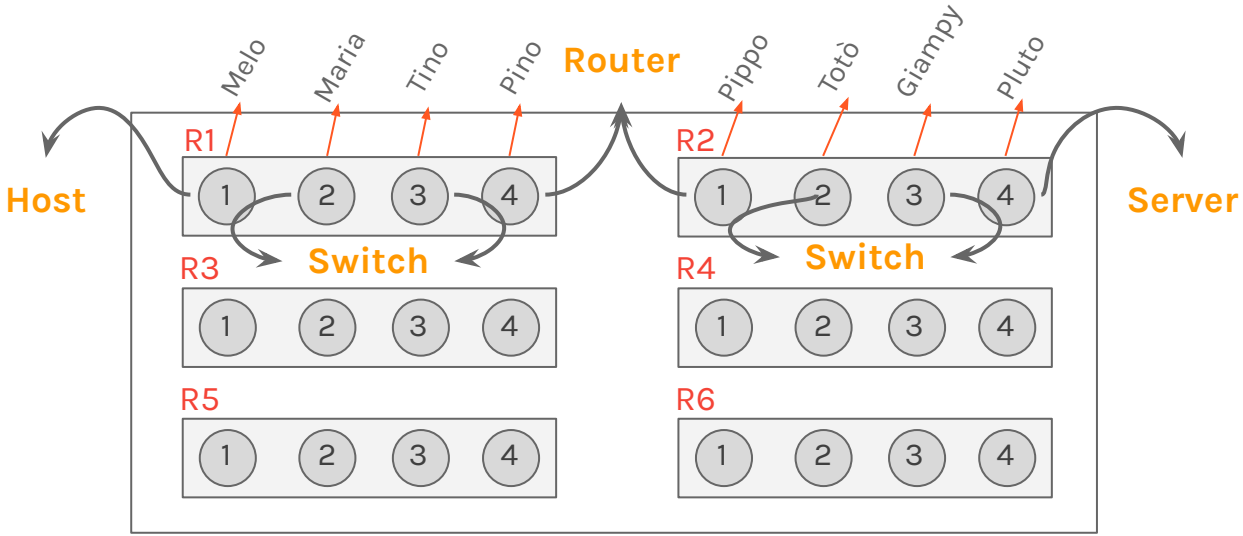
Who will receive the message?

Then, in the same network, two devices can not have same logical address!

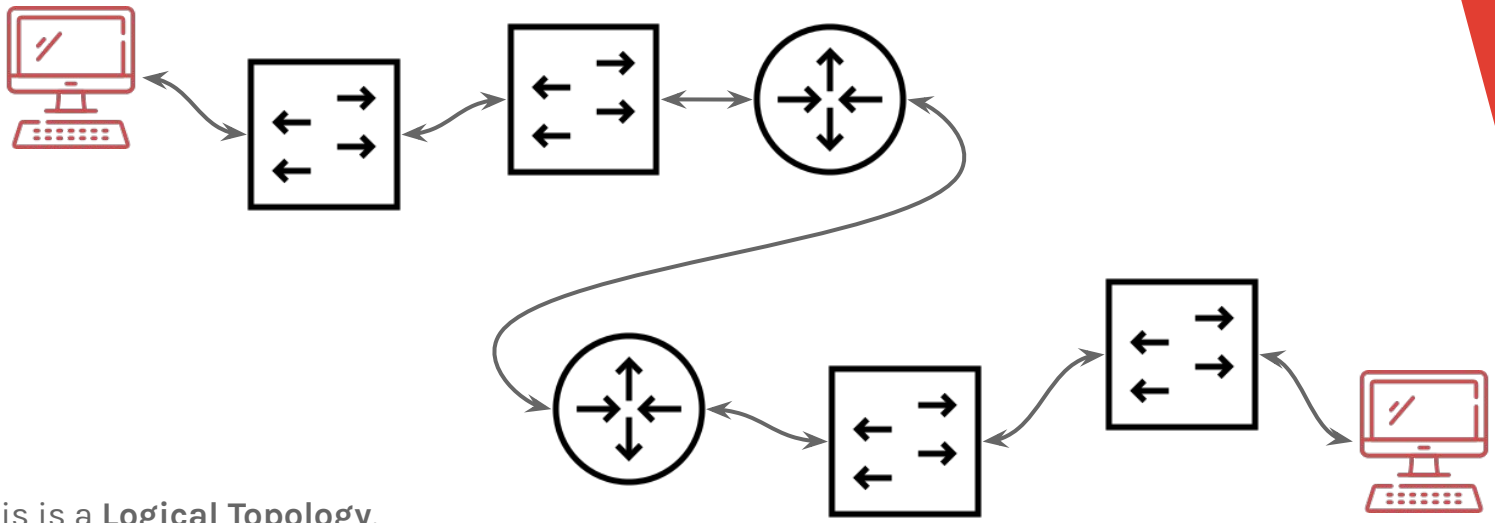
PHYSICAL AND LOGICAL TOPOLOGY

This is a **Physical Topology**,

Physical is the real stuff, where the wires are connected to, interfaces that are being used, where thing are located in the physical environment.



PHYSICAL AND LOGICAL TOPOLOGY

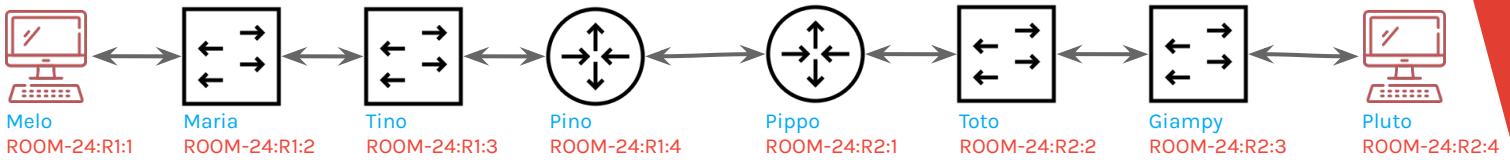


This is a **Logical Topology**,

Logical is all about the behaviour of how something is going to act.

NETWORK STACK AND ROUTING

Then, considering the Physical Topology, we can re-design the Logical Topology with Physical and Logical Addresses.

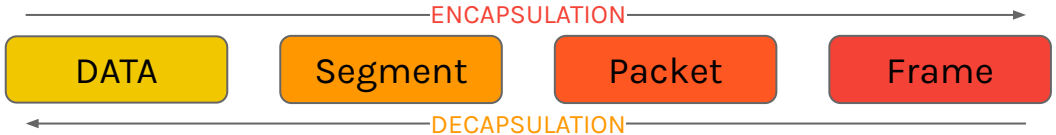


Now imagine that each Node is going to have a series of communication, starting from **Melo** which wants send a message to **Pluto**.

What Melo needs to do? Well, **encapsulate** the Data!

To do that, we need to remember the Network Stack and each node need to use the same Network Stack.

But remember, **not all Network Devices could have the complete Stack**, some special devices could have only a part of it.



NETWORK STACK AND ROUTING

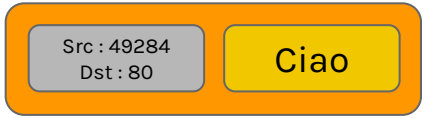
Melo wants send the message “Ciao” to **Pluto**, the first thing to do is to **segment** the message using the **Transport Layer**!

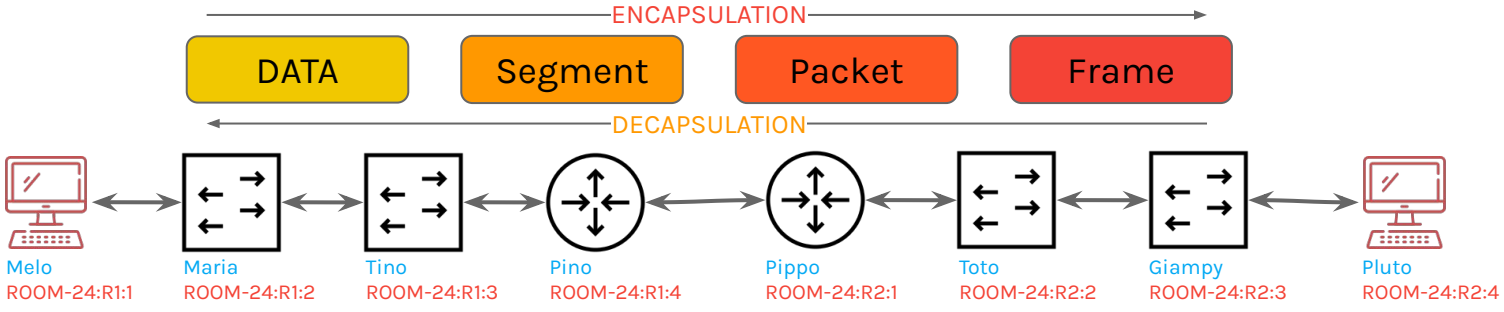
Then decide how many “pieces” of the message can be put inside a Segment (in this case all pieces for instance).

The Segment has a **Header**, where we need to put the Source and Destination addresses, there we are going to use the **Port Addresses**.

The destination address(port) is defined by **Pluto** (e.g. 80).

The source will choice a random free address (port) (e.g. 49284)





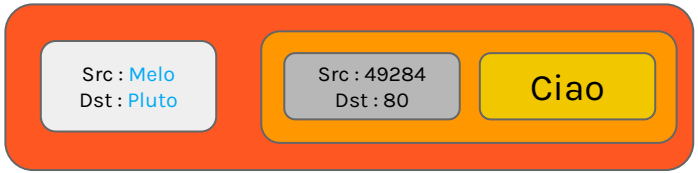
NETWORK STACK AND ROUTING

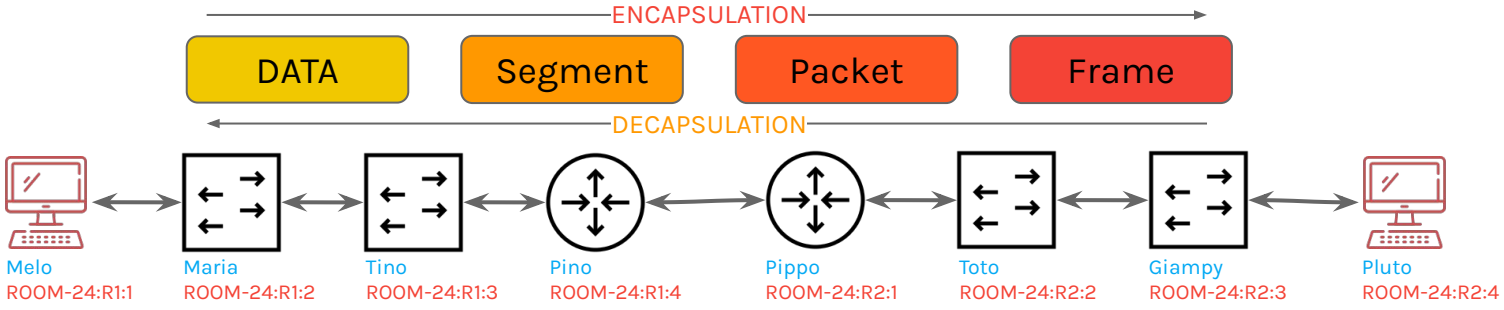
Melo now need to create a **Packet** using the **Internetwork Layer**. Inside the Packet, Melo will plut the Segment created before.

Also in the Packet we have a Header where we are going to use, as Source and Destination addresses, the **Logical Addresses**.

Source address is **Melo**

Destination address is **Pluto**





NETWORK STACK AND ROUTING

Melo now need to create a **Frame** using the **Network Access Layer**, and this one has also a **Header** where we will use, as source and destination addresses, the **Physical Addresses**.

But what address **Melo** will use? **This is VERY CONDITIONAL**. The first important thing to think is "**Is the destination in the same Network?**"

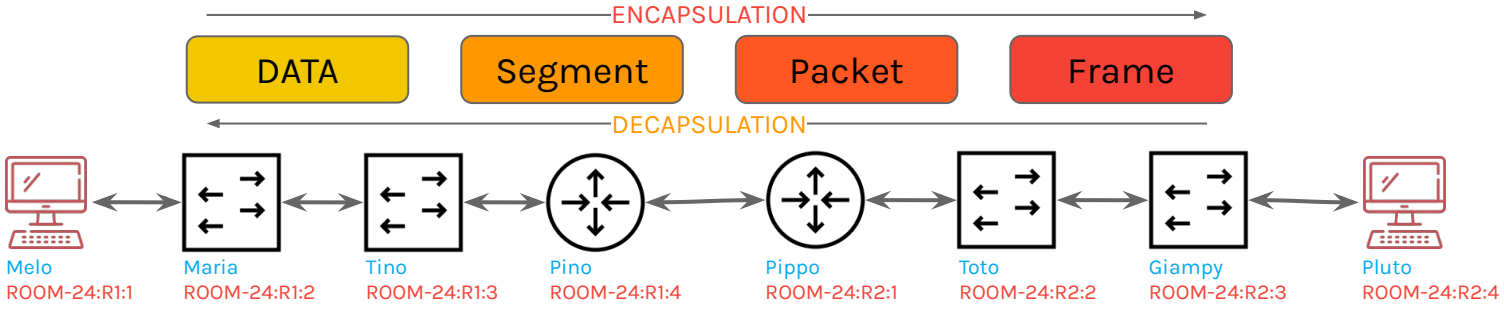
To do this, the Physical Addresses are compared, for instance, if a destination address starts with ROOM-25, **Melo** understand that he need to leave the room, then **exit from the main door**, to find the destination, in another room.

Then all Frames must be sent to the **Melo**'s Gateway because it is the only point connected to another network.

In fact, the Gateway logical address is recommendable on the network nodes.

Then the Source Address will be **ROOM-24:R1:1**

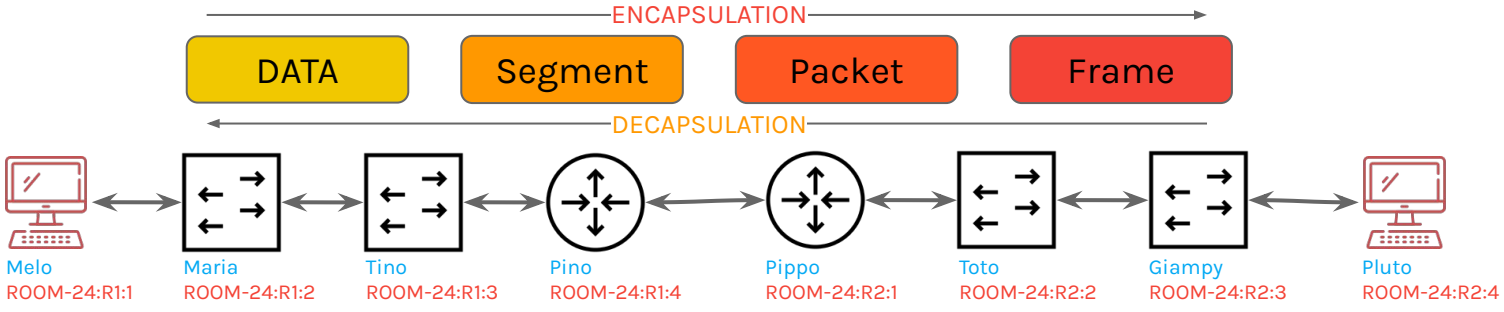
The Destination Address will be **???**



NETWORK STACK AND ROUTING

Then **Melo** needs to find the Physical address of its **Gateway** (**Pino**, in this case). To do this, **Melo** must "scream" to all its network nodes and devices, with a special request, such as "If you are **Pino**, then send to me your Physical address".

To send this to all network nodes, as Destination Physical Address it will use a special address, called **Broadcast** (in this example **TO_ALL**)



NETWORK STACK AND ROUTING

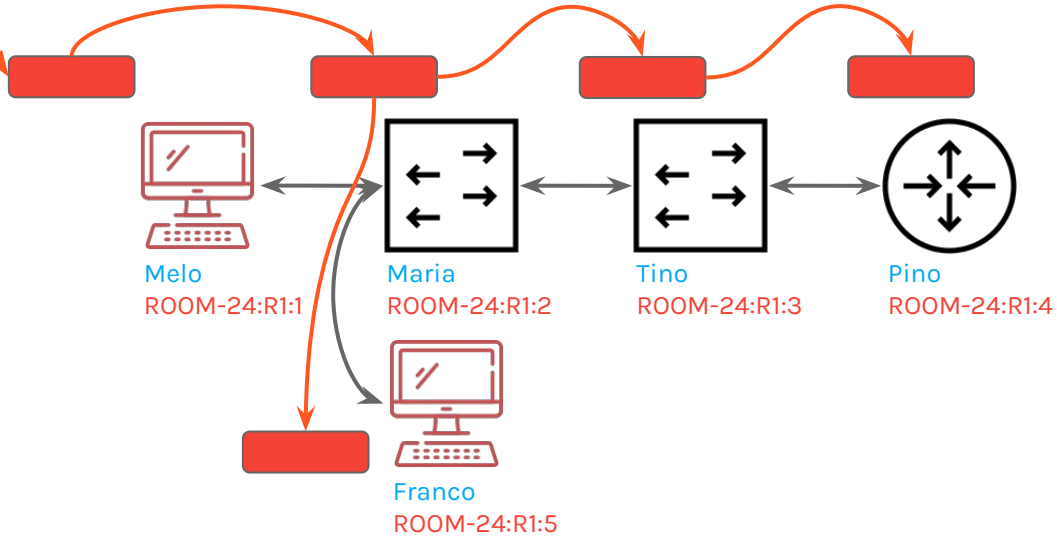
Src : ROOM-24:R1:1
Dst : TO_ALL

If you are **Pino**, then send to me your Physical address

This message arrives to **Maria**, which is a **Switch**. **Maria** does not know where is Pino, then replicate the request to all. At the same time, **Maria** is learning that, **Melo** is placed to her left and save **Melo**'s address into an physical addresses book.

Franco will receive the message but will drop it because he is not **Pino**.

Tino will receive the message but is a **Switch** and does not know where is **Pino**, then replicate the message to the others but it learned that, **Melo** is place to left.



Finally, **Pino** receives the request and, because the name is his, does not drop the message and will prepare a special response, directed to **Melo**.

Then the same process will begin, but with some important differences!

NETWORK STACK AND ROUTING

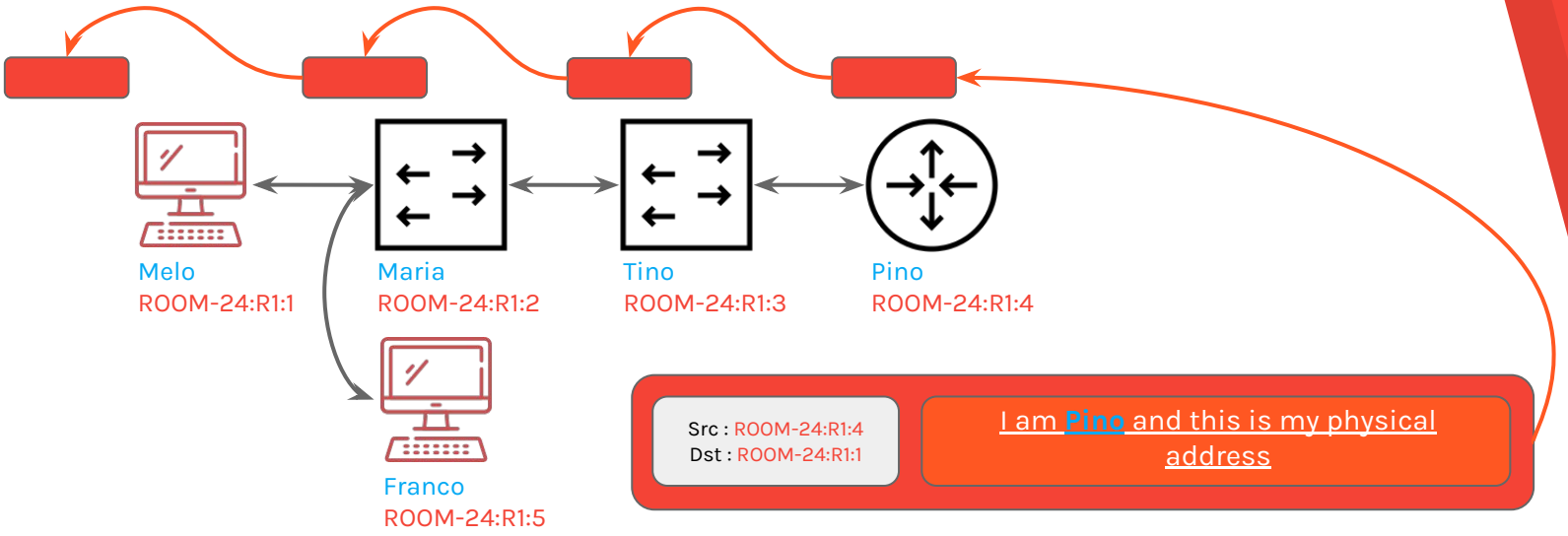
Pino will send the response, with its Physical Address, to **Melo** and send the message to **Tino**. **Tino**, which is a **Switch**, receive the message but, instead of replicate it, it will send the message to **Maria** because, previously, he learned that Melo is to him Left. Also, he learned that, **Pino** in to the right and save Pino's physical address to its physical addresses book.

Same as **Maria**.

Finally, **Melo** will receive the message with **Pino**'s Physical address and can use it to send the encapsulated message (the original one). This process is called **Address Resolution Protocol**, or **ARP**.

The Physical Addresses Book is called **MAC Table** and associate port to address. (**Left** ->**ROOM-24:R1:1**)

Melo learned the Physical Address of Pino and save it and associate to **Pino**'s Logical address into special Logical Addresses Book, that is called **ARP Table** (**Pino** -> **ROOM-24:R1:4**)



NETWORK STACK AND ROUTING

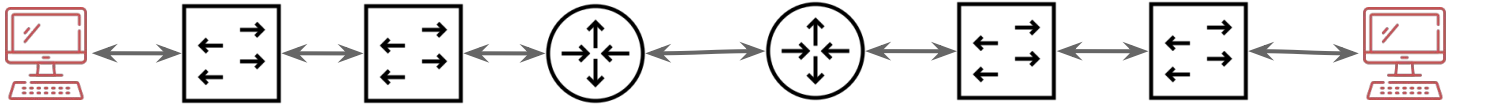
Now the message can be delivered to **Pino**, witch is the Gateway for Melo to another Network.

Before going toward, we learned that, Switch works with Physical Addresses, then we can say that **Switches are Layer 2 devices**!

Now **Pino** can receive the message, but how he will send it to another network?

Routers know the nodes inside their network or networks, because a Router can be connected to different networks in a **Direct Connection (DC)** or via another Router, In this case, or someone told to Pino that all other nodes are connected via Pippo (**Static Route**) or Pino and Pippo shared their knowledge (**Dynamic Route**). If there are any route to destination, **the message will dropped**. These information are saved into a **Routing Table**.

When **Pippo** receive the message for **Pluto**, he does not know the Physical Address of **Pluto**, well another ARP game starts!



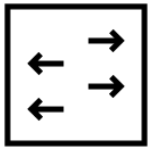
NETWORK STACK AND ROUTING

Ok now let's talk about real Network and start to associate the example to real things!
Then recap:

- Nodes can **send** and **receive** message, creating them via **Encapsulation** and receiving via **Decapsulation**.
- Nodes can talk with other Nodes **on the same Network** via **Physical Addresses**.
- If the Node does not have the Physical Address of a same Network Node, can find it via **ARP Protocol**.
- When a Node learn the Physical Address relative to a Logical Address, the Node will save it into **ARP Table**.
- If a Node needs to send the message to Node of another Network, he must send it to its **Gateway**.
- To check if a Logical Address of destination is in the same Network, the Node will check the Logical Address with Logical Network Address (**Net Mask or Subnet**).

NETWORK STACK AND ROUTING

Ok now let's talk about real Network and start to associate the example to real things! Then recap:



- Switches can receive Frames and send them via specific **Network Interface**.
- If the destination is known, as the Network Interface to reach it, the message will send over that Network Interface.
- If the destination is unknown or broadcast, **the message is duplicate to the other Network Interfaces**.
- If a switch receives a messages from a unknown source, it will save the address into **MAC Table** where it will associate the source to a specific Network Interface.
- Switches are “transparent” network devices, nevertheless they can have Logical and Physical addresses, but could not be considered while the routing.
- These device works on Layer 2.

NETWORK STACK AND ROUTING

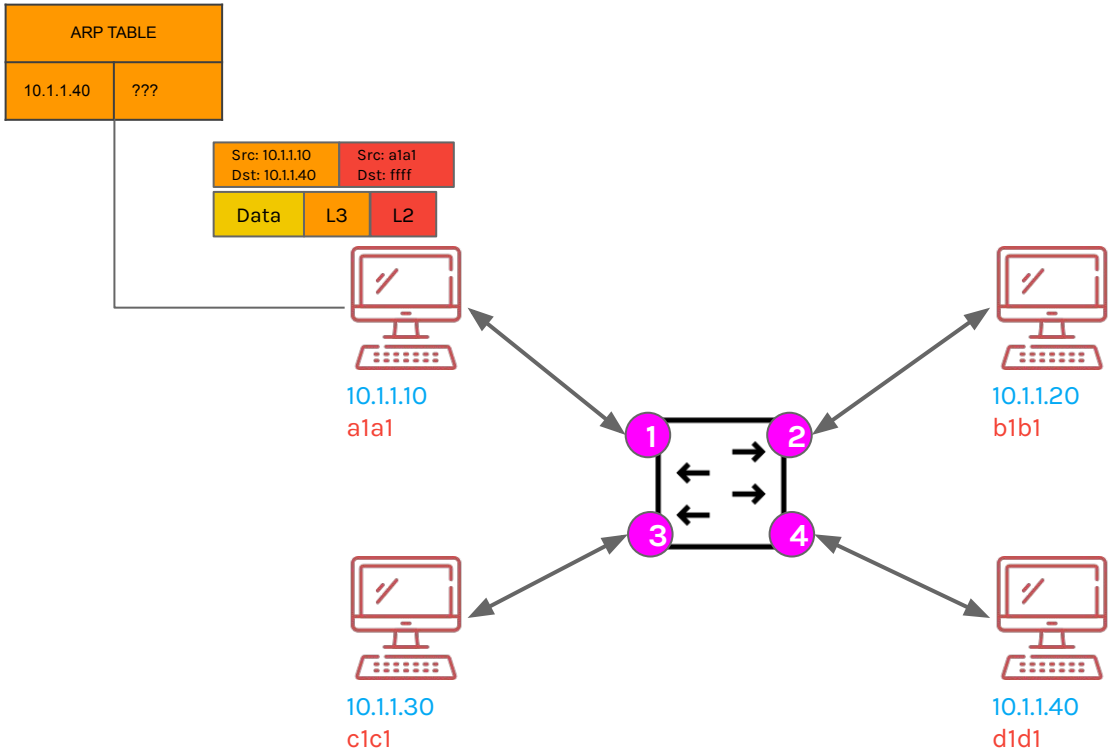
Ok now let's talk about real Network and start to associate the example to real things!
Then recap:

- Routers can send and receives Frames for him, for its network or another Network.
- To check if the messages is directed to specific Network, it needs to decapsulate the Frame to Packet format to get the Logical Address.
- Router knows Network Nodes, via the same ARP Protocol, then had their **ARP Table**.
- Router knows how deliver the message to specific network or other router via rules saved on **Routing Table**.
- This table can be filled via **Direct Connection**, manually (**Static Route**) or exchange with other routers (**Dynamic Route**) with some special protocols (RIP, etc.)
- Essentially, Routers are Nodes but can route messages!
- Then, they have a Logical and Physical Addresses!
- Routers are Layer 3 devices.

From now, Logical Addresses are **IP Addresses** and
Physical Addresses are **MAC Addresses**.

NETWORK STACK AND ROUTING

Imagine to have a network like this, let's try to route the messages over the same network (now using TCP/IP addresses).

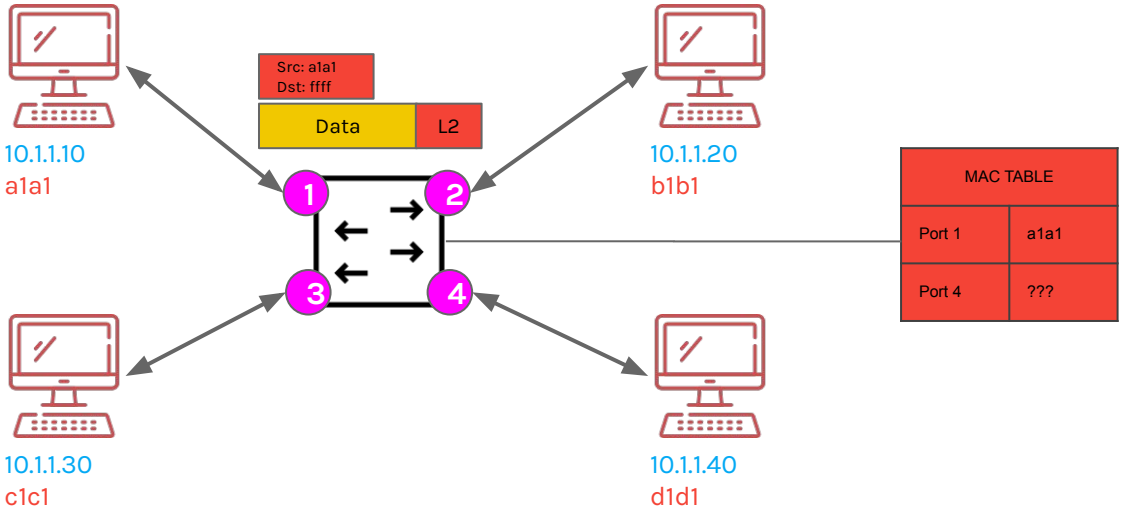


NETWORK STACK AND ROUTING

When the Switch receives the message, because it is Layer 2, all information above Layer 2 are Data for the Switch. At the same time Switch learned that, the a1a1 is connected to the Network Interface 1 (Port 1).

The destination is unknown, then the address is the Broadcast address.

Then, the Switch Learned and replicate the same message to all ports.

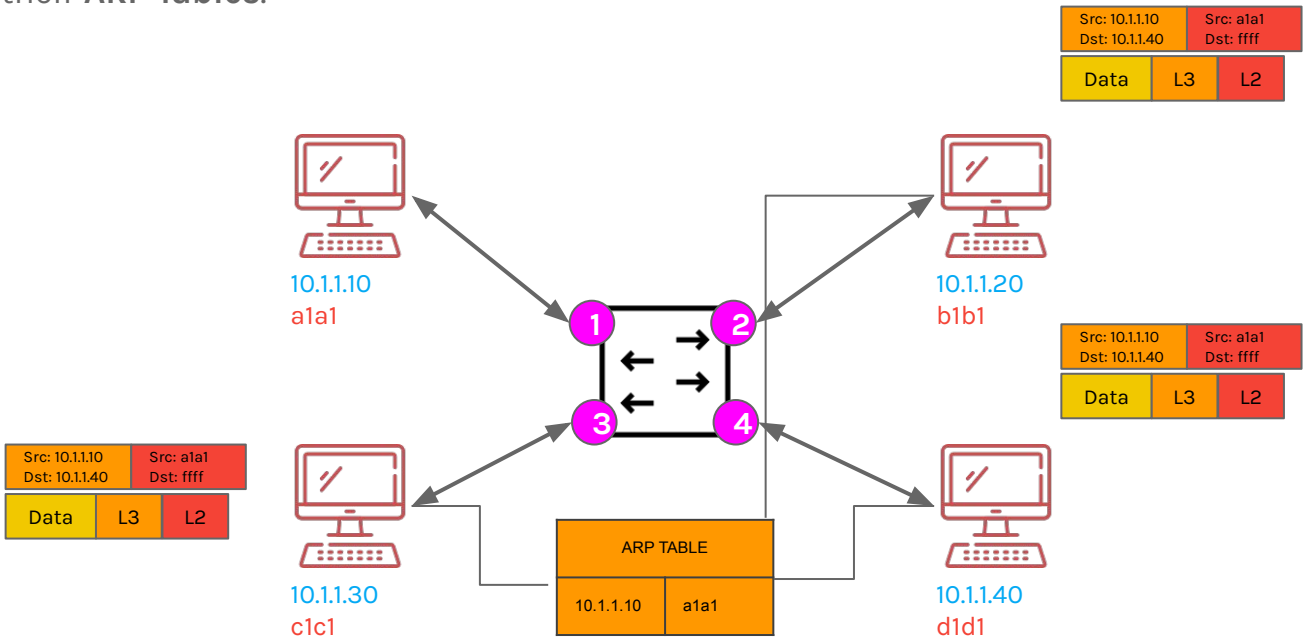


NETWORK STACK AND ROUTING

The other nodes will drop the message, because the IP Address is not theirs. To check this, **the nodes made the Decapsulation to get the IP Address Destination**.

The right receiver, receive the message.

In this case, all nodes Learned that, the IP 10.1.1.10 is the a1a1 MAC address, then update their **ARP Tables**.



NETWORK STACK AND ROUTING

The previous process was possible because the destination was part of the same network, but **how did the sender know?**

Nodes use the Subnet or Net Mask to understand if the node is in the same network!

But what if the node is not in the same network? It must send the message to the Router, then its Gateway.

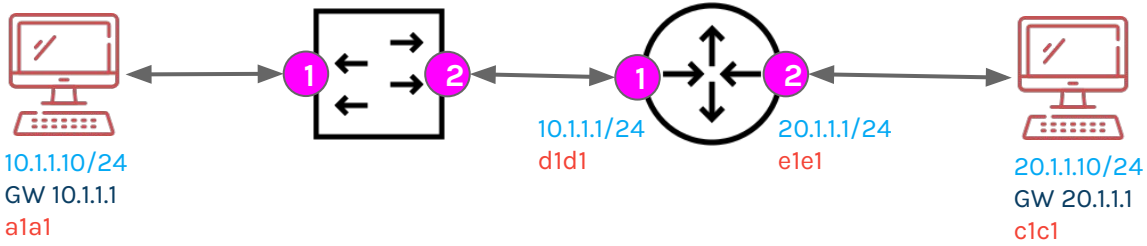
Then a node, to be able to send messages to the same network, must have a **Subnet**!

Also, a node, to be able to send messages to other network, must have a **Gateway IP**!

Routers, for each Network Interface, have an IP Address, a Subnet and MAC Address.

Of Course also the Switch might have an MAC and IP address, so act like a Node, but only for the management of the Switch (Layer 3 Switches). For the process described they are not used.

But how the Routers will route the messages to another Network?



NETWORK STACK AND ROUTING

A Router is a Node, the main difference is that, if it receives a message that is not for him, it will try to delivery the message, before dropping it.

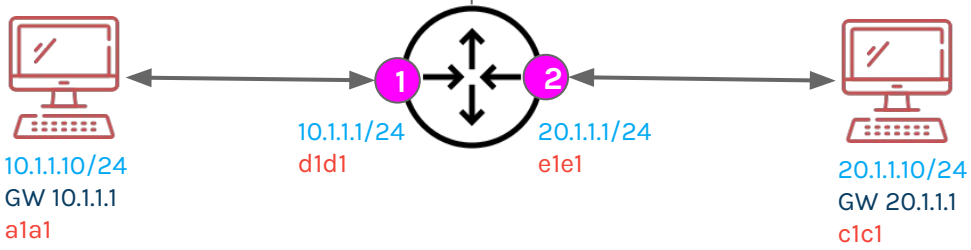
Imagine to define two networks, the 10.1.1.x/24 and the 20.1.1.x/24 which are connected via a single Router, then it needs two Network Interfaces, one for each Network.

Routers must maintain a map of all Networks they know about. This map is the **Routing Table,** where are defined the routes.

A **Route** is an instruction for how reach a specific Network via specific Network Interface. As the MAC Table with MAC Address and Network Interface, in a Route are defined the **Network IP Address** (IP+Subnet) and the Network Interface.

There are three way to populate the Routing Tables Directly Connected, Static Route or Dynamic Route.

ROUTING TABLE	
10.1.1.0/24	Port 1
20.1.1.0/24	Port 2

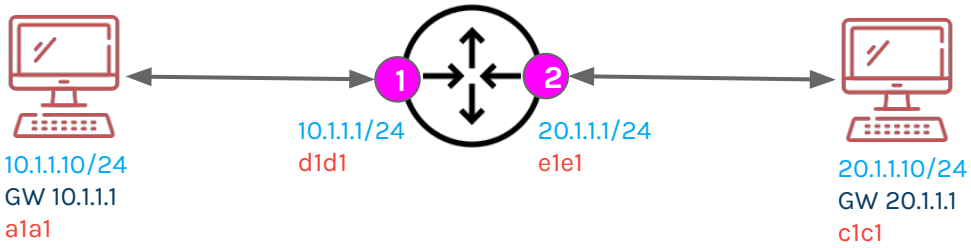


NETWORK STACK AND ROUTING

Directed Connected (or DC)

This route exists for every network that a router is directly attached to.

ROUTING TABLE		
DC	10.1.1.0 / 24	Port 1
DC	20.1.1.0 / 24	Port 2



NETWORK STACK AND ROUTING

Static Route

Imagine to want connect more than two networks, then via another router!

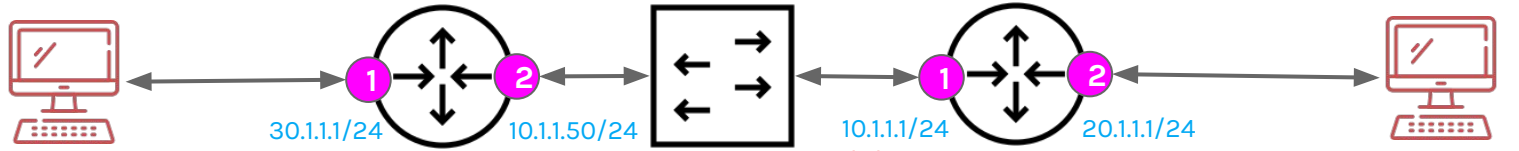
Each router had its Routing Table, with its DC rules.

A **Static route** is a route manually provided by network administrator.

Instead of using a Network Interface port, a Static Route will use IP Address of the Gateway of the passthrough Network.

ROUTING TABLE		
DC	30.1.1.0 / 24	Port 1
DC	10.1.1.0 / 24	Port 2
Static	20.1.1.0 / 24	10.1.1.1

ROUTING TABLE		
DC	10.1.1.0 / 24	Port 1
DC	20.1.1.0 / 24	Port 2
Static	30.1.1.0 / 24	10.1.1.50



30.1.1.10/24
GW 30.1.1.1
a1a1

30.1.1.1/24
f1f1

10.1.1.50/24
a2a2

10.1.1.1/24
d1d1

20.1.1.1/24
e1e1

20.1.1.10/24
GW 20.1.1.1
c1c1

NETWORK STACK AND ROUTING

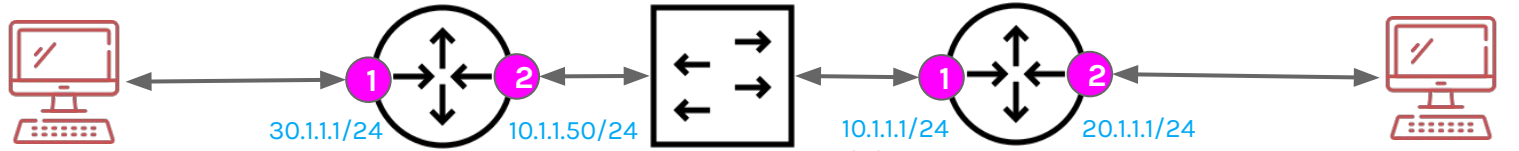
Dynamic Route

Instead of manually add routes, the routers exchange their Routing Tables with the other routers.

This technique is governed by Dynamic Routing Protocols, such as RIP, OSPF, BGP, etc.

ROUTING TABLE		
DC	30.1.1.0 / 24	Port 1
DC	10.1.1.0 / 24	Port 2
Dyn	20.1.1.0 / 24	10.1.1.1

ROUTING TABLE		
DC	10.1.1.0 / 24	Port 1
DC	20.1.1.0 / 24	Port 2
Dyn	30.1.1.0 / 24	10.1.1.50



30.1.1.10/24
GW 30.1.1.1
a1a1

30.1.1.1/24
f1f1

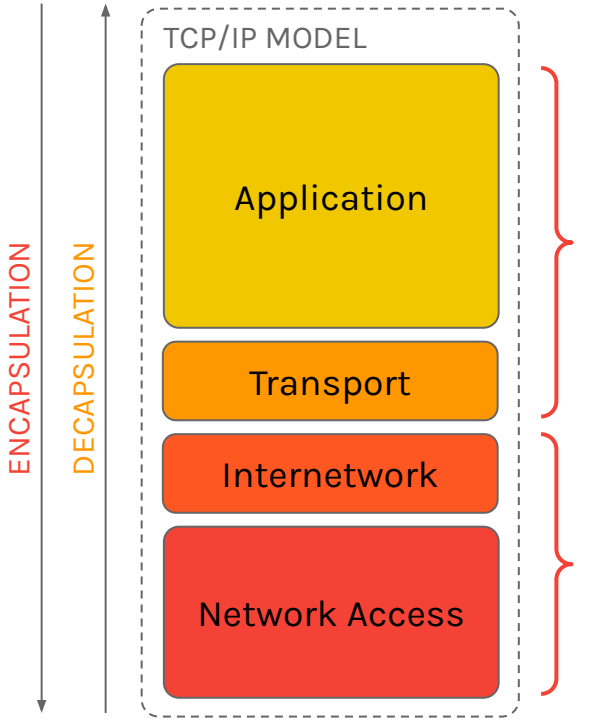
10.1.1.50/24
a2a2

10.1.1.1/24
d1d1

20.1.1.1/24
e1e1

20.1.1.10/24
GW 20.1.1.1
c1c1

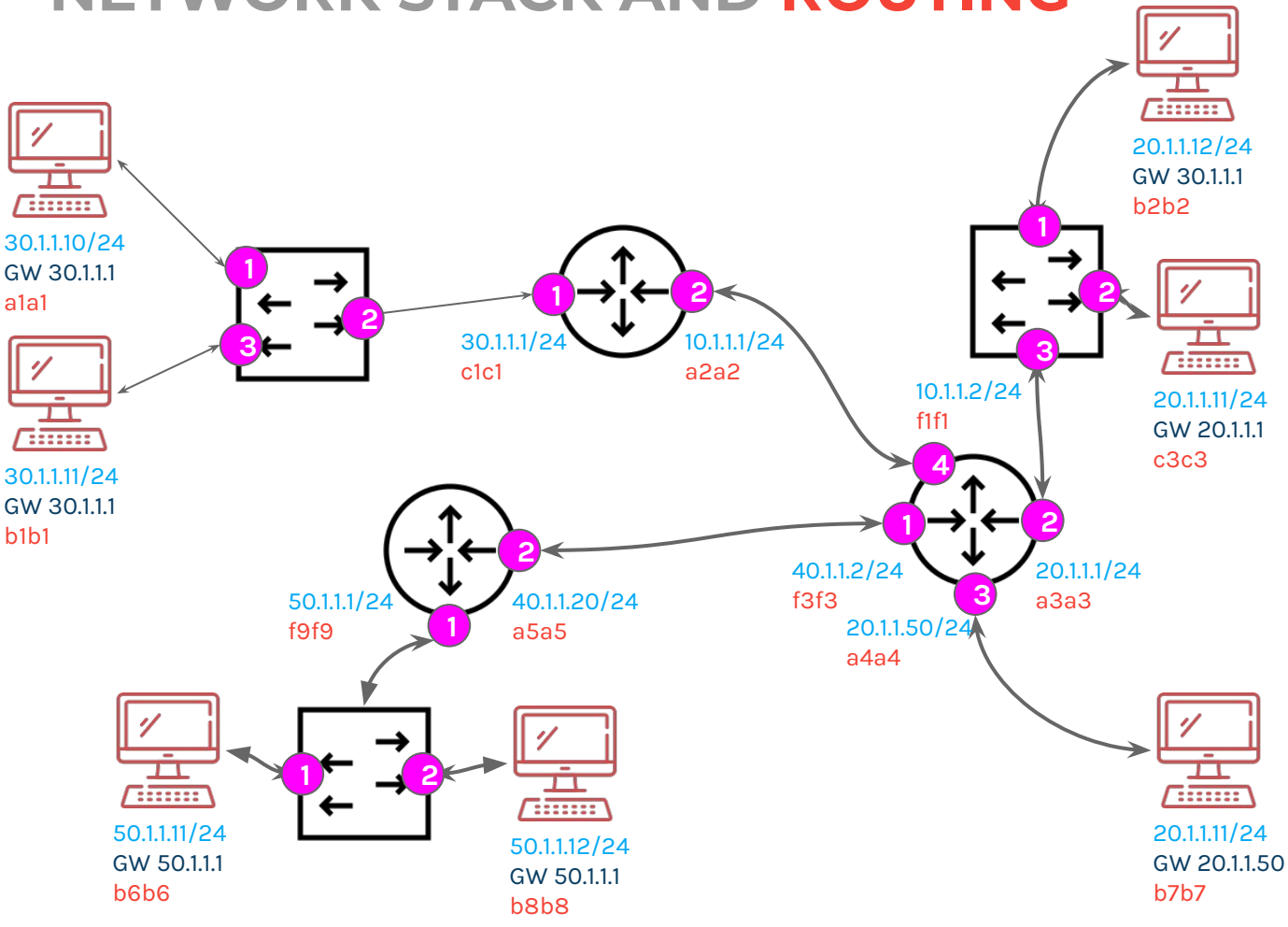
NETWORK STACK AND ROUTING



Then the Connectivity problems are not related to Login or Bugs. The Bugs are not related to the connectivity!

EXERCISE TIME!

NETWORK STACK AND ROUTING



RECAP!

- Learned about Logical and Physical Addresses
- The Physical and Logical topology
- How use Encapsulation and Decapsulation to send message
- How the message will be routed in the same network
- How the message will be routed to another network
- ARP Protocol
- Routes!
- MAC Table, ARP Table and Routing Table
- But how could we make these devices and setup a network?

4.

NETWORK ADMINISTRATION

ssh root@10.0.0.2

PROF. SANTORO FEDERICO FAUSTO

NODES AND VIRTUALISATION

Always remembering the TCP/IP network stack and how routing works, the first thing to understand is how to build networks already seen in real life.

It will take a lot of computers and other network devices that we do not have, then we should rely on virtualisation!

Virtualisation is a way of virtualising or emulating hardware by other hardware or software. Unlike containerisation, virtualisation actually occupies the resources allocated to it, effectively emulating a kernel.

To do this, we need a virtualiser.

NODES AND VIRTUALISATION

First of all, it is necessary to understand how virtualization works, and to understand this, we must understand what a **Hypervisor** is.

An Hypervisor is a piece of software which pool the resources from the physical machine (Host) to to the virtual environments. There are two types of Hypervisor, called Type 1 and Type 2.

- **Type 1** (Baremetal)
  Is installed on top of Host machine, allows to achieve best performances and security, for instance KVM.

- **Type 2** (Hosted)
  In installed on top of O.S., such as VirtualBox

With these we can create a Virtual Machine (VM), a software based computer.

VMs are isolated and can contain different emulated hardware and O.S. and can be moved to another Hypervisor.

NODES AND VIRTUALISATION

For our purpose, VirtualBox could be a good choice, but you can use any Hypervisor which you want.

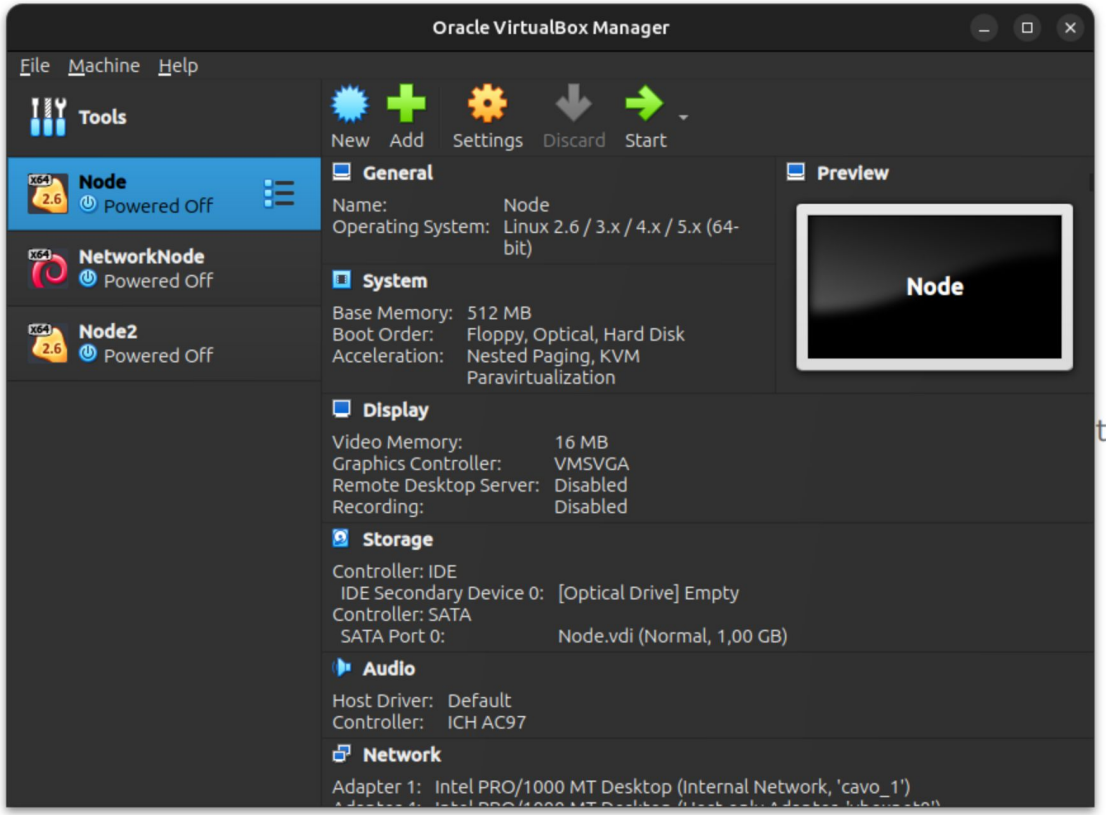
You can download it from website:
https://www.virtualbox.org/

You must install it on your O.S. following the screen instructions, but be careful, maybe you must enable the virtualisation on your computer from your BIOS.

If you are using a MacBook with ARM, i really do not know :D

NODES AND VIRTUALISATION

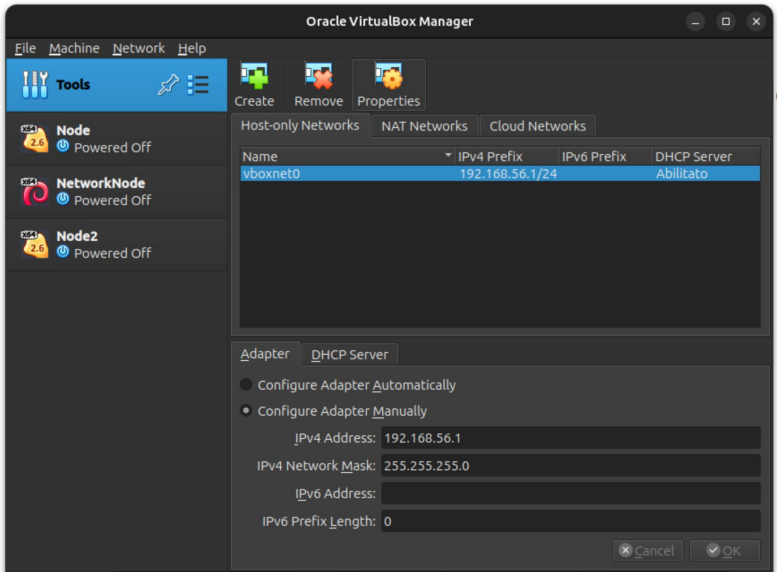
After the installation, you will going to see the main VirtualBox window

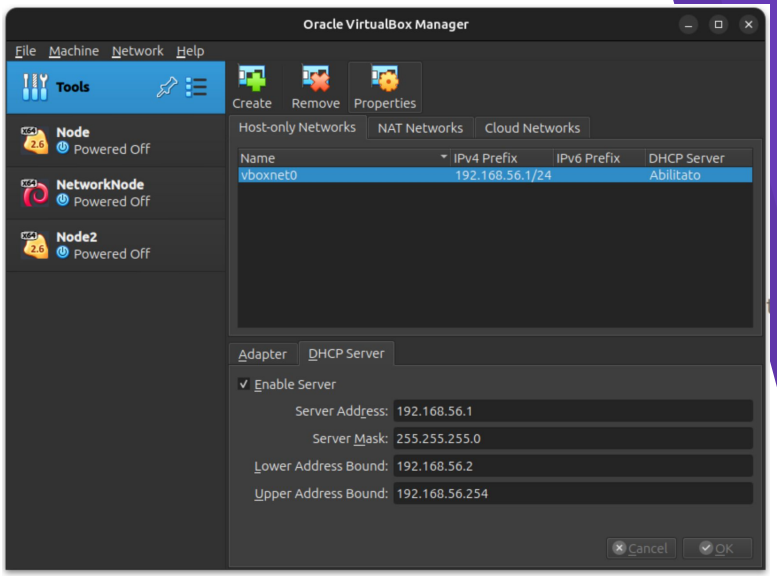


NODES AND VIRTUALISATION

In Tools you can create a new Host-only Networks, which will be used to simulate a real Network Interface on your Host to communicate with the VMs.

You can also simulate a DHCP Server for VMs to simplify the setup of specific network interface.





NODES AND VIRTUALISATION

Now we need an O.S. to use the VMs which we are going to create to simulate a network.

Essentially, when we talk about Network or Server, we talk about **Linux**/Unix O.S.

There are a lot families of Linux Distributions, the most famous are the **Arch** family, the Red Hat family (or **RHEL**) and **Debian** Family. Yes i know there is also Gentoo Family...

The family of a distribution change the how the system manages the device tree and, more important, the **Networking**.

The Networking management depends on Kernel, Software (such as NetworkManager service) and file system.

For our purposes, we will use Alpine Linux or Debian.

Then, you must download the .ISO of the operating system!

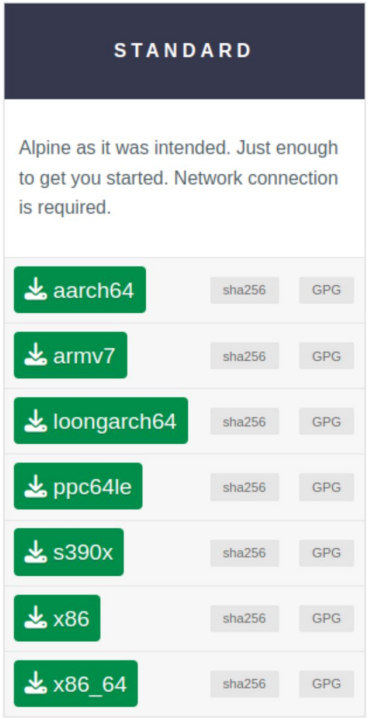
NODES AND VIRTUALISATION

Talking about Alpine Linux, you can download the .ISO from their web site:

https://alpinelinux.org/downloads/

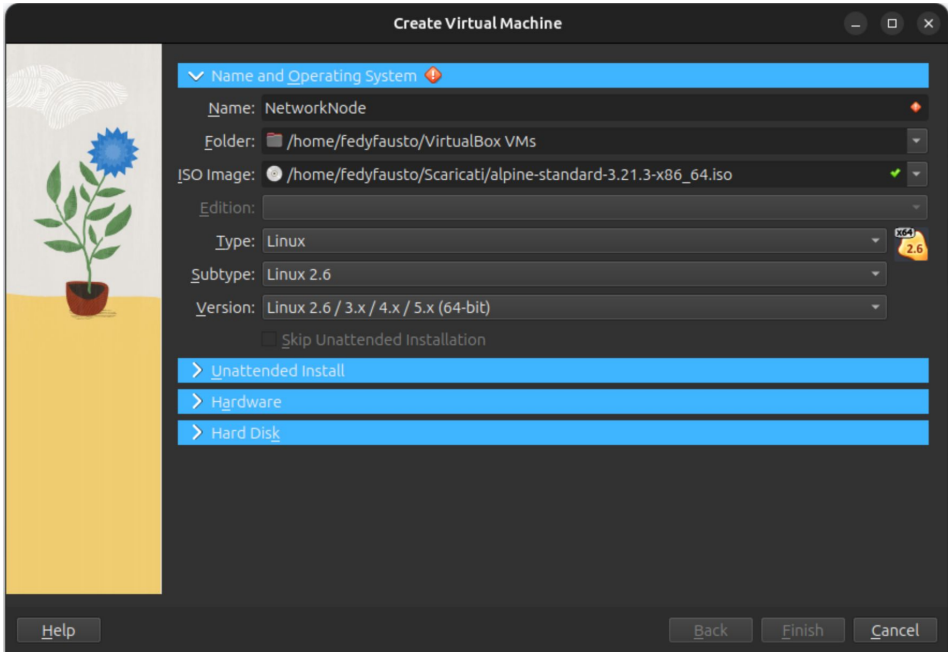
You must download the Standard Edition and the same architecture of your O.S. (this will be useful in the future).

When the download is completed, let us create our first Virtual Machine



NODES AND VIRTUALISATION

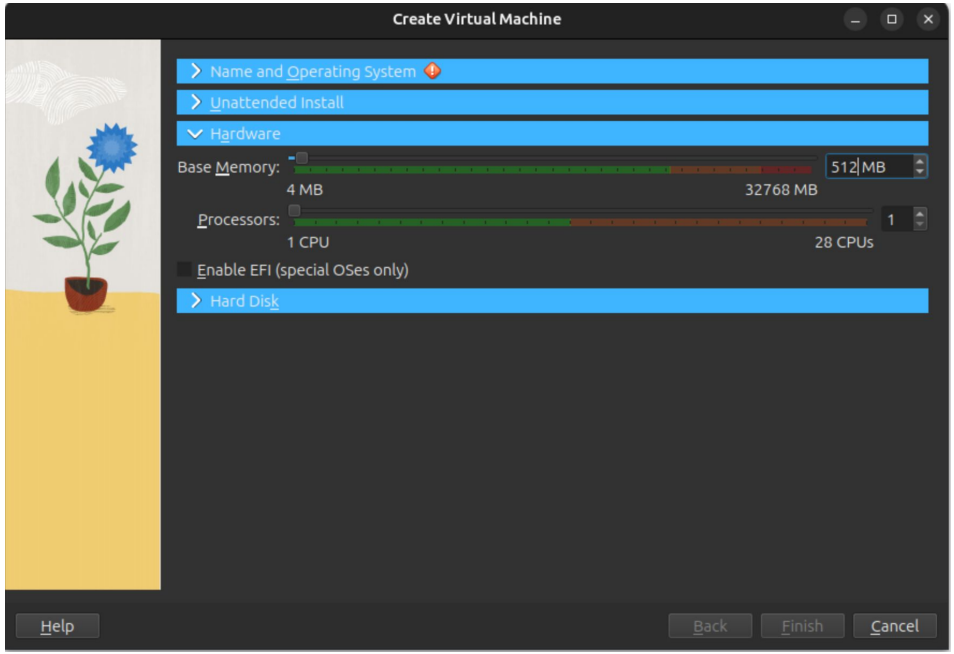
First of all, you must define a name for your Virtual Machine, select the ISO image downloaded before and the Linux version.



NODES AND VIRTUALISATION

Under Hardware tab, select how many resources, in terms of CPU and RAM, give to the VM.

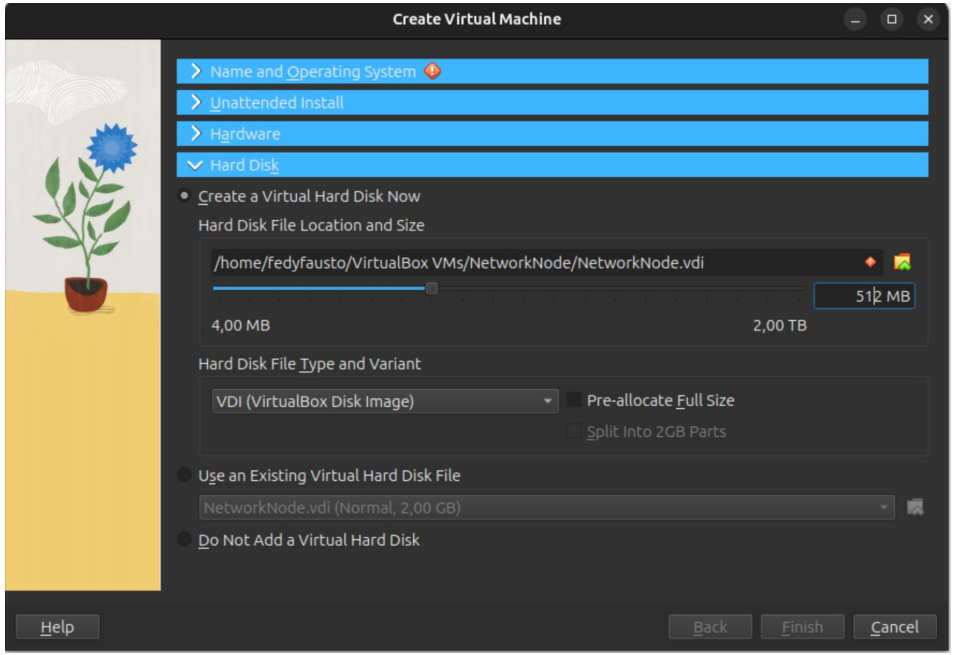
For an Alpine Linux you can give about 512MB of Ram and only one CPU. The RAM could be decreased but it is a trial and error.



NODES AND VIRTUALISATION

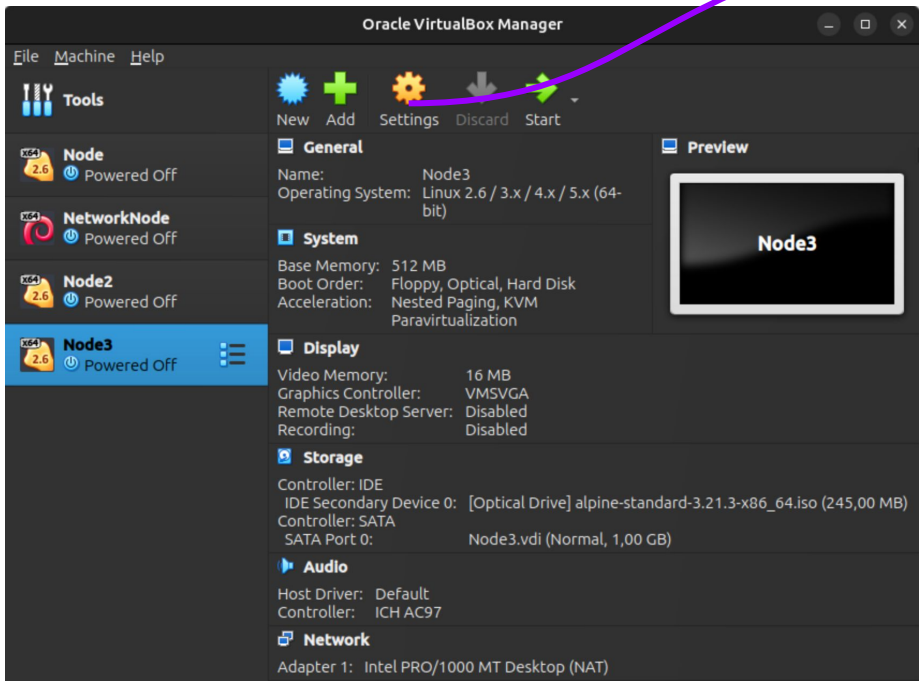
Under Hard Disk tab you can select how many Bytes allocate for the virtual Hard Disk.

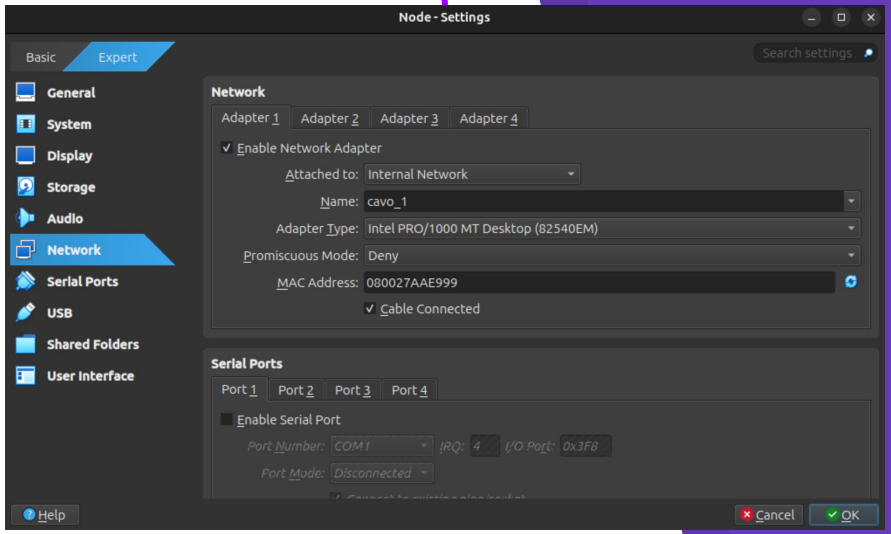
For an Alpine linux this could be about 1 GB.



NODES AND VIRTUALISATION

Finally, the machine is ready to be powered on, but we need to setup the Network Interfaces!





NODES AND VIRTUALISATION

From Settings you can manage your VM and setup the Network Interfaces. From the GUI you can only virtualise 4 NICs but you can add more than 4 NIC via CLI command. From here you can change also the MAC Addresses of NICs.

From here we can decide the type of NIC, in particular:

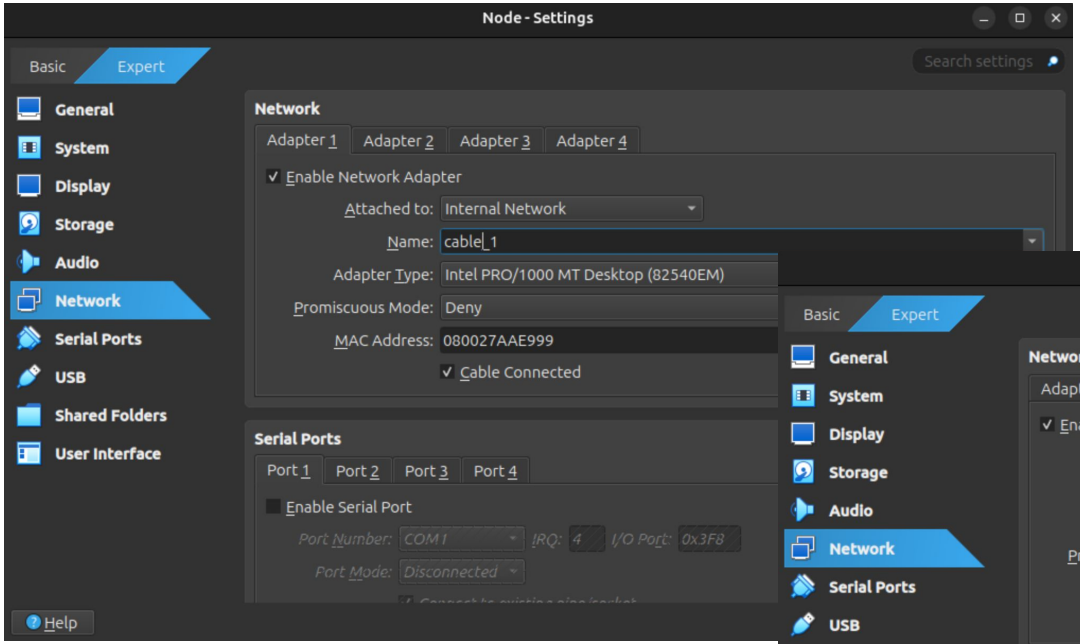
- **Internal Network**
  The one that we will use to emulate the cable connection between nodes. The name field is needed for emulate the cable! (Then two nodes connected with the same cable have the same name)

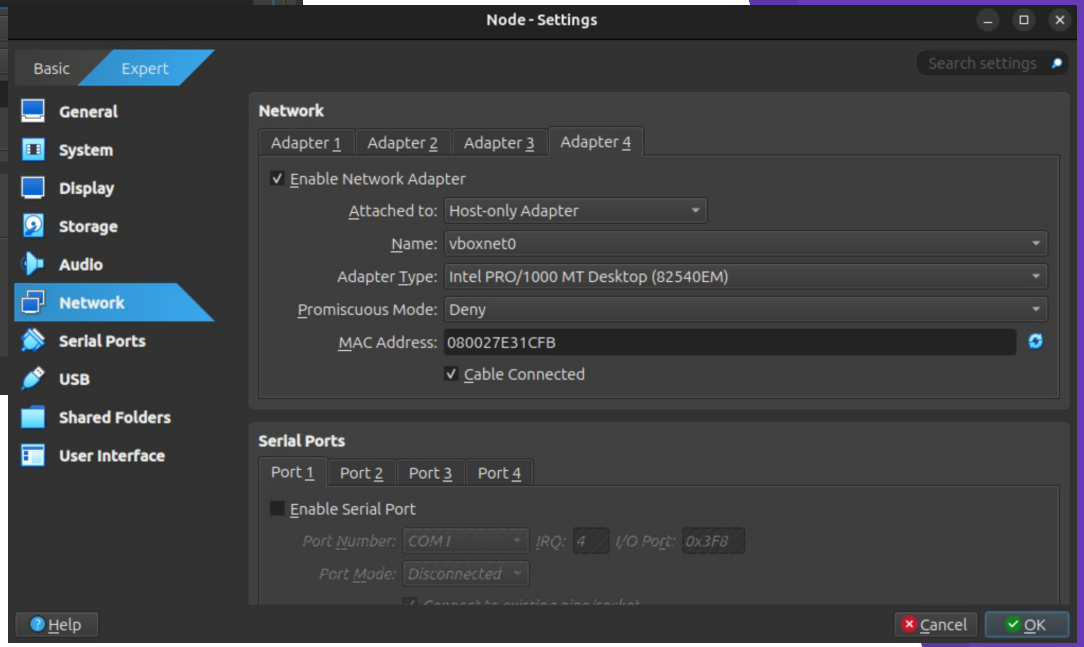
- **Host-only adapter**
  This is used to connect the VMs to a virtual NIC on Host system, in this way we are able to contact or connect to the VM.

- **NAT**
  Simulates a NIC under NAT, such as a home computer. This works well on Windows Host O.S., on linux use Host-Only

NODES AND VIRTUALISATION

PROF. SANTORO FEDERICO FAUSTO





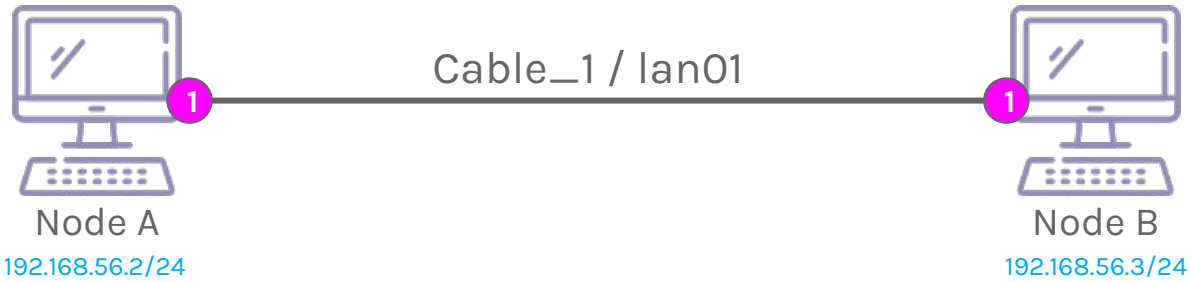
NETWORK COMMANDS

As first exercise we want connect two nodes. Remember, disable the NAT or Host-Only NIC for now.

Each node must have a IP address and same or compatible NetMask.

Then we need to setup the IP addresses on each Node with correct NetMask but to do this, we need to know few commands in Linux.

The most used command to manage the IP Layer is, ironically, the IP command.



NETWORK COMMANDS

The IP command is more than a command, **is suit of utilities**. In fact, with IP command we can add a route, set up a static IP address, manipulate interfaces and lot more.

This is the replacement of two older commands, `ifconfig` and `route` commands.

Then IP command takes the responsibilities of these commands.

Ok try the following command:

`ip addr show`

Or:

`ip a`

```
node:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:aa:e9:99 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:feaa:e999/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 08:00:27:e3:1c:fb brd ff:ff:ff:ff:ff:ff
node:~#
```

NETWORK COMMANDS

As you can see, this command gives the information about all network interfaces.

The first one is the loopback interface, a virtual network interface used by O.S. to identify the local network communication, such as communication between processes (now you got it).

The second one (in my case eth0) is the internal network of the VM.

The eth1 interface is the Host-Only but disabled.

From here, we can see also the MAC Addresses of each NICs.

```
node:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:aa:e9:99 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:feaa:e999/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 08:00:27:e3:1c:fb brd ff:ff:ff:ff:ff:ff
node:~#
```

NETWORK COMMANDS

If you want see these details only for a specific NIC, you must use:

```
ip addr show dev [DEVICE]
```

In my case:

```
ip addr show dev eth0
```

With IP command you can also manage the Data Link Layer, then for instance you can put off or on a NIC:

If you see the sudo command that means **that command will modify some system settings**, then **you need root powers!** (instead of sudo you can see the special characters, such as **#** that means root permissions or **$** (or nothing) that means normal user permissions)

```
sudo ip link set [DEVICE] down // set off the NIC
# ip link set [DEVICE] up // set on the NIC
```

NETWORK COMMANDS

Ok now we want assign or add a IP address with a NetMask to specific NIC, to do this we will use:

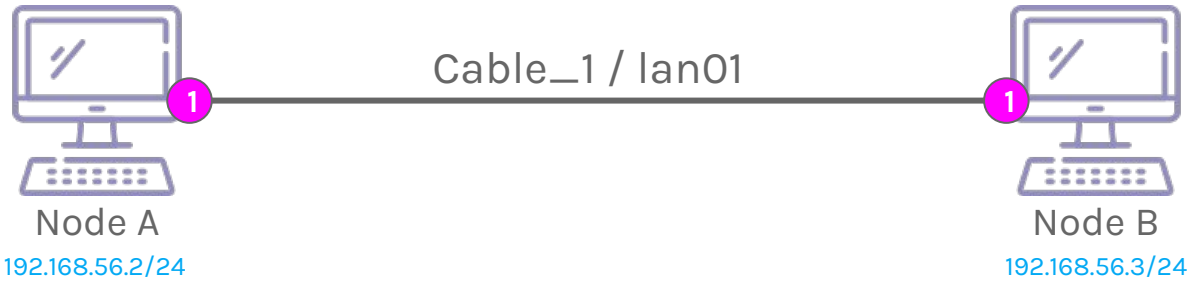
```
# ip addr add [IP ADDRESS / NETMASK] dev [DEVICE]
```

In this case, on node A:

```
# ip addr add 192.168.56.2/24 dev eth0
```

In node B:

```
# ip addr add 192.168.56.3/24 dev eth0
```



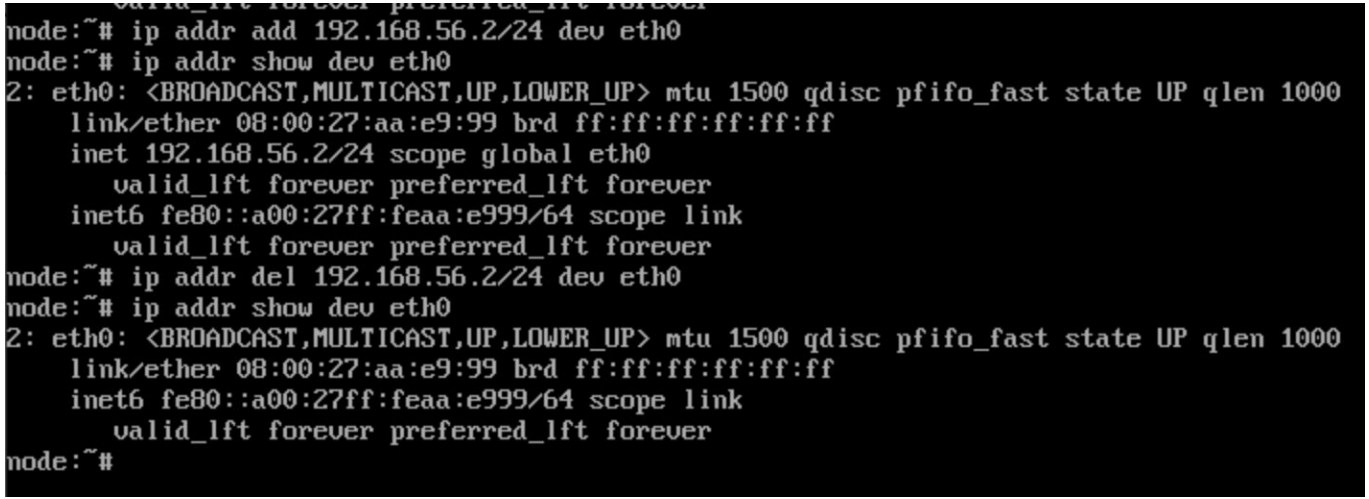
NETWORK COMMANDS

If we want delete an IP address we can use a similar command:

```
# ip addr del [IP ADDRESS / NETMASK] dev [DEVICE]
```

In this case, on node A:

```
# ip addr del 192.168.56.2/24 dev eth0
```



NETWORK COMMANDS

A little resume:

```
$ ip addr show
$ ip addr show dev [DEVICE]
# ip addr del [IP ADDRESS / NETMASK] dev [DEVICE]
# ip addr add [IP ADDRESS / NETMASK] dev [DEVICE]

# ip link set [DEVICE] (down | up)
```

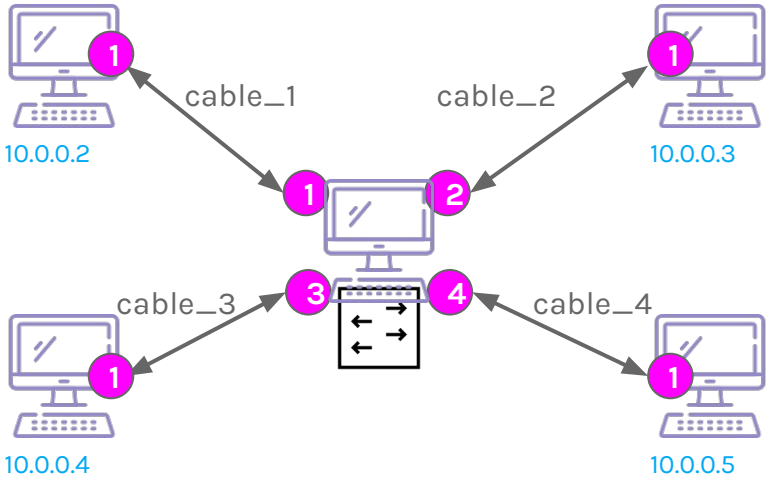
Anyway, we are able to set the correct IP addresses to our VMs and check if the connection work, maybe using the ping command:

```
$ ping [IP ADDRESS]
```

NETWORK COMMANDS

Now suppose to want a working network like this one where we need a Switch to create a more complex local network, then we need few VMs as nodes and one VM as **Switch**!

To do this, we need to manage the Link Layer with IP command.



NETWORK COMMANDS

Considering the Switch node with four NIC as Internal Network and remember, a Switch is essentially a fusion between Hub and Bridge, but a Bridge is physically a device that can connect two Hubs, but now they are defined as software, then we can add multiple ports!

If you are using VirtualBox, set Promiscuous Mode allow All to all NICs

First of all we need create a Bridge with the IP command:

```
# ip link add br0 type bridge
```

And add each NIC to this bridge

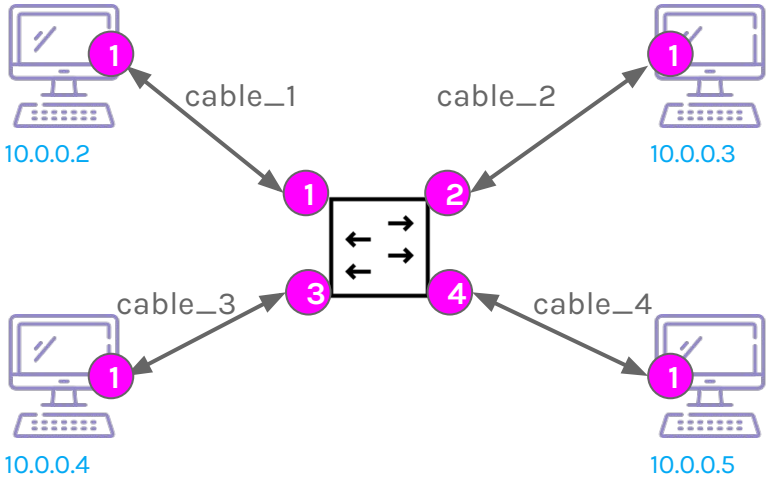
```
# ip link set [DEVICE] master br0
```

And activate the promiscuous mode:

```
# ip link set [DEVICE] promisc on
```

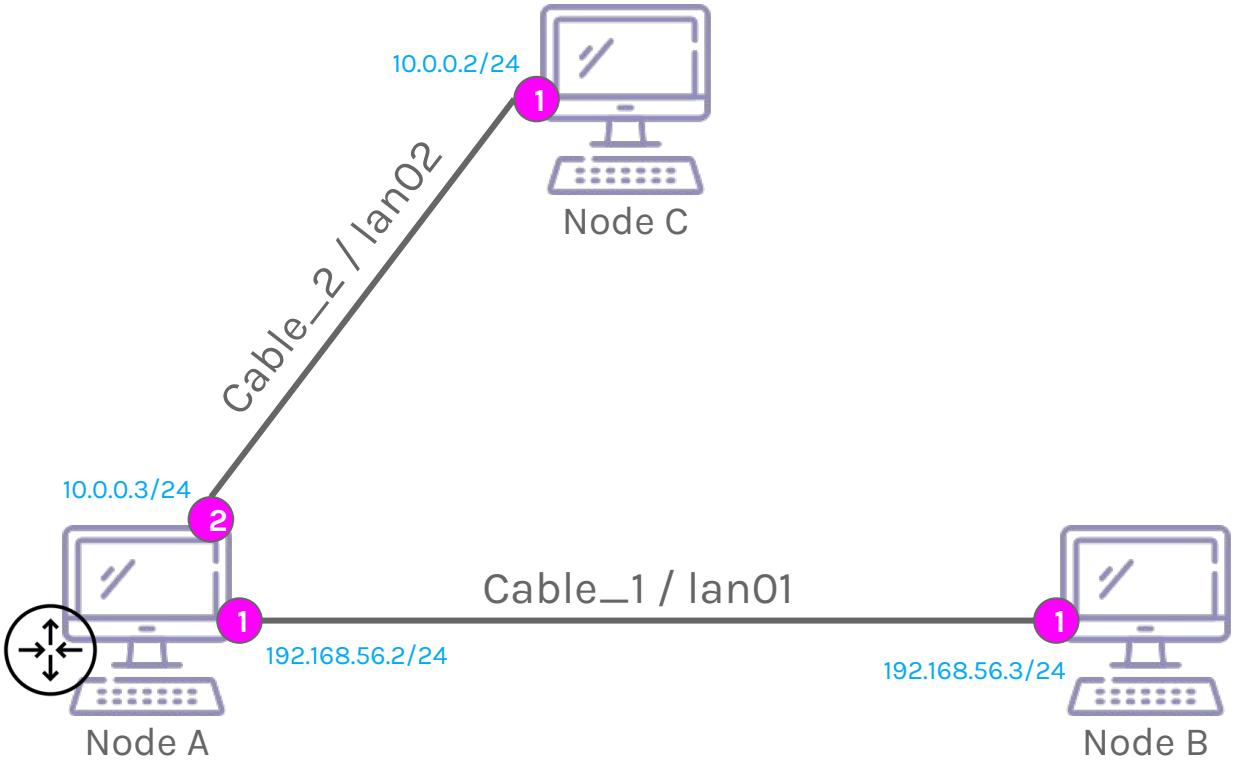
NETWORK COMMANDS

Now we are able to create a network like this one, but the network is the same, then how could connect a network to another one? We need to define a **Router** as Node.



NETWORK COMMANDS

Now suppose to want a working network like this one where Node B wants communicate with Node C through Node A, then in this case, is a Router,



NETWORK COMMANDS

We need to look at routing, then to the Routing Table management.

```
$ ip route show
```

This will print the Routing Table of the Machine, now try to add a new route:

```
# ip route add [NET ADDRESS / NETMASK] via [NODE's IP] dev [DEVICE]
```

Then the idea is to define the Static Routes into specific Nodes to route packet to a specific IP or Network VIA specific IP Node.

Considering the example propose, we want add a Static Route into Node B telling that every packet for the Node C network must pass to Node A:

```
# ip route add 10.0.0.0/24 via 192.168.56.2 dev eth0 //or
# ip route add 10.0.0.2 via 192.168.56.2 dev eth0
```

Off-course you can also delete a routing table record:

```
# ip route del [NET ADDRESS / NETMASK] via [NODE's IP] dev [DEVICE]
```

NETWORK COMMANDS

On Node A the routing table is automatically filled by the Directed Connection nodes but the communication can not work because, the node will drops every packet that is not for him.

Then, to allow the Node A to dispatch the packets to other network, based on Routing Table, we need to tell to him that it is Router.

To do this, **we need to modify a configuration file which allows the routing among different interfaces**, according to the routing table.

echo 1 > /proc/sys/net/ipv4/ip_forward

Or editing the file `/etc/sysctl.conf` adding:

```
net.ipv4.ip_forward = 1
```

And running the following command:

```
sysctl -p
```

NETWORK COMMANDS

You need to add a return route also in the Node C.

Another solution could be to add a worst case then a **default** route, then a Gateway:

ip route add default via [IP GATEWAY] dev [DEVICE]
Or using an illegal IP that could be used only in this case:

ip route add 0.0.0.0/0 via [IP GATEWAY] dev [DEVICE]

Another important thing could be the disabling of ping response by specific nodes or routers.

To do this, such as the ip forwarding, you must edit the same file and add this line:

```
net.ipv4.icmp_echo_ignore_all=1
```

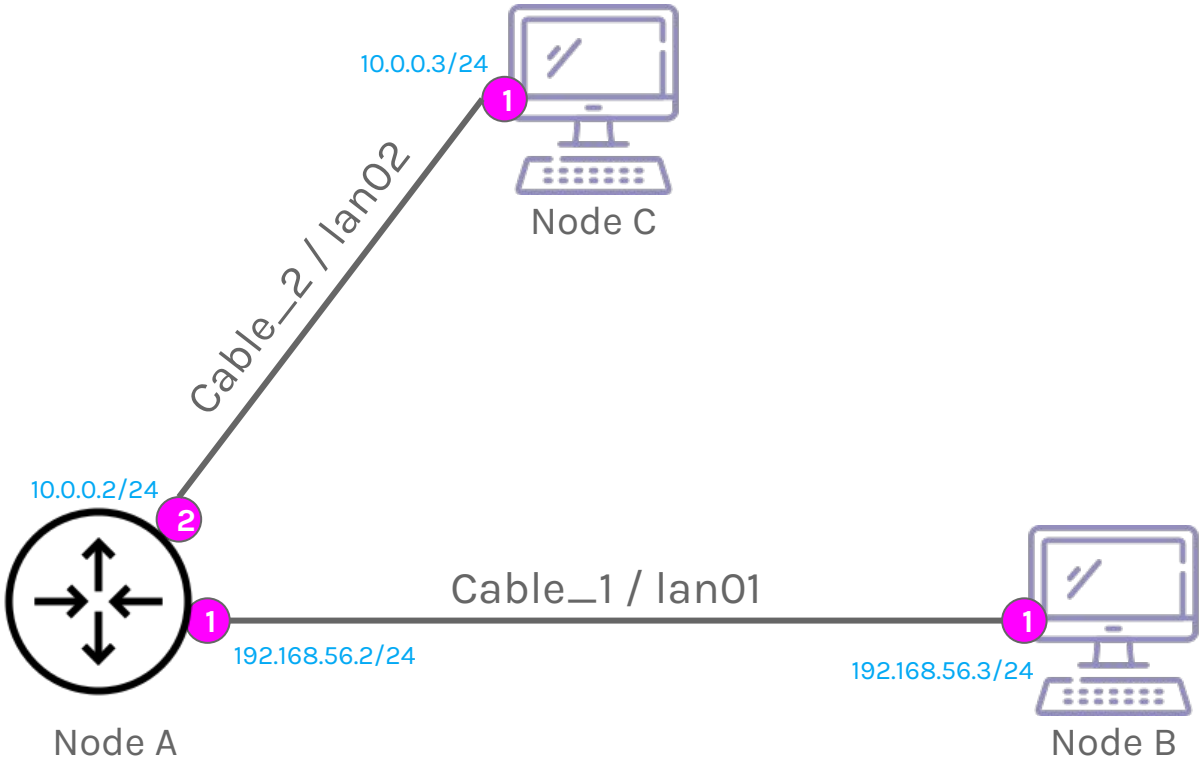
And running the following command:

```
sysctl -p
```

This could be useful to hide your nodes or routers in a network!

NETWORK COMMANDS

Finally, we are able to create Router Nodes!



NETWORK COMMANDS

A resume:

```
# ip link add [BRIDGE NAME] type bridge
# ip link set [DEVICE] master [BRIDGE NAME]
# ip link set [DEVICE] promic on

# ip route show
# ip route add [NET ADDRESS / NETMASK] via [NODE's IP] dev [DEVICE]
# ip route del [NET ADDRESS / NETMASK] via [NODE's IP] dev [DEVICE]
```

echo 1 > /proc/sys/net/ipv4/ip_forward //enable ip4 routing

NETWORK COMMANDS

What about ARP Table and MAC Table? IP command has the neigh option to manage the ARP Table fused with MAC Table:

```
$ ip neigh // show ARP/MAC table
# ip neigh flush // empty the ARP/MAC table
# ip neigh add [IP] lladdr [MAC ADDRESS] dev [DEVICE]
# ip neigh del [IP] lladdr [MAC ADDRESS] dev [DEVICE]
# ip neigh get [IP] dev [DEVICE] # ip neigh
```

CONFIGURATION FILES

All commands introduced before are temporarily then, when you restart the node, the configuration will be lost.

To maintain the configuration, you must edit the Network configuration files.

The big problem is that each Linux Distribution Family uses different type of software for the network management, then different files and structure of files.

Typically, all Debian Family (and Alpine) distribution, should use the same configuration files and syntax (anyway, any distribution has its documentation for that, believe me).

Then, how could we set the IP address of a node and save it?

CONFIGURATION FILES

Typically, the configuration file is the **/etc/network/interfaces** file.

These is the loopback configuration (do not touch it) and you can add your configurations.

For instance, you can set one of the NIC in DHCP mode:

```
auto [DEVICE]
iface [DEVICE] inet dhcp
```

Or set a static IP information (tabulation is optional):

```
auto [DEVICE]
iface [DEVICE] inet static // or iface [DEVICE]
     address [IP]      // or [IP/MASK]
     netmask [NETMASK] //
     gateway [IP]
```

Then save the file and restart the Network Service:

```
# rc-service networking restart // or service networking restart
```

CONFIGURATION FILES

To add and save specific Static Routes, you can use the same interfaces file to add them on interface up event:

Or set a static IP information:

```
auto [DEVICE]
iface [DEVICE] inet static
    ...
    up ip route add [IP/MASK or default] via [IP]
    up ip route add [IP/MASK or default] via [IP]
    down ip route del [IP/MASK or default] via [IP]
```

In the same way you can set a down commands that are executed when the NIC is put in down state. Then you can create your script and run them in these events!

Do not forget to restart the service!

```
# rc-service staticroute restart // or service staticroute restart
```

CONFIGURATION FILES

If you want use DNSs for connections, such as google.it you need to set up the IP of a DNS server (almost one). To do this you must modify the file `/etc/resolv.conf` and add, for each line, a nameserver with its ip:

```
nameserver 1.1.1.1
...
nameserver 8.8.8.8
```

Another useful thing is the modification of the **Hostname**, a human readable name of the specific Node. To do this you can modify the file **`/etc/hostname`** and put the name of your Node!

You can also add your Hostnames related to specific IPs (such as a local DNS with high priority!) into each machine, modifying the file **`/etc/hosts`**:

```
...
192.168.56.2 node_1 facebook youtube.com
```

CONNECTION COMMAND

If a Node is remotely you have to connect to it via special protocol, such as **SSH**. To be used, the Node must activate the **SSH Server Service** (SSHD, D means Daemon).

To connect to specific machine with specific user you must use the following command:

```
$ ssh [USERNAME]@[IP ADDRESS]
```

Some examples:

```
$ ssh 192.168.1.2
$ ssh fedyfausto@192.168.56.2
```

The username is optional but could be requested after the connection try.

You can also use your RSA public key to authenticate yourself!

COPY COMMAND

Finally, if you need to copy some files or software remotely in another Node, you must use **FTP** Protocol via some client (Filezilla) over SSH or use **SCP** Command:

```
$ scp [LOCAL PATH] [USERNAME]@[IP ADDRESS]:/[REMOTE PATH]
```

Or:

```
$ scp [SOURCE PATH] [DESTINATION PATH]
```

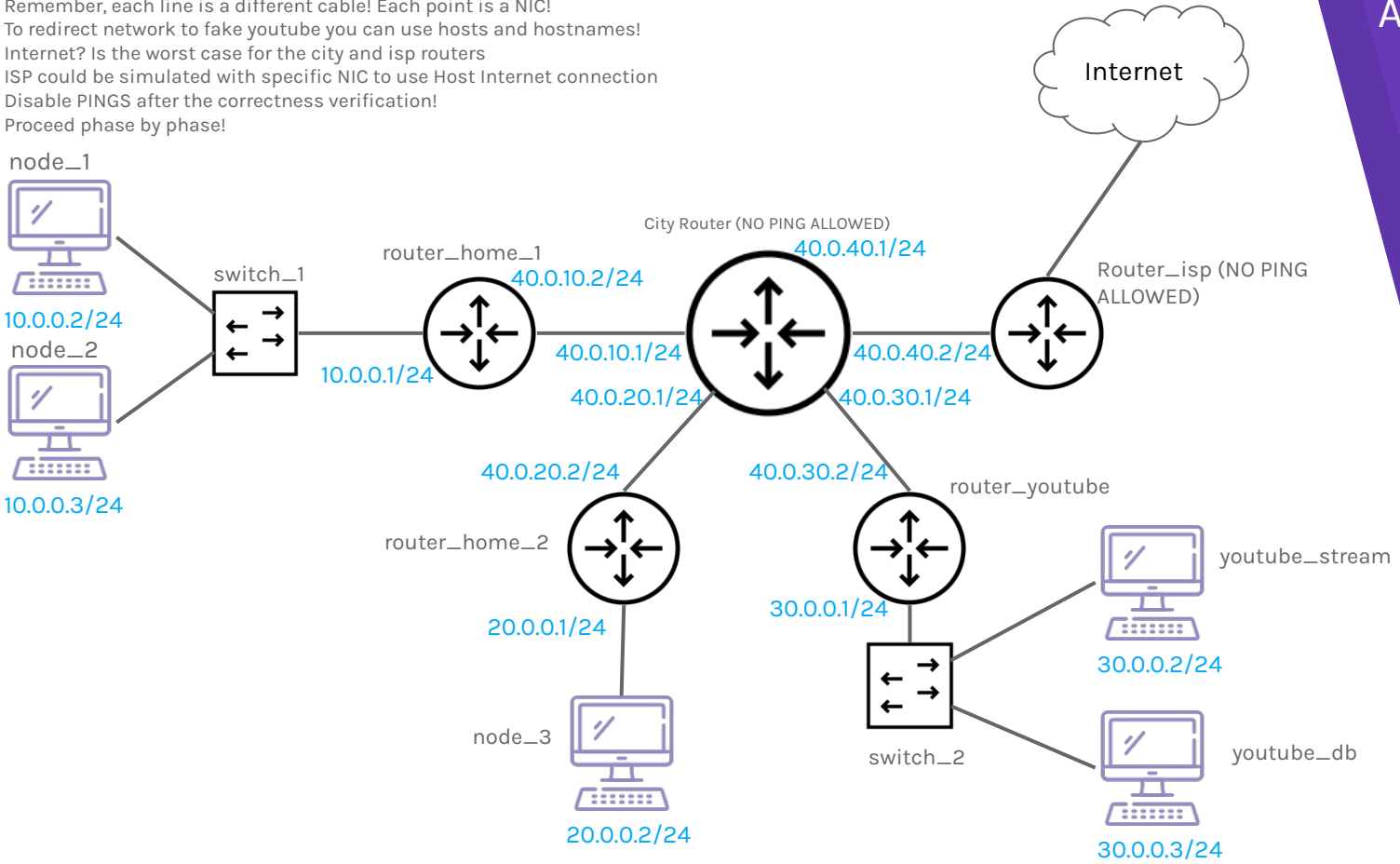
Examples:

```
$ scp server_exec usernode@192.168.1.2:/home/usernode/
```

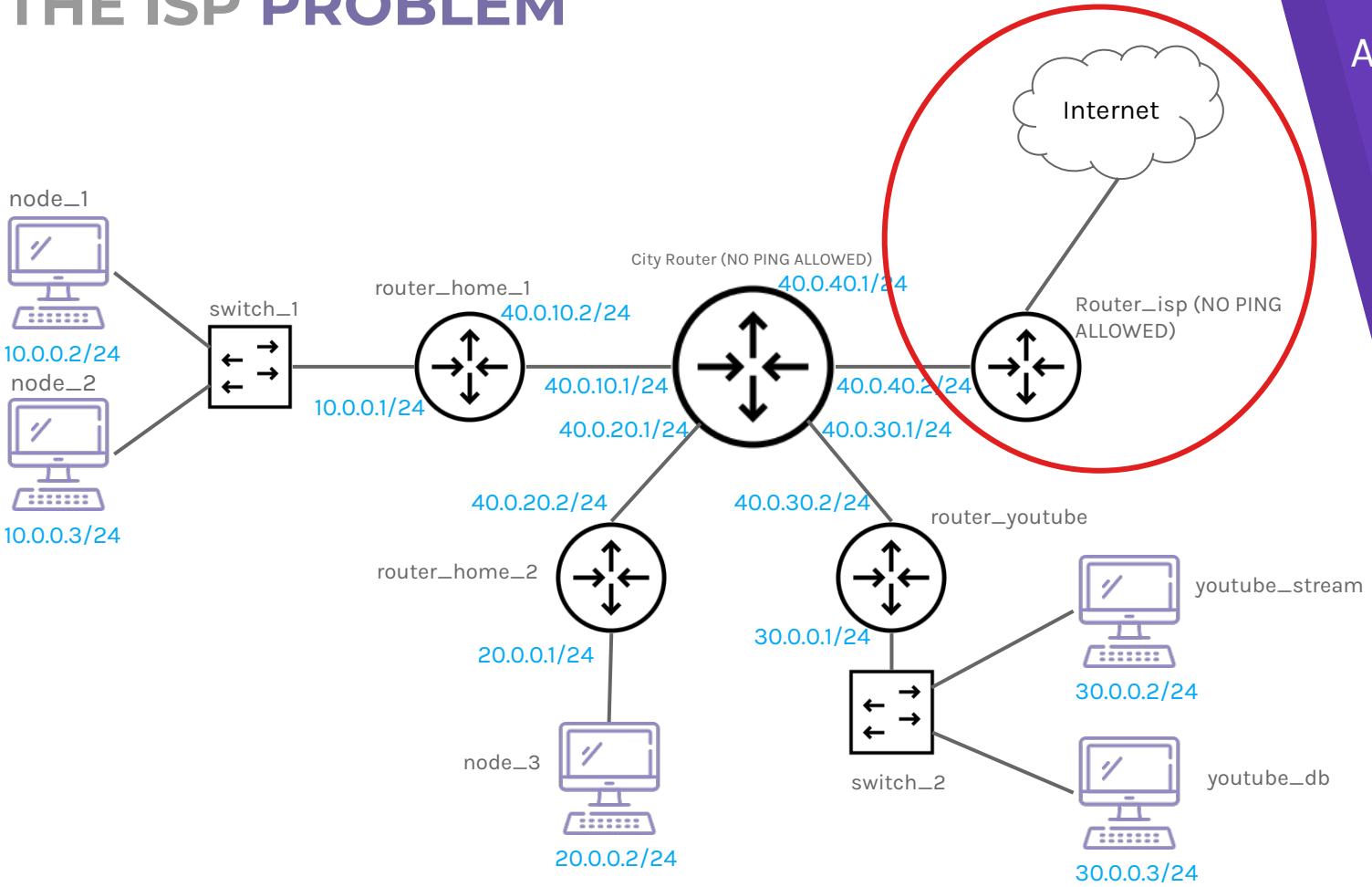
EXERCISE TIME!

REMEMBER SLIDE 21?

Remember, each line is a different cable! Each point is a NIC!
To redirect network to fake youtube you can use hosts and hostnames!
Internet? Is the worst case for the city and isp routers
ISP could be simulated with specific NIC to use Host Internet connection
Disable PINGS after the correctness verification!
Proceed phase by phase!



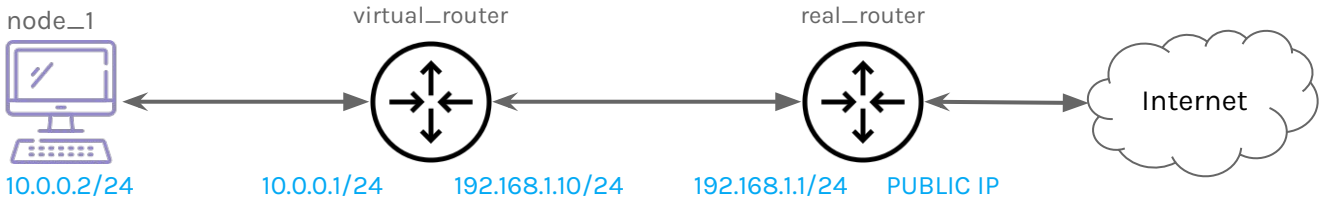
THE ISP PROBLEM



THE ISP PROBLEM

The goal is allow any node to reach Internet through ISP router via another router, which we can not manipulate and administrate!

Then we can not reach the Internet connection without add a Static Route on the external router and without it, the return traffic can not be delivered to the right network!



When node_1 sends a packet to the internet (1.1.1.1) the source IP address is 10.0.0.2. The internal network works fine to the virtual_router because these nodes are managed by us, then have static routes. When the virtual_router sends the packet to the real_router or ISP router, the source IP of the packets remains 10.0.0.2 then when a response arrives, the real_router does not know how reach the 10.0.0.2, then the packet will be dropped.

To fix this, our router needs to change the source IP with its IP and know, when receives a response, which IP exchange to send it back to the right node!

THE ISP PROBLEM

Then you might think: "*If I enable IP forwarding on node_1 and set node_1's gateway to virtual_router, should not it work?*"

YES, but only if the real_router knows how to route traffic back to node_1's subnet!

If we can not manage the real_router (for some security or other reason) our virtual_router must does a Network Address Translation (**NAT**), that must occur so that replies from the internet return to virtual_router, which then forwards them back to node_1.

Remember, enabling the ip_forward allows virtual_router to act as a router, forwarding packets between **interfaces**, but It does not rewrite packet headers!

Without NAT, the internet's replies never reach node_1!

To manage this and other mechanism, we need **iptables** tool!

THE IPTABLES

`iptables` is a command-line tool for configuring Linux kernel's `netfilter` firewall. It enables administrators to define chained rules that control incoming and outgoing network traffic.

It manages:

- **Packet Filtering** (allow/block traffic).
- **Network Address Translation** (NAT) (rewrite IP/port headers).
- Port Forwarding.
- Traffic Shaping.

iptables uses rules to determine what to do with a network packet. The utility consists of the following:

- **Tables**: Tables are files that group similar rules. A table consists of several rule chains.
- **Chains**: A chain is a string of rules. When a packet is received, iptables finds the appropriate table and filters it through the rule chain until it finds a match.
- **Rules**: A rule is a statement that defines the conditions for matching a packet, which is then sent to a target.
- **Targets**: A target is a decision of what to do with a packet. The packet is either accepted, dropped, or rejected.

THE IPTABLES - INSTALLATION

To install iptables (if not present) on Alpine linux, you can run the following command:

```
# apk add iptables
```

And then you can show the version with this:

```
# iptables --version
```

THE IPTABLES - TABLES

iptables have **five default tables** that manage different rule chains:

- **Filter**: This is the default and perhaps the most widely used table. This table contains the rules that provides the actual firewall functionality, it lets you decide whether a packet should be allowed or denied to reach its destination.
- **Network Address Translation (NAT)**: This table contains rules that allows you to route packets to different hosts on NAT networks by **changing the source and destination addresses of packets**. It is often used to allow access to services that can't be accessed directly, because they're on a NAT network.
- **Mangle**: This table contain the rules that allows you to alter packet headers and other forms of packet alteration.
- **Raw**: Bypass connection tracking (rarely used).
- **Security**: present in few linux distributions.

Tables reside at the top of the hierarchy, their function and distinguishing factor being the roles the rules they contain provide.

A table also contains multiple **chains** which are essentially a list of rules, grouped by the point in the flow of the packet. (e.g. just as they arrive on the NIC, just as they leave etc.).

THE IPTABLES - CHAINS

A **chain** is a string of rules. When a packet is received, it triggers a `netfilter` hook, which corresponds to a default chain. `iptables` finds the appropriate table, then runs it through the chain and traverses a list of rules until it finds a match.

A **rule** is a statement that tells the system what to do with a packet. Rules can block one type of packet, or forward another type of packet.

The outcome of a rule or where a packet is sent, is called a **target**.

- **PREROUTING**: Rules in this chain apply to packets as they just arrive on the network interface.
- **INPUT**: Rules in this chain apply to packets just before they're given to a local process.
- **OUTPUT**: The rules here apply to packets just after they've been produced by a process.
- **FORWARD**: The rules here apply to any packets that are routed through the current host.
- **POSTROUTING**: The rules in this chain apply to packets as they just leave the network interface.

THE IPTABLES - TARGETS

A **target** is a decision of what to do with a packet. Typically, this is to accept it, drop it, or reject it, which sends an error back to the sender.

- **ACCEPT**: This causes iptables to accept the packet.
- **DROP**: iptables drops the packet. To anyone trying to connect to your system, it would appear like the system didn't even exist.
- **REJECT**: iptables "rejects" the packet. It sends a "connection reset" packet in case of TCP, or a "destination host unreachable" packet in case of UDP or ICMP.

Table - Chain mapping:

- filter — INPUT OUTPUT FORWARD
- raw — PREROUTING OUTPUT
- nat — PREROUTING OUTPUT POSTROUTING
- mangle — PREROUTING OUTPUT INPUT FORWARD POSTROUTING

THE IPTABLES - PACKET FLOW

Routing - Inbound

When a packet first enters, it is run through the PREROUTING chains of the raw, mangle and nat tables, in that order.

- The raw table rules check if the packet is to be bypass the firewall or not.
- The mangle table rules check if the packet headers needs to be modified in any way.
- The nat table rules are consulted when a packet that creates a new connection is encountered.

After this PREROUTING processing, a decision is made whether the packet is intended for the host system or not.

If it is, it is then sent to the Host Firewall section, otherwise it is sent to the FORWARD processing area of the Routing section.

THE IPTABLES - PACKET FLOW

Host Inbound Firewall

After a packet enters this section, it is then sent to this INPUT processing section where it is run through the mangle, filter and security tables, in that order.

- The mangle table rules check if any headers or packet alteration is to be made.
- The filter table INPUT chain is where most of the commonly used firewall functionality happens. The rules in this table are what decides whether a packet is dropped or accepted before it is allowed to reach a host application.
- The security table INPUT chain implements any mandatory access control, mostly checking for SeLinux Context in data packets.
- The packet is finally received by the Host Application.

THE IPTABLES - PACKET FLOW

Host Outbound Firewall

You can also implement an outbound firewall to control what data is leaving your computer. This is done by OUTPUT chain processing where it is run through the raw, mangle, nat, filter and security tables in that order.

- The raw table OUTPUT chain rules decides whether the packet has privileges to bypass the firewall.
- The mangle table OUTPUT chain rules decides whether the packet headers needs to be modified.
- The nat table rules are consulted when a packet that creates a new connection is encountered.
- The filter table OUTPUT chain rules is where we decide whether we want to allow or deny the packet.
- The security table OUTPUT chain rules implements access control and SeLinux Control of data packets.
- It is then sent to Routing Outbound where it is decided whether the packet is routable or not.

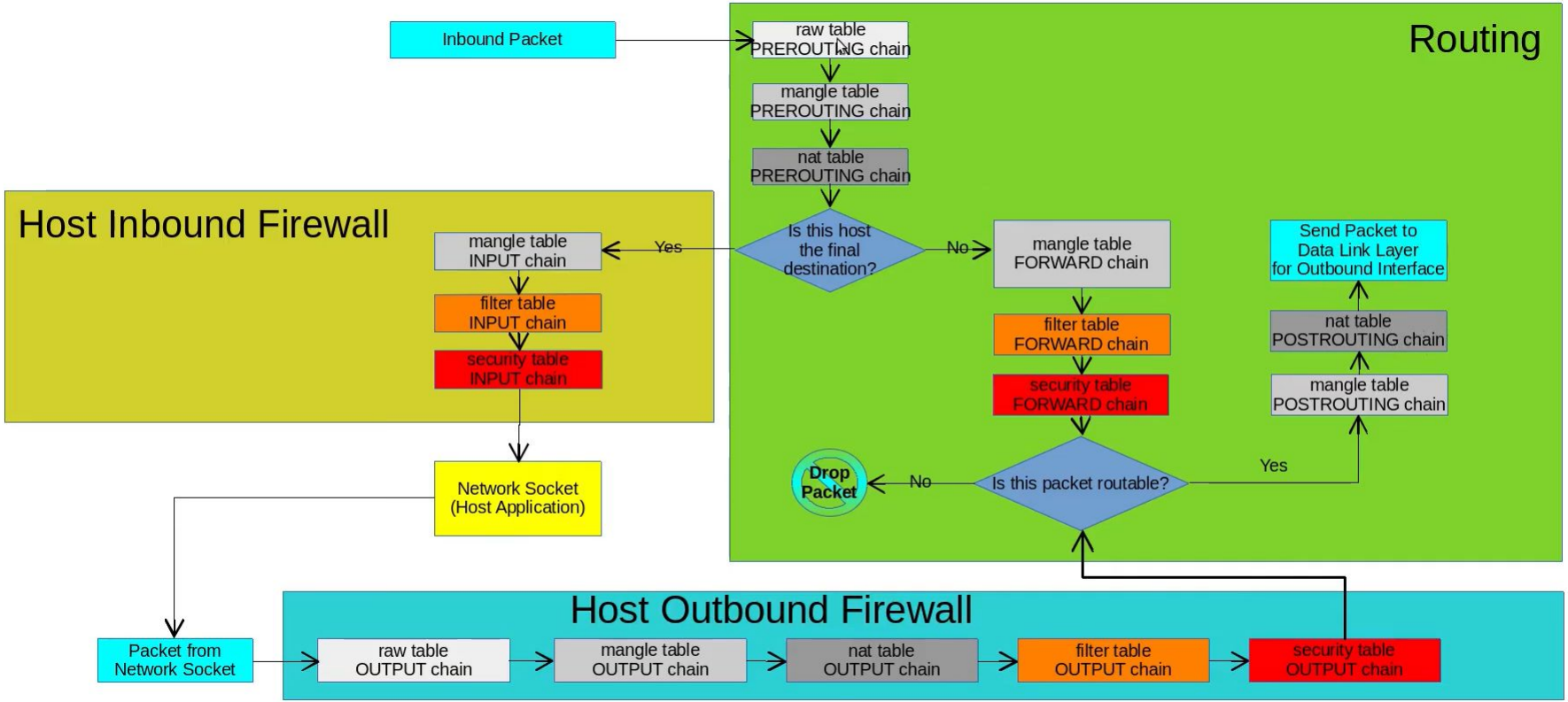
THE IPTABLES - PACKET FLOW

Routing Outbound

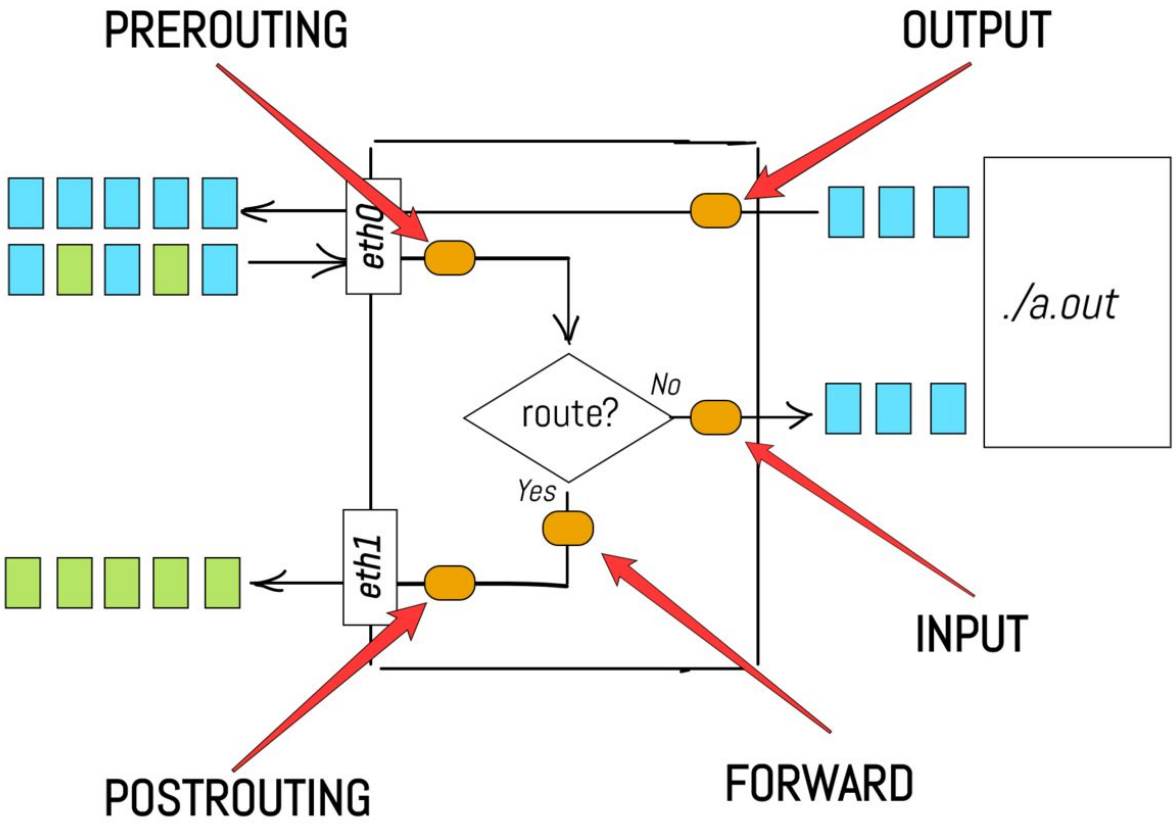
This is the area of the routing section where a packet is redirected outside the host. It is sent through the FORWARD chains of the mangle, filter and security tables, in that order.

- The mangle tables rules decide if any packet headers needs to modified before routing it out.
- The filter table rules allows us to decide whether or not we want to allow this packet to be routed.
- The security table rules decides whether the selinux or access control wants to allow this packet to be routed.
- Then it reaches this section, which is the same as step 6 in the outbound firewall, here it is first checked if this packet is routable or not.
- Its dropped if it is not routable.
- If it is routable then it is sent through POSTROUTING chains of the mangle and nat tables in that order.
- The mangle table rules decide if any packet headers needed to be modified after finding out it is routable.
- The nat table rules are consulted when a packet that creates a new connection is encountered.
- The packet is finally sent to the outbound interface.

THE IPTABLES - PACKET FLOW



THE IPTABLES - PACKET FLOW



THE IPTABLES - COMMAND

Let's take a look at the command syntax specified in iptables itself:

```
# iptables [option] [chain] [-j target]
```

Examples:

```
# iptables –A INPUT –s 192.168.0.28 –j ACCEPT
# iptables --append INPUT --source 192.168.0.28 --jump DROP
```

```
# service iptables save        # save configuration
# service iptables restart     # restart service
```

THE IPTABLES - COMMAND

Blocking IP Scenario:

```
# iptables \
--table filter \              # Use the filter table
--append INPUT \              # Append to the INPUT chain
--source 59.45.175.62 \       # This source address
--jump REJECT                 # Use the target Reject

# iptables \
--table filter \              # Use the filter table
--delete INPUT \              # Delete from the INPUT chain
--source 59.45.175.62 \       # This source address
--jump REJECT                 # Use the target Reject

# iptables \
--table filter \              # Use the filter table
--replace INPUT \             # Replace from the INPUT chain
--source 59.45.175.62 \       # This source address
--jump REJECT                 # Use the target Reject
```

THE IPTABLES - COMMAND

Blocking Port Scenario:

```
# iptables \
--append INPUT \
--protocol tcp \              # Specify TCP protocol
--match tcp \                 # Load the TCP module
--dport 22 \                  # Destination port
--source 59.45.175.0/24 \
--jump DROP

# iptables \
--append INPUT \
--protocol tcp \
--match multiport \          # Load multiport module
--dports 22,5901 \           # multiple ports
--source 59.45.175.0/24 \
--jump DROP
```

THE IPTABLES - COMMAND

Blocking ICMP Scenario:

```
# iptables \
--append INPUT \
--jump REJECT \
--protocol icmp \
--icmp-type echo-request
```

With no error message:

```
# iptables \
--append INPUT \
--protocol icmp \
--jump DROP \
--icmp-type echo-request

# iptables \
--append OUTPUT \
--protocol icmp \
--jump DROP \
--icmp-type echo-reply
```

THE IPTABLES - COMMAND

Port Redirection Scenario:

```
# iptables \
--table nat \
--append PREROUTING \
--protocol tcp \
--dport 80 \
--jump REDIRECT \
--to 8080

OR DIFFERENT NODE (Proxy)

# iptables
--table nat
--append PREROUTING
--protocol tcp
--dport 5001 \
--jump DNAT --to-destination 123.123.123.123:110

# Change sender to redirecting machine:
# iptables -t nat -A POSTROUTING -p tcp --dport 110 -j MASQUERADE
```

THE IPTABLES - COMMAND

LAN to Internet Scenario:

```
# iptables \
--table nat \
--append POSTROUTING \
-o eth1 \     # packets that leave on the second NIC
--jump MASQUERADE \    # replacing the sender's address

# iptables \
--append FORWARD \
-i eth0 \          # traffic coming from eth0
-o eth1 \     # traffic going out via eth1
--jump ACCEPT

# iptables \
--append FORWARD \
-i eth1 \          # traffic coming from eth1 (Internet)
-o eth0 \     # traffic going out via eth0 (LAN)
--match state --state RELATED,ESTABLISHED \
-j ACCEPT
# load state module to match packets that are part of existing connections
```

RECAP!

- Now we know a little bit about Virtualisation
- We can have multiple Linux computers!
- Wow we know how connect two Nodes!
- Additionally, we know how create a Switch!
- Moreover, we can create a Router and define Static Routes!
- We know which files must be modified to preserve modifications!
- We just touched and managed only DLL and IP Levels!
- Now, we are ready to touch Application and Transport Layers!

5.

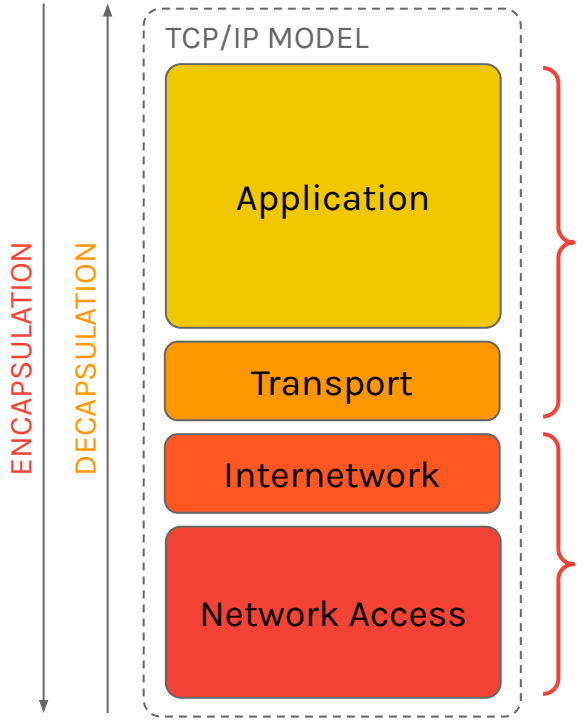
NETWORK PROGRAMMING

Protocols and Sockets!

PROF. SANTORO FEDERICO FAUSTO

UNTIL NOW

The Connectivity problems are not related to Bugs.
The Bugs are not related to the connectivity!



Then we will develop a software and choose the right Transport protocol to use for it

WHAT WE NEED?

- Basic knowledge of C/C++ programming language
- Basic knowledge of O.S., in particular Linux
- Basic knowledge of Python programming language
- Basic knowledge about how Threads work
- How manage the filesystem
- Strong theory knowledge about Transport Protocols
- A LINUX O.S.! (not really but could be useful for training)

These points are mandatory to understand and achieve the main objective, create processes which need and can communicate in some way.

WHY SOFT NEED TO COMMUNICATE?

Until now, probably, you knew the connect of communication in normal programming, in terms of Threads or Processes that need to communicate or share some information.

Typically, this can be achieved using some Shared Memory and using some protection to check the access to this resource, such as Mutex or Semaphores.

This is called **Concurrent Programming** and it must run on the same machine!

If two processes can share information via Network, then the processes can be run on different machines, this is called **Distributed Programming**!

When you use both, this is called **Concurrent and Distributed Programming**

Ok before going on, we need to know how send data and what sockets are.

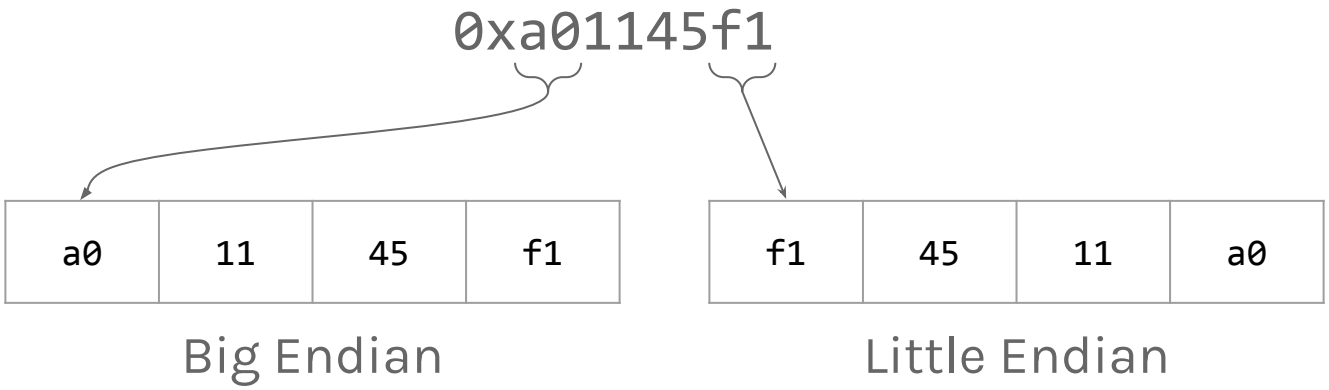
THE ENDIANNESS

The word endian-ness come from the story of Gulliver's Travels. In the story, Gulliver finds himself on the island of Lilluput whose inhabitants often quarrel over trivial things. As the story unfolds the lilliputians become divided over the matter of whether to crack boiled egg, at the little end or the big end.

Those that crack the egg at the little end become known as little endians and those that crack the egg at the big end big enfiends.

In computing we have a similar decision to make given a multi-byte piece of data, like a number! Then, how should we represent it in memory?

THE ENDIANNESS



Should we put the most significant byte first, which would be **Big Endian** or should we put the least significant byte first, which would be **Little Endian**?

Assuming that we read left to right, humans write numbers in Big Endian.

It is helpful to separate the concepts of the **value** and **representation**, theme these are representation but with same value

THE ENDIANNESS

It is important to agree on endianness because if we do not know the endianness of a representation, we do not know what value is being represented is, for instance the representation **aabb** is **0xaabb** or **0xbbaa**?

That is the why when Nodes communicate to each other over the network, they must agree on a endianness.

According to the standards, **all the fields in network headers must be Big Endian**.

Then, if a node speaks little endian, it must convert the values in the fields to little endian before interpreting them, or convert to big endian to send them.

This depends on your architecture, for instance Intel and AMD use little endian, Power PC and IBM use big endian, ARM..... BOTH (bi-endian)

THE ENDIANNESS

Try this code on your machine!

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main (){
   int x = 0x1234567;
   uint8_t buffer[256];
   memcpy(buffer, &x, sizeof(x));
   for(int i = 0; i < sizeof(x); i++){
       printf("%02hhx ", buffer[i]);
   }
   printf("\n");

}
```

THE ENDIANNESS

Remember, **endianness is about Byte ordering**, no bit ordering!

Endianness only applies to **multi-byte** values (0xff is 0xff in any case)

This has some implications that might not obvious, for instance, having an ASCII encoded string “mario” where each character is a byte, **in this case the endianness is not applicable**.

You must use the standard byte order, called **Network Byte Order**!

There are a lot of functions, such as `htonl` (host-to-network long) or `htons` (host-to-network short), respectively 32bit or 16bit.

An alternative is send data as text because text will be interpreted in the same way on all machines! (remember text based protocols?)

THE ENDIANNESS

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <arpa/inet.h>

int main (){
   int x = 0x1234567;
   uint8_t buffer[256];
   memcpy(buffer, &x, sizeof(x));
   for(int i = 0; i < sizeof(x); i++){
       printf("%02hhx ", buffer[i]);
   }
   printf("\n");

   uint32_t network_order = htonl(x);
   uint8_t buffer2[256];
   memcpy(buffer2, &network_order, sizeof(network_order));
   for(int i = 0; i < sizeof(network_order); i++){
       printf("%02hhx ", buffer2[i]);
   }
   printf("\n");

   char text[] = "Hello";
   uint8_t buffer3[256];
   memcpy(buffer3, text, sizeof(text));
   for(int i = 0; i < sizeof(text); i++){
       printf("%02hhx (%c) ", buffer3[i], buffer3[i]);
   }
   printf("\n");
}
```

NETWORK SOCKETS

Sockets are the **low level endpoint** used for processing information across a network. We can create a real connection between two Nodes!

You can image Sockets placed between Transport and Application Layer because, are used by Application to use specific protocols of Transport.

Them are called low level because the Transport protocols, like TCP or UDP, are low-level network protocols, but HTTP or FTP, which rely on Sockets, are High level network protocols.

These are interfaces that the O.S. gives that allow to perform network communication.

From the point of view of O.S., **a Socket works like a file**, where you can read or write information (be careful about concurrency!).

There are two types of sockets, **Stream** and **Datagram** Sockets.

There are also UNIX, LOCAL and other types of sockets, read the man!

NETWORK SOCKETS

Stream Sockets are used to make communication like a stream of bytes. You can put or pull data from the stream, collecting bytes in the same order that they arrived.

Keep in mind an important thing, if you send a big block o data across the network, that block will be broken up arbitrarily in a small segments, but if you use Stream Sockets, this process in transparent for the developer!

Stream Sockets are reliable that means the order of these segments or the missing of a segment, is managed by the socket it self, accordly with the used protocol.

Then a lost segment will be resent, the incorrect order of a segment will be fixed, etc.

Usually, the Transport protocol used is **TCP** that will take care of all of this.

NETWORK SOCKETS

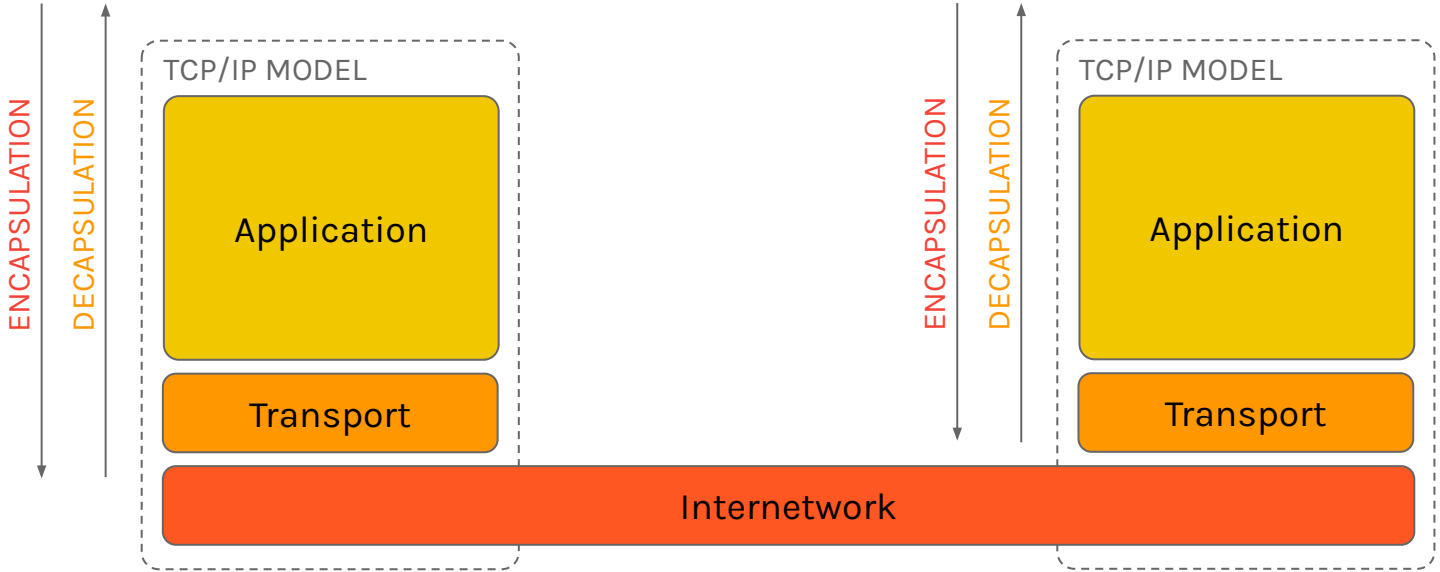
Suppose that we do not want all of these functionalities, such as resending, congestion control, reordering, for some reason, for instance, our application does not case about the ordering of the missing of some data (like video streaming) we need use Datagram Sockets.

Datagram Sockets allow sending individual Segments (or Datagrams..), without the Stream functionalities, also without congestion control or real connection between two machines.

Then if you are going to use this type of Socket, be careful in the implementation, using the best behaviour possible.

Usually, the Transport Protocol used is **UDP**.

NETWORK SOCKETS



A socket can be seen as a tunnel between processes and machines over some network, it is not really important (but it needs to work) and can be represented by a software construct as a combination of protocol, IP address and Port Address:

tcp://192.168.1.2:8080

NETWORK SOCKETS

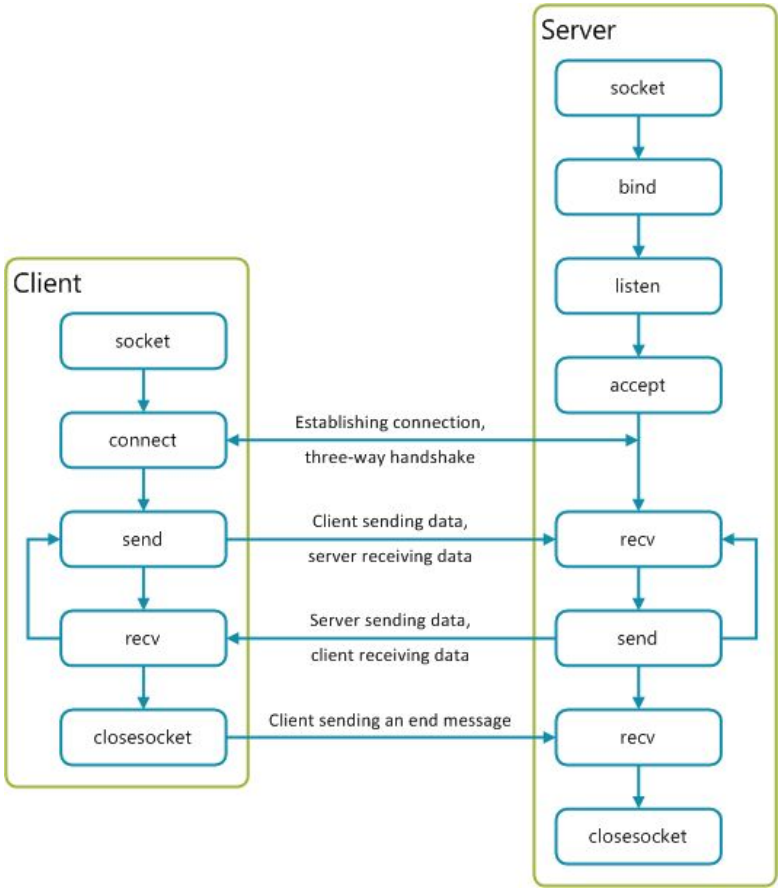
A Berkeley (**BSD**) socket is an application programming interface (API) for Internet domain sockets and Unix domain sockets, used for inter-process communication (IPC).

Berkeley sockets evolved with little modification from a de facto standard into a component of the POSIX specification. **The term POSIX sockets is essentially synonymous with Berkeley sockets**, but they are also known as BSD sockets.

All modern operating systems implement a version of the Berkeley socket interface. It became the standard interface for applications running in the Internet. Even the `Winsock` implementation for MS Windows, created by unaffiliated developers, closely follows the standard.

The BSD sockets API **is written in the C programming language**. Most other programming languages provide similar interfaces, typically written as a wrapper library based on the C API.

NETWORK SOCKETS



Function Call	Description
Socket()	To create a socket
Bind()	It's a socket identification like a telephone number to contact
Listen()	Ready to receive a connection
Connect()	Ready to act as a sender
Accept()	Confirmation, it is like accepting to receive a call from a sender
Send()	To send data
Recv()	To receive data
Close()	To close a connection

NETWORK SOCKETS

Typically, one of the ends of a socket-based data communication is a **server**, the other is a **client**. The one function used by both, clients and servers, is socket(). It is declared this way:

```
int socket(int domain, int type, int protocol);
```

The return value is of the same type as that of open, an **integer**. That is what allows sockets to be treated the same way as files.

The `domain` argument tells the system what protocol family you want it to use. Many of them exist, some are vendor specific, others are very common. They are declared in `sys/socket.h`.

Use `PF_INET` for UDP, TCP and other Internet protocols (IPv4).

Five values are defined for the type argument, again, in sys/socket.h. All of them start with "`SOCK_`". The most common one is `SOCK_STREAM`, which tells the system you are asking for a reliable stream delivery service (which is TCP when used with `PF_INET`).

If you asked for `SOCK_DGRAM`, you would be requesting a connectionless datagram delivery service (in our case, UDP).

Finally, the protocol argument depends on the previous two arguments, and is not always meaningful. In that case, use 0 for its value.

Nowhere, in the socket function have we specified to what other system we should be connected. Our newly created socket remains unconnected.

NETWORK SOCKETS

Various functions of the sockets family expect the **address** of (or pointer to, to use C terminology) a small area of the memory. The various C declarations in the `sys/socket.h` refer to it as struct sockaddr. This structure is declared in the same file:

```
/*
* Structure used by kernel to store most
* addresses.
*/
struct sockaddr {
   unsigned char  sa_len;      /* total length */
   sa_family_t    sa_family;   /* address family */
   char           sa_data[14]; /* actually longer; address value */
};
#define SOCK_MAXADDRLEN 255     /* longest possible addresses */
```

Please note the *vagueness* with which the `sa_data` field is declared, just as an array of 14 bytes, with the comment hinting there can be more than 14 of them.

This vagueness is quite deliberate. Sockets is a very powerful interface. While most people perhaps think of it as nothing more than the Internet interface-and most applications probably use it for that **nowadays-sockets can be used for just about any kind of interprocess communications**, of which the Internet (or, more precisely, IP) is only one.

NETWORK SOCKETS

The `sys/socket.h` refers to the various types of protocols sockets will handle as address families, and lists them right before the definition of `sockaddr`:

```
/*
* Address families.
*/
#define AF_UNSPEC   0       /* unspecified */
#define AF_LOCAL    1       /* local to host (pipes, portals) */
#define AF_UNIX     AF_LOCAL    /* backward compatibility */
#define AF_INET     2       /* internetwork: UDP, TCP, etc. */
#define AF_IMPLINK  3       /* arpanet imp addresses */
//....
#define AF_APPLETALK    16      /* Apple Talk */
#define AF_ROUTE    17      /* Internal Routing Protocol */
#define AF_LINK     18      /* Link layer interface */
//....
#define AF_INET6    28      /* IPv6 */
#define AF_NATM     29      /* native ATM access */
#define AF_ATM      30      /* ATM */
//....
```

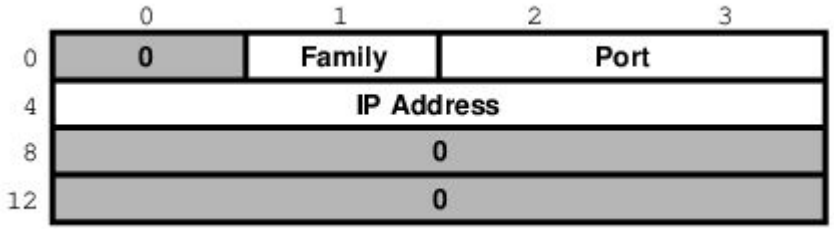
The one used for IP is `AF_INET`. It is a symbol for the constant 2.

It is the address family listed in the `sa_family` field of `sockaddr` that decides how exactly the vaguely named bytes of sa_data will be used.

NETWORK SOCKETS

Specifically, whenever the address family is AF_INET, we can use struct sockaddr_in found in netinet/in.h, wherever sockaddr is expected:

```
/*
* Socket address, internet style.
*/
struct sockaddr_in {
   uint8_t      sin_len;
   sa_family_t sin_family;
   in_port_t    sin_port;
   struct  in_addr sin_addr;
   char     sin_zero[8];
};
```



The three important fields are sin_family, which is byte 1 of the structure, sin_port, a 16-bit value found in bytes 2 and 3, and sin_addr, a 32-bit integer representation of the IP address, stored in bytes 4-7.

Let us assume we are trying to write a client for the daytime protocol, which simply states that its server will write a text string representing the current date and time to port 13. We want to use TCP/IP, so we need to specify AF_INET in the address family field. AF_INET is defined as 2. Let us use the IP address of 193.204.114.105, which is the time server of INRiM - Istituto nazionale di ricerca metereologica Italy.

NETWORK SOCKETS

By the way the sin_addr field is declared as being of the struct in_addr type, which is defined in `netinet/in.h`:

```
/*
* Internet address (a structure for historical reasons)
*/
struct in_addr {
   in_addr_t s_addr; //32 bit integer
};
```

The 193.204.114.105 is just a convenient notation of expressing a 32-bit integer by listing all of its 8-bit bytes, starting with the most significant one.

So far, we have viewed `sockaddr` as an abstraction. Our computer does not store short integers as a single 16-bit entity, but as a sequence of 2 bytes. Similarly, it stores 32-bit integers as a sequence of 4 bytes.

NETWORK SOCKETS

Suppose we coded something like this:

```
sa.sin_family      = AF_INET;
sa.sin_port        = 13;
sa.sin_addr.s_addr = (((((193 << 8) | 204) << 8) | 114) << 8) | 105;
```

What would the result look like? Depends on ENDIANESS! Remember to use the Network Byte Order!

Someone else has created the **htons** and **htonl** C functions to convert a short and long respectively from the host byte order to the network byte order, and the **ntohs** and **ntohl** C functions to go the other way.

```
sa.sin_family      = AF_INET;
sa.sin_port        = htons(13);
sa.sin_addr.s_addr = htonl((((193 << 8 | 204) << 8 | 114) << 8 | 105);
```

Or using **inet_addr** to specify the address via string :

```
sa.sin_addr.s_addr = inet_addr("193.204.114.105");
```

NETWORK SOCKETS

Once a client has created a socket, it needs to connect it to a specific port on a remote system. It uses **connect**:

```
int connect(int s, const struct sockaddr *name, socklen_t namelen);
```

The **s** argument is the socket, i.e., the value returned by the socket function. The **name** is a pointer to sockaddr, the structure we have talked about extensively. Finally, **namelen** informs the system how many bytes are in our sockaddr structure.

If connect is successful, it returns 0. Otherwise it returns -1 and stores the error code in errno. There are many reasons why connect may fail. For example, with an attempt to an Internet connection, the IP address may not exist, or it may be down, or just too busy, or it may not have a server listening at the specified port. Or it may outright refuse any request for specific code.

NETWORK SOCKETS - FIRST CLIENT

```c
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>

int main() {
 int s, numBytes;
 struct sockaddr_in sa;
 char buffer[BUFSIZ+1];  // max buffer for I/O operations for the system + null termination

 if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {
   perror("socket");
   return 1;
 }

 memset(&sa, '\0', sizeof(sa));

 sa.sin_family = AF_INET;
 sa.sin_port = htons(13);
 sa.sin_addr.s_addr = inet_addr("193.204.114.105");
 if (connect(s, (struct sockaddr *)&sa, sizeof sa) < 0) {
   perror("connect");
   close(s);
   return 2;
 }

 while ((numBytes = read(s, buffer, BUFSIZ)) > 0)
   write(1, buffer, numBytes); // file descriptor 1 is the stdout

 close(s);
 return 0;
}
```

Compile with:
`cc test.c -o test`
or
`gcc test.c -o test`

NETWORK SOCKETS - SERVER

The typical server does not initiate the connection. Instead, it waits for a client to call it and request services. It does not know when the client will call, nor how many clients will call. It may be just sitting there, waiting patiently, one moment,
The next moment, it can find itself swamped with requests from a number of clients, all calling in at the same time.

There are **65535** IP ports, but a server usually processes requests that come in on only one of them. It is like telling the phone room operator that we are now at work and available to answer the phone at a specific extension. We use **bind** to tell sockets which port we want to serve.

```
int bind(int s, const struct sockaddr *addr, socklen_t addrlen);
```

Beside specifying the port in addr, the server may include its IP address. However, it can just use the symbolic constant **INADDR_ANY** to indicate it will serve all requests to the specified port regardless of what its IP address is. This symbol, along with several similar ones, is declared in **netinet/in.h**

```
#define INADDR_ANY      (u_int32_t)0x00000000
```

NETWORK SOCKETS - SERVER

The server ensures the listen on that port with the **listen** function.

```
int listen(int s, int backlog);
```

In here, the **backlog** variable tells sockets how many incoming requests to accept while you are busy processing the last request. In other words, it determines the maximum size of the queue of pending connections.

You have now established a connection with your client. This connection remains active until either you or your client hang up. The server accepts the connection by using the **accept** function.

```
int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

Note that this time **addrlen** is a pointer. This is necessary because in this case it is the socket that fills out addr, the **sockaddr_in** structure. The return value is an integer. Indeed, the accept returns a new socket. You will use this new socket to communicate with the client.
What happens to the old socket? It continues to listen for more requests (remember the backlog variable we passed to listen?) until we close it.

NETWORK SOCKETS - FIRST SERVER

```c
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define ever (;;)
#define BACKLOG 4 //max clients in queue
#define PORT 8080

int create_server(uint16_t port){

   int socketServerFD;
   struct sockaddr_in addr;
   socklen_t addr_len = sizeof(addr);
   if ((socketServerFD= socket(AF_INET, SOCK_STREAM, 0)) < 0) {
       perror("socket failed");
       return -1;
   }

   memset(&addr, 0, sizeof(addr));
   addr.sin_family = AF_INET;
   addr.sin_port = htons(port);
   addr.sin_addr.s_addr = htonl(INADDR_ANY);

   // Bind and listen
   if (bind(socketServerFD, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
       perror("bind failed");
       return -2;
   }

   if (listen(socketServerFD, BACKLOG) < 0) {
       perror("listen failed");
       return -3;
   }

   return socketServerFD;
}
```

NETWORK SOCKETS - FIRST SERVER

```c
int main() {

    int server_fd, client_fd;
    struct sockaddr_in addr;
    socklen_t addr_len = sizeof(addr);
    server_fd = create_server(PORT);
    if (server_fd < 0 ) exit(1);
    printf("Server running on port %d\n", PORT);

    for ever {
        // Accept new connection
        if ((client_fd = accept(server_fd, (struct sockaddr*)&addr, &addr_len)) < 0) {
            perror("accept failed");
            continue;
        }
        // Fork a child process to handle client
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork failed");
            close(client_fd);
        }
        else if (pid == 0) {  // Child process (client)
            close(server_fd);  // Child doesn't need listener

            // Get current time
            time_t now = time(NULL);
            struct tm *tm_info = gmtime(&now);
            char buffer[1024];

            // Format time string
            strftime(buffer, sizeof(buffer), "%Y-%m-%dT%H:%M:%SZ\n", tm_info);

            // Send response
            send(client_fd, buffer, strlen(buffer), 0);

            // Cleanup and exit
            close(client_fd);
            _exit(0);
        }
        else {  // Server process
            close(client_fd);  // Server doesn't need client socket
        }
    }

    close(server_fd);
    return 0;
}
```

NETWORK SOCKETS - MESSAGING

Ok ok.. Let is start from the basics! Assume that we want create a simple messaging software, with which users can communicate together. Start with the client!

We want create a software that requires the IP (or hostname) and PORT of main Server. To do this, we need to check the arguments!

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, char* argv[]) {

   if(argc <3){
       //remember, argv[0] is the same program!
       fprintf(stderr, "Usage: %s <IP> <Port>\n", argv[0]);
       exit(1);
   }

}
```

NETWORK SOCKETS - MESSAGING

Assuming that we are going to write into the main code (except for the defines), we need to define and create our socket! We need a buffer to save the data that we want send and receive. The buffer size is related to the data that we want send.

```
#define BUFFER_SIZE 1024

// save the socket file descriptor
   int sockfd;
   struct sockaddr_in serv_addr;
   char buffer[BUFFER_SIZE];

   // create socket (ip, using TCP)
   if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
       perror("socket creation failed");
       exit(1);
   }
```

NETWORK SOCKETS - MESSAGING

Ok now we need to fill the fields of the sockaddr_in structure! Remember that we can use the IP address as string!

```
// configure server address
   // reset all fields
   memset(&serv_addr, 0, sizeof(serv_addr));
   serv_addr.sin_family = AF_INET;
   serv_addr.sin_port = htons(atoi(argv[2]));

   // convert IP address from text to binary form
   if (inet_pton(AF_INET, argv[1], &serv_addr.sin_addr) <= 0) {
       perror("invalid address");
       exit(1);
   }
```

NETWORK SOCKETS - MESSAGING

Finally, we can try to connect to the server and start our infinite loop!

```
// connect to server
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("connection failed");
        exit(1);
    }

    printf("Connected to %s:%s\n", argv[1], argv[2]);
    printf("Type messages (enter 'exit' to quit):\n");

    for(;;){
        // main loop

    }
```

NETWORK SOCKETS - MESSAGING

Ok now we want get messages from user, then we get it from standard input, managing a special message (exit) to break the main loop.

```
// flush the stdout
 printf("> ");
 fflush(stdout);

 // read input from user
 if (!fgets(buffer, BUFFER_SIZE, stdin)) {
     break;  // exit on EOF (Ctrl+D)
 }

 // remove newline and check for exit command
 buffer[strcspn(buffer, "\n")] = '\0';
 if (strcmp(buffer, "exit") == 0) {
     break;
 }
```

NETWORK SOCKETS - MESSAGING

Ok out data is ready to be sent to the server, then send the data and prepare our code to receive some response!

```
// send message to server
if (send(sockfd, buffer, strlen(buffer), 0) < 0) {
    perror("send failed");
    break;
}

// receive response from server
int bytes_received = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);
if (bytes_received < 0) {
    perror("recv failed");
    break;
} else if (bytes_received == 0) {
    printf("Server closed connection\n");
    break;
}
```

NETWORK SOCKETS - MESSAGING

The client code is complete! But there is a problem, the client can send but only receive only one message after each send, because the receive is a blocking operation! We can use fork or threads to manage this situation

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 1024

int sockfd;  // global socket descriptor for thread access
```

NETWORK SOCKETS - MESSAGING

We need to define the thread functions to manage the receive and send.

```
void* receive_handler(void* arg) {
    char buffer[BUFFER_SIZE];
    for(;;) {
        ssize_t bytes = recv(sockfd, buffer, BUFFER_SIZE-1, 0);
        if(bytes <= 0) {
            if(bytes == 0)
                printf("\nServer closed connection\n");
            else
                perror("Receive error");
            exit(1);
        }
        buffer[bytes] = '\0';
        printf("\nReceived: %s\n> ", buffer);
        fflush(stdout);
    }
    return NULL;
}
```

NETWORK SOCKETS - MESSAGING

We need to define the thread functions to manage the receive and send.

```
void* send_handler(void* arg) {
    char buffer[BUFFER_SIZE];
    for(;;) {
        printf("> ");
        fflush(stdout);

        if(!fgets(buffer, BUFFER_SIZE, stdin)) break;

        buffer[strcspn(buffer, "\n")] = '\0';
        if(strcmp(buffer, "exit") == 0) {
            send(sockfd, buffer, strlen(buffer), 0);
            close(sockfd);
            exit(0);
        }

        if(send(sockfd, buffer, strlen(buffer), 0) < 0) {
            perror("Send failed");
            exit(1);
        }
    }
    return NULL;
}
```

NETWORK SOCKETS - MESSAGING

Finally, we can use the same main code to init the socket and connect and then start the threads and wait the termination of them.

```c
int main(int argc, char *argv[]) {

    // ... init the socket and connect as before
    printf("Connected to %s:%s\n", argv[1], argv[2]);

    pthread_t send_thread, recv_thread;
    pthread_create(&recv_thread, NULL, receive_handler, NULL);
    pthread_create(&send_thread, NULL, send_handler, NULL);

    pthread_join(send_thread, NULL);
    pthread_join(recv_thread, NULL);

    close(sockfd);
    return 0;
}
```

NETWORK SOCKETS - MESSAGING

The Client TCP section is ready to send and receive simultaneously messages from Server using threads! Now we need to develop the Server section.
The Server needs to manage multiple clients and broadcast messages from a client to the others.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX_CLIENTS 10
#define BUFFER_SIZE 1024
```

NETWORK SOCKETS - MESSAGING

We need (in C) to define a structure to save a “client information” and a list of these. The list is a critical section because a client can add itself or delete itself during connection and disconnection, then we need the usage of mutex!

```
typedef struct {
    int sockfd;
    struct sockaddr_in address;
} client_info;

// shared data structure with mutex
typedef struct {
    client_info *clients[MAX_CLIENTS];
    int count;
    pthread_mutex_t mutex;
} client_list;

client_list clients = { .count = 0 };
```

NETWORK SOCKETS - MESSAGING

Now define a function to broadcast a message from a specific client. To do this, we need to iterate the client list, check if the client is not the sender and then, send the message to the connected client socket.

There we need the mutex because some client could add or delete itself while the broadcasting!

```
void broadcast_message(const char *message, int sender_fd) {
    pthread_mutex_lock(&clients.mutex);

    for(int i = 0; i < clients.count; i++) {
        if(clients.clients[i]->sockfd != sender_fd) {
            send(clients.clients[i]->sockfd, message, strlen(message), 0);
        }
    }

    pthread_mutex_unlock(&clients.mutex);
}
```

NETWORK SOCKETS - MESSAGING

We need to define the thread function to manage each client. This function will print the client information, add the client into the client list, manages the messages from the client and manage the delete of the client when it disconnects. The function will be named `handle_client`

```
void *handle_client(void *arg) {
    client_info *info = (client_info *)arg;
    int client_sock = info->sockfd;
    char buffer[BUFFER_SIZE];
    char client_ip[INET_ADDRSTRLEN];

    inet_ntop(AF_INET, &info->address.sin_addr, client_ip, INET_ADDRSTRLEN);
    int client_port = ntohs(info->address.sin_port);

    printf("New connection from %s:%d\n", client_ip, client_port);

    // Add client to list
    pthread_mutex_lock(&clients.mutex);
    clients.clients[clients.count++] = info;
    pthread_mutex_unlock(&clients.mutex);
```

NETWORK SOCKETS - MESSAGING

We need to define the thread function to manage each client. This function will print the client information, add the client into the client list, manages the messages from the client and manage the delete of the client when it disconnects. The function will be named `handle_client`

```
void *handle_client(void *arg) {
    // code….
    for(;;) {
        ssize_t bytes_received = recv(client_sock, buffer, BUFFER_SIZE - 1, 0);
        if(bytes_received <= 0) {
            if(bytes_received == 0)
                printf("Client %s:%d disconnected\n", client_ip, client_port);
            else perror("recv failed");
            break;
        }

        buffer[bytes_received] = '\0';

        // format message: [IP:Port] message
        char formatted_message[BUFFER_SIZE + INET_ADDRSTRLEN];
        snprintf(formatted_message, sizeof(formatted_message),"[%s:%d] %s", client_ip, client_port, buffer);
        printf("%s", formatted_message);
        broadcast_message(formatted_message, client_sock);

        if(strcmp(buffer, "exit\n") == 0) {
            break;
        }
    }
```

NETWORK SOCKETS - MESSAGING

We need to define the thread function to manage each client. This function will print the client information, add the client into the client list, manages the messages from the client and manage the delete of the client when it disconnects. The function will be named `handle_client`

```
void *handle_client(void *arg) {
    // code….
// remove client from list
    pthread_mutex_lock(&clients.mutex);
    for(int i = 0; i < clients.count; i++) {
        if(clients.clients[i]->sockfd == client_sock) {
            free(clients.clients[i]);
            for(int j = i; j < clients.count - 1; j++) {
                clients.clients[j] = clients.clients[j + 1];
            }
            clients.count--;
            break;
        }
    }
    pthread_mutex_unlock(&clients.mutex);

    close(client_sock);
    return NULL;
}
```

NETWORK SOCKETS - MESSAGING

Now the main function of our server. In the main we need to create the Server Socket and initialise the mutex of clients list

```
int main() {
    int server_fd;
    struct sockaddr_in address;

    // initialize mutex
    pthread_mutex_init (&clients.mutex, NULL);

    // create server socket
    if((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(1);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // bind socket
    if(bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(1);
    }

    // listen for connections
    if(listen(server_fd, MAX_CLIENTS) < 0) {
        perror("listen");
        exit(1);
    }
    printf("Chat server listening on port  %d...\n", PORT);
```

NETWORK SOCKETS - MESSAGING

Finally, the main loop must manage new clients sockets connection and start, for each one, a new thread.

```
    printf("Chat server listening on port  %d...\n", PORT);

    for(;;) {
        client_info *client = malloc(sizeof(client_info));
        socklen_t addrlen = sizeof(client->address);

        // accept new connection
        if((client->sockfd = accept(server_fd, (struct sockaddr *)&client->address, &addrlen)) < 0) {
            perror("accept");
            free(client);
            continue;
        }

        // Create thread for new client
        pthread_t thread_id;
        if(pthread_create(&thread_id, NULL, handle_client, (void *)client) != 0) {
            perror("pthread_create");
            close(client->sockfd);
            free(client);
        } else {
            pthread_detach(thread_id);
        }
    }

    close(server_fd);
    pthread_mutex_destroy(&clients.mutex);
    return 0;
}
```

NETWORK STRING PROBLEM

We learned that sending Strings as messages is easy and we can define, as string, a protocol message.

```
// Client sends
const char *msg = "Hello Server!";
send(sockfd, msg, strlen(msg), 0);

// Server receives
char buffer[1024];
recv(sockfd, buffer, sizeof(buffer), 0);
printf("Received: %s\n", buffer);
```

It is easy to implement because there is not complex data formatting, is Human-Readable and is flexible, then you can use them with any text-based protocol. There are also a few drawbacks, in fact there are no metadata (sender, timestamp, type), it is not type safety and the parsing is fragile.

NETWORK STRUCTURE SOLUTION

```c
#pragma pack(push, 1)    // disable the architecture padding
typedef struct {
    char sender[32];     // fixed-size sender name
    char message[256];   // fixed-size message
    uint8_t msg_type;    // 0=text, 1=file, 2=command
    uint32_t timestamp; // unix timestamp
} ChatMessage;           // total 302 bytes

// ... other messages structures

#pragma pack(pop)        // re-enable the architecture padding
```

Considering that the client and server have the same structure, now we have a structured message that define our simple application protocol!

To use this, we need **serialise** and **deserialise** this message!

NETWORK STRUCTURE SOLUTION

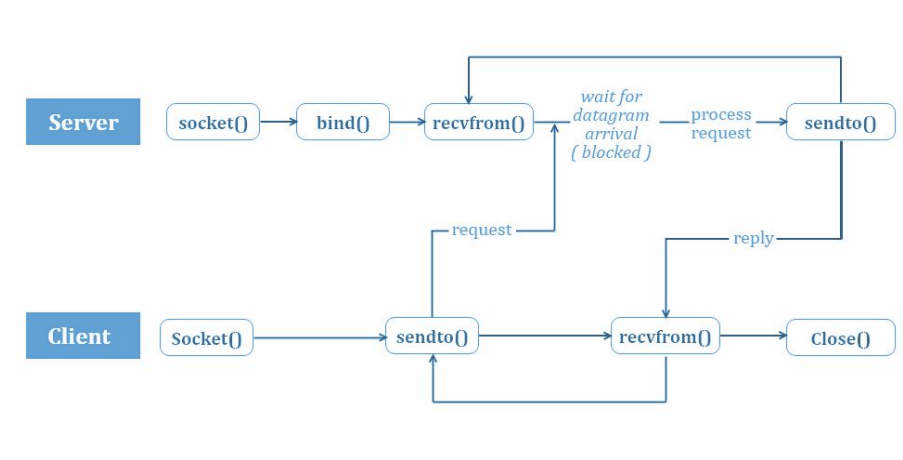
```
// client sends
ChatMessage msg;

// serialise
strncpy(msg.sender, "Alice", sizeof(msg.sender));
strncpy(msg.message, "Hello World!", sizeof(msg.message));
msg.msg_type = 0;
msg.timestamp = time(NULL);

// send the message
send(sockfd, &msg, sizeof(msg), 0);

// server receives
ChatMessage received_msg;

// deserielise directly into the empty structure
recv(sockfd, &received_msg, sizeof(received_msg), 0);

printf("[%s][%lu] %s\n", received_msg.sender, received_msg.timestamp,
received_msg.message);
```

NETWORK UDP DATAGRAM

In UDP, the client does not form a connection with the server like in TCP and instead sends a datagram. Similarly, the server need not accept a connection and just waits for datagrams to arrive. Datagrams upon arrival contain the address of the sender which the server uses to send data to the correct client.

We can call a function called `connect()` in UDP but it does not result anything like it does in TCP. There is no 3 way handshake. It just checks for any immediate errors and store the peer's IP address and port number.

`connect()` is storing peers address so no need to pass server address and server address length arguments in `sendto()`.



NETWORK UDP DATAGRAM

As in TCP, you need to create and bind (in the server) a socket using `socket` and `bind` function as normal but specifying the Datagram type of socket.

```c
int create_server(uint16_t port){

    int socketServerFD;
    struct sockaddr_in addr;
    socklen_t addr_len = sizeof(addr);
    if ((socketServerFD = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket failed");
        return -1;
    }

    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    // Bind and listen
    if (bind(socketServerFD, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind failed");
        return -2;
    }
    return socketServerFD;
}
```

As you can see, there is not a `listen` function, because we are in a connectionless context!.

NETWORK UDP DATAGRAM

Another important difference is how we can send and receive data, because there is not connection between process, then we need to define, each time, the destination of our messages!

To do this, we need to use new functions **sendto** and **recvfrom**.

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen)

ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen)
```

Then when we need to send data, we need to specify, with classic `sockaddr` struct the destination IP and PORT.

When we receive data, if we need this information, we can pass the pointer to a `sockaddr` structure where the `recvfrom` function will save the IP and PORT of sender.

NETWORK UDP MESSAGING

To try the UDP communication, we can simulate a messaging service. First of all, we need to define our simple messaging protocol:

```c
#include <stdint.h>

#define NICKNAME_LEN 32
#define MSG_LEN 256

#pragma pack(push, 1)
typedef struct {
    uint8_t command;  // 0=normal, 1=register, 2=exit
    char nickname[NICKNAME_LEN];
    char message[MSG_LEN];
} chat_message;
#pragma pack(pop)
```

NETWORK UDP MESSAGING - SERVER

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#include "protocol.h"

#define PORT 8080
#define MAX_CLIENTS 10

typedef struct {
   struct sockaddr_in addr;
   char nickname[NICKNAME_LEN];
} client_info;

client_info clients[MAX_CLIENTS];
int client_count = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void broadcast_message(chat_message *msg, struct sockaddr_in *sender) {
   pthread_mutex_lock(&mutex);

   for(int i = 0; i < client_count; i++) {
       // Skip sender and compare addresses
       if(memcmp(&clients[i].addr, sender, sizeof(struct sockaddr_in)) != 0) {
           int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
           sendto(sockfd, msg, sizeof(chat_message), 0,
                  (struct sockaddr*)&clients[i].addr, sizeof(clients[i].addr));
           close(sockfd);
       }
   }

   pthread_mutex_unlock(&mutex);
}
```

NETWORK UDP MESSAGING - SERVER

```c
int main() {
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    chat_message msg;

    // create UDP socket
    if((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket failed");
        return 1;
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if(bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        return 2;
    }

    printf("UDP Chat Server running on port %d\n", PORT);

    while(1) {
        // Receive message
        ssize_t bytes = recvfrom(sockfd, &msg, sizeof(msg), 0,
                                 (struct sockaddr*)&client_addr, &addr_len);
        if(bytes != sizeof(msg)) continue;  // Discard partial messages

        // Handle registration
        if(msg.command == 1) {  // REGISTER command
            pthread_mutex_lock(&mutex);
            // Check if already registered
            int exists = 0;
            for(int i = 0; i < client_count; i++) {
                if(memcmp(&clients[i].addr, &client_addr, sizeof(client_addr)) == 0) {
                    exists = 1;
                    break;
                }
            }
            // Register new client
            if(!exists && client_count < MAX_CLIENTS) {
                clients[client_count].addr = client_addr;
                strncpy(clients[client_count].nickname, msg.nickname, NICKNAME_LEN);
                printf("%s joined from %s:%d\n", msg.nickname,
                       inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
                client_count++;
            }
            pthread_mutex_unlock(&mutex);
        }
        // Handle exit
        else if(msg.command == 2) {  // EXIT command
            pthread_mutex_lock(&mutex);
            for(int i = 0; i < client_count; i++) {
                if(memcmp(&clients[i].addr, &client_addr, sizeof(client_addr)) == 0) {
                    printf("%s left\n", clients[i].nickname);
                    // Remove by shifting array
                    for(int j = i; j < client_count-1; j++) {
                        clients[j] = clients[j+1];
                    }
                    client_count--;
                    break;
                }
            }
            pthread_mutex_unlock(&mutex);
        }
        // Broadcast normal messages
        else {
            printf("Broadcasting: [%s] %s\n", msg.nickname, msg.message);
            broadcast_message(&msg, &client_addr);
        }
    }

    close(sockfd);
    return 0;
}
```

NETWORK UDP MESSAGING - CLIENT

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>
#include "protocol.h"

#define SERVER_IP "127.0.0.1"
#define SERVER_PORT 8080

int sockfd;
struct sockaddr_in server_addr;
char nickname[NICKNAME_LEN];

void *receive_handler(void *arg) {
    chat_message msg;
    struct sockaddr_in from_addr;
    socklen_t addr_len = sizeof(from_addr);

    while(1) {
        ssize_t bytes = recvfrom(sockfd, &msg, sizeof(msg), 0,
                                 (struct sockaddr*)&from_addr, &addr_len);
        if(bytes == sizeof(msg)) {
            printf("\n[%s] %s\n> ", msg.nickname, msg.message);
            fflush(stdout);
        }
    }
    return NULL;
}
```

```c
void send_message(const char *text, uint8_t cmd) {
    chat_message msg;
    memset(&msg, 0, sizeof(msg));
    strncpy(msg.nickname, nickname, NICKNAME_LEN);
    strncpy(msg.message, text, MSG_LEN);
    msg.command = cmd;

    sendto(sockfd, &msg, sizeof(msg), 0,
           (struct sockaddr*)&server_addr, sizeof(server_addr));
}
```

NETWORK UDP MESSAGING - CLIENT

```c
int main() {
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if(sockfd < 0) {
        perror("socket failed");
        return 1;
    }
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    printf("Enter nickname: ");
    fgets(nickname, NICKNAME_LEN, stdin);
    nickname[strcspn(nickname, "\n")] = '\0';

    // Register with server
    send_message("", 1);

    // Start receive thread
    pthread_t recv_thread;
    pthread_create(&recv_thread, NULL, receive_handler, NULL);

    printf("Connected! Type messages (/exit to quit)\n> ");
    while(1) {
        char input[MSG_LEN];
        fgets(input, MSG_LEN, stdin);
        input[strcspn(input, "\n")] = '\0';

        if(strcmp(input, "/exit") == 0) {
            send_message("", 2);  // Send exit command
            break;
        }

        send_message(input, 0);  // Send normal message
    }

    close(sockfd);
    return 0;
}
```

RECAP!

- We touched the Application Layer!
- We learned how use Transport Layer
- How the TCP Streams works
- How the UDP Datagrams works
- How define a messaging protocol
- How user Structs
- How serialise (pack)
- How deserialise (unpack)
- How processes communicate via network protocols
- But can different technologies communicate each other?

6.

INTERPROCESS COMMUNICATION

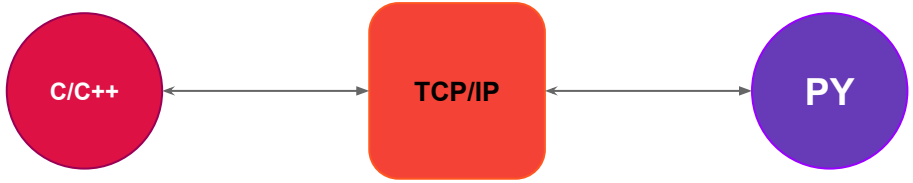
Java, Python, C, Rust, etc.

PROF. SANTORO FEDERICO FAUSTO

INTRODUCTION TO INT. COMM.

We learned how the sockets work using the most low level language that we could use since now.

Of course there are a lot of other low level languages, such us Rust, Golang, Assembly... but C/C++ is a good start (and we can stop there!)



Remember, we talked about Client and Server, about Sender and Receiver, but these are Processes, Softwares! Then we learned how two Application Software can communicate using the TCP/IP Stack, as a consequence, we can create two different software using two different technologies!

INTRODUCTION TO INT. COMM.

To try this we can use ANY LANGUAGE or technology that support the TCP/IP stack for communication, then the BSD Sockets.

In this section we will introduce, as we learned in C, the BSD Sockets programming using Python Language

The choice is made considering the difficulty to learn the Python language, but you can try to do the same operations using any language that you want.

If you did not install Python on your machine, you can follow the instructions on official website:

https://www.python.org/about/gettingstarted/

From there you can also study the basics of the language!

INTRODUCTION TO PYTHON SOCKETS

First of all we will try to create a simple TCP Client/Server in Python. To use python socket connection, we need to import socket module. Then, sequentially we need to perform the same tasks to establish connection between server and client than we seen before.

```
import socket

def client_program():
    host = "127.0.0.1"
    port = 8080

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # instantiate
    client_socket.connect((host, port))  # connect to the server

    message = input("> ")  # take input

    while massage.lower().strip() != 'exit':
        client_socket.send(message.encode())  # send message
        data = client_socket.recv(1024).decode()  # receive response

        print('Received from server: ' + data)  # show in terminal

        message = input("> ")  # again take input

    client_socket.close()  # close the connection

if __name__ == '__main__':
    client_program()
```

INTRODUCTION TO PYTHON SOCKETS

PROF. SANTORO FEDERICO FAUSTO

INT. COMM.

```python
import socket

MAX_CLIENTS = 24

def server_program():

    host = "0.0.0.0"
    port = 8080
    server_socket = socket.socket()  # get instance
    # look closely. The bind() function takes tuple as argument
    server_socket.bind((host, port))

    # configure how many client the server can listen simultaneously
    server_socket.listen(MAX_CLIENTS)
    # conn is the client socket file descriptor
    conn, address = server_socket.accept()  # accept new connection
    print("Connection from: " + str(address))

    while True:
        # receive data stream. it won't accept data packet greater than 1024 bytes
        data = conn.recv(1024).decode()
        if not data:
            # if data is not received break
            break
        print("from connected user: " + str(data))
        data = "ok"
        conn.send(data.encode())  # send data to the client

    conn.close()  # close the connection


if __name__ == '__main__':
    server_program()
```

INTRODUCTION TO PYTHON SOCKETS

A server can utilise threads to handle multiple clients simultaneously as we seen before, using the threading module. This approach allows the server to process multiple client connections concurrently, enhancing its responsiveness and ability to handle a large number of clients efficiently.

```
import socket
import threading

MAX_CLIENTS = 24

def handle_client(client_socket):
    # receive data stream.
    data = client_socket.recv(1024).decode()
    if not data:
        # if data is not received break
        client_socket.close()
        return
    print("from connected user: " + str(data))
    data = "ok"
    client_socket.send(data.encode())
    client_socket.close()
```

```
def server_program():

    host = "0.0.0.0"
    port = 8080
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((host, port))

    # configure how many client the server can listen simultaneously
    server_socket.listen(MAX_CLIENTS)

    while True:
        # The server continuously listens for incoming client connections.
        client_socket, address = server_socket.accept()
        print("Connection from: " + str(address))
        # When a new client connects, a new thread is created to handle the client.
        client_thread = threading.Thread(target=handle_client, args=(client_socket,))
        client_thread.start()

    conn.close()  # close the connection

if __name__ == '__main__':
    server_program()
```

INTRODUCTION TO PYTHON SOCKETS

Like we seen before, we can also define our protocol using structs. In python there is the `struct` module with which we can pack and unpack structs between processes!

Python struct module can be used in handling binary data stored in files, database or from network connections etc.

It uses format Strings as compact descriptions of the layout of the C structs and the intended conversion to/from Python values.

There are five important functions in struct module: `pack()`, `unpack()`, `calcsize()`, `pack_into()` and `unpack_from()`. In all these functions, we have to provide the format of the data to be converted into binary, like:

```
STRUCT_FORMAT = "!H I B f B B"
```

For all types of format, you can read the complete list from here:

https://docs.python.org/3.7/library/struct.html#format-characters

INTRODUCTION TO PYTHON SOCKETS

As we said before, three of most important functions in struct module are `pack()`, `unpack()` and `calcsize()`. These are useful and needed to be used to transform data to binary, binary to data and calculate the possibile size of struct:

```
pack(format, v1, v2, ...)    # convert values → bytes (binary) using format string
unpack(format, buffer)       # convert bytes → Python values using format string
calcsize(format)             # calculate size of packed data (in bytes)
```

To define the string format we need to know the byte order and the field types:

[Byte Order] [Format Characters]

Character	Name	Byte Order	Example
@	Native	Native	Platform-dependent
<	Little-endian	Intel x86	`b'\x34\x12'`
>	Big-endian	Network order	`b'\x12\x34'`
!	**Network**	**Big-endian**	**Same as >**

INTRODUCTION TO PYTHON SOCKETS

Let is take a look on a Python Sender and C Receiver:

```c
// Server C
#pragma pack(push, 1)
typedef struct {
    uint16_t ID;
    uint32_t timestamp;
    uint8_t riccobene;
    float temperature;
    uint8_t humidity;
    uint8_t quality;
} SensorRead;
#pragma pack(pop)

void handle_packet(SensorRead *data) {
    // Convert network to host byte order
    data->ID = ntohs(data->ID);
    data->timestamp = htonl(data->timestamp);

    // convert float (special handling)
    uint32_t temp_net;
    memcpy(&temp_net, &data->temperature, 4);
    temp_net = ntohl(temp_net);
    memcpy(&data->temperature, &temp_net, 4);

    printf("Temperature: %.2f\n", data->temperature);
}
```

```python
# Python Client

import socket
import struct

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# network order uint16 uint32 uint8 float uint8 uint8
format_str = "!H I B f B B"  # matches SensorRead struct

data = struct.pack(format_str,
    1234,              # ID
    1689424245,        # timestamp
    42,                # riccobene
    23.5,              # temperature
    65,                # humidity
    95)                # quality

sock.sendto(data, ("192.168.1.2", 8080))
```

INTRODUCTION TO PYTHON SOCKETS

PROF. SANTORO FEDERICO FAUSTO
INT. COMM.

Or a Python Server and C Client...

```python
# Server Python

# define network byte order format for SensorRead struct
STRUCT_FORMAT = "!H I B f B B"  # 2+4+1+4+1+1 = 13 bytes

def start_server(host='0.0.0.0', port=8080):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((host, port))

    while True:
        try:
            # receive exactly 17 bytes
            data, addr = sock.recvfrom(13)
            if len(data) != 13:
                print(f"Invalid size: {len(data)} bytes")
                continue

            # unpack using network byte order (unpack returns
tuples)
            sensor_id, timestamp, riccobene, temp, humidity,
quality = struct.unpack(STRUCT_FORMAT, data)

            print(f"ID: {sensor_id}")
            print(f"Temperature: {temp:.2f}°C")

        except Exception as e:
            print(f"🔥 Error: {str(e)}")
```

```c
// Client C
#pragma pack(push, 1)
typedef struct {
    uint16_t ID;              // 2 bytes
    uint32_t last_timestamp;  // 4 bytes
    uint8_t riccobene;          // 1 byte
    float temperature;        // 4 bytes
    uint8_t humidity;         // 1 byte
    uint8_t quality;          // 1 byte
} SensorRead;                  // Total: 13 bytes
#pragma pack(pop)

int main() {
    // create socket….
    // Prepare sensor data
    SensorRead data = {
        .ID = 1234,
        .last_timestamp = (uint64_t)time(NULL),
        .riccobene = 1,
        .temperature = 23.5f,
        .humidity = 65,
        .quality = 95
    };
    // convert to network byte order
    data.ID = htons(data.ID);
    data.last_timestamp = htonl(data.last_timestamp);
    // convert float to network byte order
    uint32_t temp_net;
    memcpy(&temp_net, &data.temperature, sizeof(float));
    temp_net = htonl(temp_net);
    memcpy(&data.temperature, &temp_net, sizeof(float));
    // send data
    sendto(sockfd, &data, sizeof(data), 0, (struct sockaddr*)&server_addr,
sizeof(server_addr));
```

RECAP!

- We introduce Python
- We learned Python's Sockets
- How serialise and deserialise data
- How create two processes that can communicate
- Now you can study this mechanism with other languages!
- Now you are ready to do anything!

7.

EXERCISES

GO GOOOOOO!

EXERCISES : INFORMATION

Going through the following exercises, it is strictly advised to try to develop the various software parts in several different languages.

In addition, it is always advisable to try the deployment of such software solutions in **a network infrastructure**, even virtualized.

These exercises are only for self-assessment of your learning and are not at all similar to an exam.

EXERCISE 1 : SENSORS

Develop a Software solution that allows a **Sensor Node** to send some sensor data to a special **Central Node.** When a Central Node receives data from sensor, if a specific condition happens, the Central Node will send an Alarm to a **Control Node**.

Sensor Node:

- Can send data every 3 seconds
- Communication could not be reliable
- Sensor data could be Temperature, Humidity and Air Quality
- Every sensor must be identified by a unique ID

Central Node:

- Will store new Sensor when receives a message from it
- If the temperature is higher than 30 or air quality is poor, it must sends a message to Control Node
- The communication must be reliable

Control Node:

- Will print the Alarms
- Will store a file LOG of all alarms

EXERCISE 2 : ROCK, PAPER, SCISSORS

Create a Network Game about Rock, Paper and Scissors.

Player:

- Player can connect to a server
- Player must register it self using email and password
- Player must login to be able to do actions
- A Player can search another one to play
- Then a player can stay in WAITING status (this is a choice of the player)
- When an opponent will be found by server, the game starts
- A game is a set of 3 rounds

Game Server:

- Game server will manages the registration and the authentication of the players.
- Game server must manages the matchmaking to create the games for two players
- The matchmaking can be design as taking the first two players that are waiting or randomly from the waiting players
- The server is the authority to check the game status and the moves by the players
- When a game finishes, the players return in the lobby status (no in waiting)

EXERCISE 3 : SENSORS RELIABLE

As the first one but with some differences:

Sensor Node:

- Can send data every 3 seconds
- Communication must be reliable
- Sensor data could be Temperature, Humidity and Air Quality
- Every sensor must be identified by a unique ID
- If the temperature is higher than 30 or air quality is poor, it must enter in Alarm Mode
- In alarm Mode can not send sensor data
- When receives a command to stop the alarm mode, it will re-enter in normal mode

Central Node:

- Will store new Sensor when receives a message from it
- If the temperature is higher than 30 or air quality is poor, it must sends a message to Control Node
- The communication must be reliable
- If the Control Node sends a stop alarm command to specific node, the Central node sends this command to the specific Sensor Node

Control Node:

- Will print the Alarms
- Will store a file LOG of all alarms
- After 5 seconds, if a Sensor Node enters in Alarm mode, will stop it

EXERCISE 4 : CHAT SERVICE

Create a Chat Service where users can register and login into the chat service and communicate to specific users.

Users:

- Can register using email and password
- Can send a message to specific user (if connected)

Main Server:

- Manages the registration and authentication
- Manage the communication system between users connected
- The users information must be saved into a database (a file)
- When startup, the service must reload the database (the file)

EXERCISE 5 : CHAT SERVICE P2P

Create a Chat Service where users can register and login into the chat service and communicate to specific users via P2P communication.

Users:

- Can register using email and password
- Can send a message to specific user (if connected)
- To communicate to other users, the client must use a not reliable communication
- To do this, clients will publish to the server its special communication port

Main Server:

- Manages the registration and authentication
- The users information must be saved into a database (a file)
- When startup, the service must reload the database (the file)
- The server exchange the client information to allow them to communication in P2P mode, without passing through the server

COMPUTER NETWORKS COURSE ENDS